

باسمه تعالی

طراحی شتاب دهنده سخت افزاری شبکه عصبی برای محاسبه لایه ضرب
ماتریس ها (MAC Array)

چکیده	۴
-------------	---

فصل اول: مقدمه و آشنایی با معماری Systolic Array

۱-۱ مقدمه و ضرورت استفاده از شتاب‌دهنده‌های سخت‌افزاری در شبکه‌های عصبی عمیق (DNN)	۵
۱-۲ آشنایی با ماهیت عملیات پایه‌ای در پردازش شبکه‌های عصبی عملگر MAC	۵
۱-۳ معرفی تئوری معماری آرایه سیستولیک (Systolic Array) و تاریخچه آن	۶
۱-۴ اهداف پروژه، مشخصات فازهای طراحی و مستندسازی سیستم مورد مطالعه	۸

فصل دوم: مروری بر معماری‌های مختلف برای پیاده‌سازی Systolic Array

۲-۱ بررسی منطق نظری آرایه‌های سیستولیک (H.T. Kung – 1982)	۱۲
۲-۲ طبقه‌بندی استراتژی‌های جریان‌داده (Dataflow) در شبکه‌های سخت‌افزاری	۱۴
۲-۳ مقایسه حالت Output-Stationary با حالت Weight-Stationary و بررسی عملکرد در شبکه‌ها	۱۶
۲-۴ مکانیزم پخش سیگنال کلاک (Clock Distribution) در المان‌های متصل متقارن	۱۸

فصل سوم: روش بررسی معماری کلی سیستم پیاده شده

۳-۱ معرفی ماژول سطح بالای کنترل‌کننده	۲۰
۳-۲ تشریح سیگنال‌ها، پورت‌ها و پین‌های دقیق رابط کاربری	۲۱
۳-۳ پارتیشن‌بندی میان المان هسته محاسباتی، رابط حافظه، واحد رجیسترهای انتقال خروجی و واحد کنترل اصلی	۲۳

فصل چهارم: بررسی معماری بلوک‌های پایه: ضرب‌کننده و جمع‌کننده

۴-۱ ضرورت طراحی تفکیک‌یافته مدارهای سنتزپذیر ضرب و جمع	۲۵
۴-۲ تحلیل مهندسی معماری جمع‌کننده (Adder Architecture)	۲۵
۴-۲-۱ مقایسه ساختار و تاخیر مدارهای جمع‌کننده RCA در برابر CLA	۲۶
۴-۲-۲ پیاده‌سازی توصیف سخت‌افزاری و اثبات دلایل مهندسی برای استفاده از ساختار CLA	۲۷
۴-۳ تحلیل مهندسی معماری ضرب‌کننده (Multiplier Architecture)	۲۹
۴-۳-۱ مقایسه ساختار ضرب‌کننده آرایه‌ای (Array) نسبت به درخت ضرب والاس (Wallace Tree)	۲۹
۴-۳-۲ استدلال سازگاری بالا برای انتخاب ساختار درخت ضرب والاس	۳۱

فصل پنجم: معماری عنصر پردازشی (PE) و تحلیل ابعاد داده

۵-۱ فلوچارت انتقال مسیر موج‌داده و اتصال‌دهی بلوک‌های محاسبه داخل عنصر محاسباتی	۳۳
۵-۲ بررسی الزامات سخت‌گیرانه فرمت ابعاد متغیرها برای عملیات MAC	۳۴
۵-۳ استدلال نظری و ریاضی محدودیت‌های عددی، بیت‌ها و جلوگیری از پدیده سرریز	۳۵
۵-۳-۱ تحلیل اثبات کفایت رجیستر ۱۶-بیت علامت‌دار برای حفظ دقت ضرب دو مقدار ۸ بیت	۳۶
۵-۳-۲ تعمیم رجیستر خروجی (Sign-Extension) به رجیستر Accumulator ۱۸-بیت علامت‌دار	۳۶
۵-۴ بررسی لچ کردن سیگنال داده و زمان‌بندی چرخه به چرخه در مسیر (Pipelining)	۳۷

فصل ششم: طراحی ماشین حالت (کنترلر سیستم FSM) و مدیریت تاخیرات

- ۶-۱ ساختار روند انتقال منطق حالت برای ماشین حالت اصلی کنترل کننده سیستم ۳۸
- ۶-۲ تشریح حالت‌های پیاده سازی شده در ماشین حالت ۴۲
- ۶-۳ معرفی سیستم ردگیری تاخیرها برای اثبات کارایی پایپ لاین سیستم ۴۵
- ۶-۴ مدل سازی تاخیر پردازشی اولیه مدار تا ظهور اولین پاسخ بر حسب شمارش پالس کلاک ۴۶
- ۶-۵ استنتاج محاسباتی مدل توان عملیاتی متوالی (Throughput Report Computation) ۴۸

فصل هفتم: رابط ورودی و خروجی، ارتباط با حافظه و پارامتری کردن ابعاد

- ۷-۱ مدار طراحی شده جهت پاسخ به استاندارد سازی برگرداندن آدرس‌ها به خروجی ۵۰
- ۷-۲ فرآیند همسان سازی مدارهای حافظه‌ای MEM_IN و MEM_WB ۵۱
- ۷-۳ Generate block ها و استفاده اصولی از Generics (پارامتر متغیر M, N) جهت گسترش مقیاس پذیری سخت افزاری ۵۳
- ۷-۴ تحلیل عملیات تغییر رفتار سائز سیستم و وابسته شدن چرخه‌های زمان به مقدار ورودی پارامتر m ۵۵
- ۷-۵ چگونگی پیاده سازی مفهوم سیستم نرونی لایه اتصال-کامل بوسیله مقدار دهی ($1 = M$) ۵۶

فصل هشتم: تحلیل شبیه سازی موجی و Testbench

- ۸-۱ معرفی فایل Testbench و بردارهای تست اعمالی ۵۸
- ۸-۲ گزارش تأخیر سیستم (Latency) و توان عملیاتی (Throughput) ۶۲
- مراجع و منابع ۶۳

چکیده

این گزارش به تشریح مراحل طراحی، توسعه و ارزیابی شبیه‌سازی یک شتاب‌دهنده سخت‌افزاری برای شبکه‌های عصبی عمیق مبتنی بر معماری آرایه سیستولیک^۱ می‌پردازد. هدف اصلی این مدار، محاسبه موازی و پرسرعت ضرب ماتریس‌ها به همراه جمع بایاس با رابطه پایه $D = A \times B + C$ می‌باشد. با توجه به اهمیت دقت و جلوگیری از خطای سرریز^۲ در عملیات‌های جمع، سیستم برای تمامی ورودی‌ها (داده‌های ماتریس اصلی، وزن‌ها و بایاس‌ها) طول ۸ بیت علامت‌دار را دریافت کرده و پس از انجام عملیات ضرب ۱۶ بیتی، نتایج نهایی هر المان را در ساختار ۱۸ بیتی علامت‌دار با دقت کامل پردازش و تجمیع می‌کند. به منظور ارتقاء کارایی مدار و کاهش زمان بحرانی تأخیر^۳ نسبت به معماری‌های سنتی، طراحی عناصر پردازشی^۴ با رویکردی نوآورانه در سطح منطقی انجام شده است؛ به گونه‌ای که برای بلوک پایه ضرب‌کننده، به جای ضرب‌کننده‌های آرایه‌ای مرسوم که تاخیری از مرتبه $O(n)$ دارند، از معماری ساختاریافته درخت والاس^۵ بهره گرفته شده است. همچنین به عنوان بلوک جمع‌کننده^۶، از جمع‌کننده‌های ۱۸ بیتی پیش‌بین نقلی^۷ استفاده شده است. ادغام این دو قطعه پردازشی سبب شد تا زنجیره تأخیر از میان رفته و سرعت محاسبه بلوک‌ها به فرم لگاریتمی بهینه شود. یکی دیگر از ابتکارات پیاده‌سازی این معماری، تکنیک بارگذاری اولیه ماتریس C است؛ به این مفهوم که مقدار انباشتگر هر سلول PE پیش از آغاز ضرب، مستقیماً توسط دیتای شبکه بایاس بارگذاری می‌شود تا به عملیات مستقل جمع نیازی نباشد.

مدیریت همگام‌سازی ورودی‌ها و استخراج دقیق نتایج بر عهده‌ی یک واحد کنترل‌گر ماشین حالت (FSM) است. این کنترلر از پنج وضعیت متوالی IDLE, LOAD, COMPUTE, WRITEBACK و DONE برای ایجاد شیفت‌های دقیق زمانی در حین وارد شدن مقادیر ورودی و ایجاد تطابق بین ماتریس‌های A و B استفاده می‌کند. این مدیریت صحیح باعث شده که خروجی‌های محاسباتی بتوانند بلافاصله پس از محاسبه کامل، یکی‌پس از دیگری آدرس‌دهی و اعتبارسنجی شوند. در نهایت، از منظر مقیاس‌پذیری و پوشش دادن بخش پارامتریک پروژه، ماژول کنترل‌گر شبکه به صورت ساختار پارامتریک مقیاس‌پذیر تنظیم شده و تنها با تغییر دو متغیر پایه‌ای m و n می‌تواند هندسه فیزیکی بین پردازنده‌های برداری را متناسب با مقیاس‌های متفاوت 1×4 یا مقیاس کامل 4×4 بدون بروز مشکل، به صورت یکپارچه مجتمع نماید. آزمایش‌های اعتبارسنجی مدار اثبات می‌کند که سیستم توصیف شده، عملکرد مطلوبی دارد.

¹ Systolic Array

² Overflow

³ Latency Delay

⁴ Processing Element (PE)

⁵ Wallace Tree

⁶ Accumulator

⁷ Carry Look-ahead Adder (CLA)

فصل اول: مقدمه و آشنایی با معماری Systolic Array

۱-۱) مقدمه و ضرورت استفاده از شتاب‌دهنده‌های سخت‌افزاری در شبکه‌های عصبی عمیق (DNN)

امروزه هوش مصنوعی و یادگیری ماشین به بخش جدایی‌ناپذیری از کاربردهای مختلفی مانند پردازش تصویر، پردازش زبان طبیعی، تشخیص گفتار، سیستم‌های خودران و... تبدیل شده‌اند. در هسته اصلی تمامی این پردازش‌های هوشمند، شبکه‌های عصبی عمیق^۱ (DNN) قرار دارند. این شبکه‌ها بر اساس اتصالات متراکم میان چندین لایه از نورون‌ها بنا شده‌اند و وظیفه اصلی آن‌ها اجرای محاسبات بسیار بزرگ بر روی داده‌هاست. مهم‌ترین عملیات پایه در پردازش این شبکه‌ها، عملیات ضرب ماتریس‌ها است. در حقیقت، چه در مرحله آموزش مدل و چه در مرحله نتیجه‌گیری و کاربرد آن، شبکه‌های عصبی عمیق نیازمند اجرای میلیاردها عملیات پایه ضرب و جمع یا همان MAC^۲ هستند. همین امر باعث شده تا ضرب ماتریسی به یکی از پرهزینه‌ترین محاسبات در حوزه هوش مصنوعی، چه از نظر حجم پردازشی و چه از نظر مصرف انرژی تبدیل شود.

به دلیل همین حجم فوق‌العاده سنگین محاسباتی، پردازنده‌های معمولی نظیر CPU های کلاسیک، توانایی ارائه عملکرد مطلوب برای پردازش شبکه‌های عصبی عمیق را ندارند؛ چرا که اجرای این الگوریتم‌ها بر روی پردازنده‌های سنتی، موجب افزایش شدید تاخیر^۳ و مصرف بسیار بالای توان می‌شود. این محدودیت، به‌ویژه در کاربردهایی که با کمبود فضا و منابع انرژی روبه‌رو هستند، بسیار جدی‌تر است. به عنوان مثال در دستگاه‌های لبه^۴ مانند تجهیزات اینترنت اشیا و موبایل‌ها، بهینه‌سازی مصرف انرژی از بالاترین سطح اهمیت برخوردار است. به همین علت، استفاده از شتاب‌دهنده‌های سخت‌افزاری کاملاً ضروری و اجتناب‌ناپذیر است. شتاب‌دهنده‌ها سیستم‌های سخت‌افزاری ویژه‌ای هستند که برای تسریع دسته‌ای خاص از محاسبات (مانند ضرب‌های پیوسته ماتریسی) طراحی می‌شوند. در سال‌های اخیر توسعه سخت‌افزارهای با کاربرد خاص، از جمله واحد پردازش تنسور^۵ شرکت گوگل، نشان داده است که می‌توان با استفاده از این شتاب‌دهنده‌ها سرعت اجرای حجم وسیع محاسبات در شبکه‌های عصبی را به شکل عالی بهبود بخشید. شتاب‌دهنده‌های مدرن عمدتاً از معماری‌های محاسباتی کارآمدی نظیر آرایه‌های سیستمولیک بهره می‌برند تا بتوانند با موازی‌سازی بالای عملیات و مصرف کمترین میزان انرژی، نیازهای محاسباتی سنگین شبکه‌های عصبی عمیق را برآورده کنند.

۱-۲) آشنایی با ماهیت عملیات پایه‌ای در پردازش شبکه‌های عصبی: عملگر MAC

همان‌طور که در بخش قبل اشاره شد، حجم اصلی محاسبات مربوط به شبکه‌های عصبی عمیق به‌شدت بر پایه ضرب ماتریس‌ها در ابعاد بزرگ استوار است. برای درک بهتر چگونگی انجام این محاسبات در بستر سخت‌افزار، باید به کوچک‌ترین و اصلی‌ترین جزء سازنده این الگوریتم‌ها نگاه کنیم. بنیاد و هسته اصلی در تمامی لایه‌های شبکه‌های عصبی، عملیاتی ساده اما پرتکرار به نام عملیات ضرب و انباشت یا عملگر مک (MAC) است.

^۱ Deep Neural Network

^۲ Multiply-Accumulate

^۳ Latency

^۴ Edge Devices

^۵ Tensor Processing Units (TPUs)

در فرآیند پردازش داده‌ها درون یک شبکه عصبی، هر گره یا نورون مقادیر متعددی را به عنوان ورودی دریافت می‌کند. هر کدام از این ورودی‌ها ابتدا باید در یک ضریب اختصاصی به نام وزن ضرب شوند. سپس حاصل ضرب‌های به‌دست‌آمده، با هم و در صورت نیاز با یک مقدار پایه¹ جمع می‌شوند تا سیگنال خروجی آن نورون شکل بگیرد. عملگر سخت‌افزاری MAC دقیقاً برای انجام سریع همین فرآیند تکرارپذیر طراحی شده است. به زبان ساده و از نظر ریاضیاتی، این عملگر به‌صورت مداوم سه مقدار ورودی را مدیریت می‌کند: دو ورودی به عنوان عملوندهای ضرب و یک ورودی برای انباشت مقدار، که ساختار منطقی این محاسبات به فرم رابطه $MAC_Output = (A \times B) + C$ نمایش داده می‌شود.

از دیدگاه سخت‌افزاری و بر اساس معماری شتاب‌دهنده‌های متداولی نظیر سیستم‌های آرایه سیستولیک، وظیفه انجام عملیات MAC بر عهده واحدهای محاسباتی پایه‌ای با نام عنصر پردازشی² است که به اختصار PE خوانده می‌شود. در معماری‌های سنتی و رایج سخت‌افزاری، ساختار درون هر PE به دو بخش مجزا تفکیک می‌شود؛ به این صورت که ابتدا یک واحد مستقل ضرب‌کننده³ تعبیه شده است و پس از آن یک ماشین جمع‌کننده⁴ قرار می‌گیرد تا نتیجه ضرب را به مقدار قبلی انباشته کند. اهمیت شناخت و بهینه‌سازی عملیات MAC از آنجا ناشی می‌شود که چه در فاز آموزش شبکه و چه در فاز استنتاج، پردازنده باید روزانه میلیاردها عدد از این عملیات را اجرا کند. این تعداد سرسام‌آور عملیات MAC باعث شده است که بلوک‌های عنصر پردازشی (PE) نقش تعیین‌کننده‌ای در طراحی پردازنده‌های هوش مصنوعی داشته باشند. در واقع سرعت و فرکانس کاری، فضای اشغال‌شده روی تراشه (سطح سیلیکون) و از همه مهم‌تر انرژی مصرفی کل سیستم پردازنده، مستقیماً توسط پیچیدگی و معماری استفاده‌شده در همین عملگرهای پایه MAC تعیین می‌شود. بنابراین، هرگونه کاهش در هزینه‌های فیزیکی و مصرفی عنصر پردازشی MAC در نهایت منجر به خلق شتاب‌دهنده‌ای بسیار پرسرعت‌تر و کم‌مصرف‌تر خواهد شد.

۳-۱) معرفی تئوری معماری آرایه سیستولیک^۵ و تاریخچه آن

توسعه روزافزون سیستم‌های محاسباتی با کارایی بالا همواره با یک چالش فیزیکی و ساختاری جدی مواجه بوده است: عدم تعادل میان سرعت محاسبات یک پردازنده و پهنای باند دسترسی به حافظه. در سیستم‌های رایج، محدودیت‌های ارتباطی موجب می‌شود که سرعت انتقال داده‌ها به مراتب کندتر از ظرفیت عملیاتی خود تراشه باشد. این مسئله، که در معماری‌های سنتی تحت عنوان گلوگاه فون‌نیومن شناخته می‌شود، باعث می‌شود توان پردازشی تا حد زیادی معطل دریافت و ارسال داده بماند. برای غلبه بر این محدودیت، مفهوم معماری و سخت‌افزارهای سیستولیک در اواخر دهه هفتاد میلادی، برای نخستین بار در دانشگاه کارنگی‌ملون^۶ بنیان‌گذاری شد. اندکی بعد در سال ۱۹۸۲، تئوری، مبانی و چارچوب بنیادین این معماری طی مقاله بسیار تاثیرگذاری توسط H.T. Kung ارائه گردید که تا امروز به‌عنوان منبع اصلی این حوزه شناخته می‌شود.

ایده مرکزی این تئوری، حل چالش در الگوریتم‌هایی است که ذاتاً به اصطلاح محدودشده به پردازش^۷ هستند. اینگونه مسائل (نظیر عملیات‌های رایج در شبکه‌های عصبی و پردازش سیگنال، مانند ضرب‌های پیوسته و تکراری ماتریس‌ها) در ساختار خود

¹ Bias

² Processing Element

³ Multiplier

⁴ Adder

⁵ Systolic Array

⁶ Carnegie-Mellon University

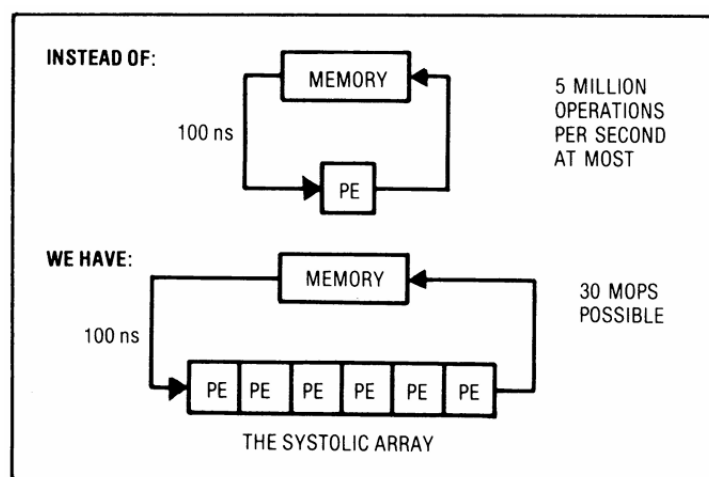
⁷ Compute-bound

به گونه‌ای می‌باشند که تعداد اعمال ریاضی روی آن‌ها، چندین برابر بزرگتر از حجم ورودی و خروجی کل الگوریتم است. معماری سیستمولیک پیشنهاد می‌دهد که اگر بتوان در طی چنین محاسباتی، استفاده‌های متعدد از تنها یک بار فراخوانی داده از حافظه انجام داد، نیاز به تبادلات ارتباطی به صورت چشم‌گیری کاهش خواهد یافت و سرعت در بستر تراشه‌های سخت‌افزاری تضمین می‌گردد.

برای درک چگونگی حل این گلوگاه پردازشی در ساختار سخت‌افزاری پیشنهادشده، نامگذاری کلمه سیستمولیک کاملاً گویاست. این اصطلاح در علم پزشکی به کار گرفته می‌شود و مربوط به انقباض ریتمیک قلب هنگام پمپاژ خون در شریان‌ها است. همان‌طور که قلب به صورت منظم، جریانی یکپارچه از خون اکسیژن‌دار را به سراسر بافت‌ها پمپاژ می‌کند تا تک‌تک سلول‌ها در مسیر آن بتوانند مواد مورد نیازشان را استخراج کرده و مصرف کنند بدون آن‌که برای هر تغذیه لازم باشد به قلب مراجعه کنند، جریان اطلاعات نیز در یک تراشه می‌تواند رویه مشابهی را در پیش گیرد. در این الگوی جذاب، سیستم‌های حافظه نقش همان قلب تپنده سخت‌افزار را بازی می‌کنند و بلوک‌های داده همان جریان خون را شامل می‌شوند. داده‌ها در این محیط از طریق یک پالس کلاک سراسری هماهنگ و با یک جریان و آهنگ از روی شبکه حرکت کرده و منتقل می‌شوند.

نتیجه اعمال این فلسفه آن است که هنگامی که اطلاعات خام تنها یک مرتبه از درگاه I/O خوانده می‌شوند، قبل از آن‌که بخواهند خروجی خود را مجدداً در آن ذخیره کنند، درون مسیری تشکیل شده از انبوه واحدهای پردازشی ساده تزریق می‌شوند. داده طی مسیر منظم، توسط چندین ماشین حساب مستقل پردازش می‌شود. در نهایت این سازوکار به گونه‌ای عمل می‌کند که پردازنده‌ها دیگر تماشاگر(!) تاخیر خواندن‌ها و نوشتن‌های مداوم نیستند.

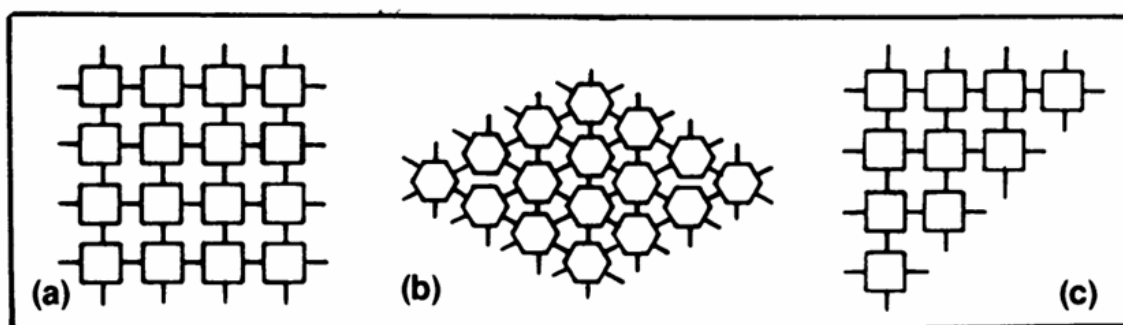
در سطح ساختاری سیستم‌های سخت‌افزاری، یک آرایه سیستمولیک شامل شبکه‌ای منظم و به هم پیوسته از واحدهای پردازشی ساده و تکرارپذیر است که به آن‌ها عناصر پردازشی یا به اختصار سلول‌های PE گفته می‌شود. این پردازنده‌ها قابلیت انجام محاسبات سریع پایه، به خصوص عملگرهای ریاضی که به تکرار بالایی نیازمندند را دارا هستند. وجه تمایز برجسته در آرایه‌های سیستمولیک نسبت به سایر معماری‌ها، قانون ارتباط آن‌ها با دنیای خارج است؛ در این شبکه‌ها انتقال اطلاعات صرفاً بر عهده مسیرهایی در لبه‌ها گذاشته شده است. یعنی برقراری تماس برای ورود و خروج اطلاعات خام و دسترسی مستقیم با حافظه‌های داخلی تنها مختص بلوک‌های حاشیه‌ای و بیرونی مرز شبکه است و سلول‌های درونی به طور کامل و خودکفا بدون دسترسی مجدد به حافظه، کارشان را به جلو می‌برند. (شکل ۱-۱)



شکل ۱-۱) اصل اساسی سیستم سیستمولیک

اطلاعات پس از دریافت شدن، در فرآیندی مانند یک ساختار خط لوله^۱ روی آرایه جریان می‌یابند. وظیفه یک سلول در سیستم‌های سیستمولیک، صرفاً دریافت یک مقدار، ضرب کردن آن در وزنش، انباشتن و در نهایت سینه به سینه تحویل دادن کل بسته اطلاعاتی به گرهی همسایه‌ی بعدی خودش برای ادامه همین روند تا رسیدن به پایان شبکه‌ی فیزیکی خواهد بود.

با در نظر گرفتن اهداف محاسباتی متفاوت پروژه‌ها (مانند ضرب معمولی داده‌ها، شبکه‌های پیچیده یادگیری، فیلترینگ و...)، مهندسين این شبکه‌های سخت‌افزاری را در ابعاد، هندسه‌ها و طرح‌های متنوعی سازماندهی می‌کنند. یک آرایه می‌تواند به ساده‌ترین حالت خود کاملاً در مسیر خطی (معماری یک‌بعدی) چیده شده باشد. از طرف دیگر به اقتضای ماهیت سیستم عامل و ابعاد دیتا می‌تواند کاملاً در قالب توپولوژی‌های مستطیلی ساده، چیدمان مثلثی و حتی اشکال شش ضلعی به خود ساختارهای شبکه دوبعدی بگیرند. (شکل ۱-۲)



شکل ۱-۲ انواع آرایه‌های سیستمولیک دو بُعدی

همین امر که عناصر PE کاملاً رفتار منظم داشته، هر دو مسیر ارتباطی و مدیریت دیتاها ساده است و کنترلر بر تمام پردازنده‌ها برای مسیریابی احاطه ندارد، سبب گردیده اتصالات کابل کشی وسیع حذف شوند. به لطف عدم وجود این خطوط اتصالاتی دوربرد که عامل محدودیت عملکرد بسیاری از سیستم‌ها در حوزه زمان و اشغال فضای سیلیکونی محسوب می‌گشتند، تئوری Kung در جمع‌بندی خود نشان می‌دهد که طراح می‌تواند شبکه‌های یک یا دوبعدی سیستمولیک را با تعداد بالا در سطح سیستم یکپارچه‌ای چون فناوری VLSI گسترش و به‌سادگی و متناسب با رشد نیازمندی‌ها پیاده‌سازی نماید، بی‌آنکه ساختار دچار مشکلات کلاک، تداخل اطلاعات و مصرف بیهوده فضا گردد.

۱-۴ اهداف پروژه، مشخصات فازهای طراحی و مستندسازی سیستم مورد مطالعه

با توجه به چالش‌های مطرح شده در خصوص محدودیت منابع پردازشی در شبکه‌های عصبی عمیق و مزایای برجسته آرایه‌های سیستمولیک، هدف اصلی و محوریت این پروژه، طراحی و پیاده‌سازی در سطح ماژول یک شتاب‌دهنده سخت‌افزاری بر پایه معماری آرایه سیستمولیک جهت محاسبه ضرب ماتریس‌ها است. در این گزارش، طراحی حول ساخت و ارزیابی یک عنصر پردازشی پایه‌ای (PE) و قرار دادن آن در یک پیکربندی شبکه‌ای متمرکز است تا بتواند عملیات اساسی محاسبات وزن‌ها، داده‌ها و مقادیر پایه (بایاس) را با بالاترین راندمان پیاده‌سازی نماید.

از منظر ریاضیات محاسباتی، شتاب‌دهنده‌ای که در این پروژه مورد توسعه قرار می‌گیرد، موظف است معادله اصلی ضرب و جمع ماتریسی را که شکل کلی آن $D = (A \times B) + C$ است را حل نماید. در روند پیش‌فرض طراحی برای این شبکه، فرض بر

¹ Pipeline

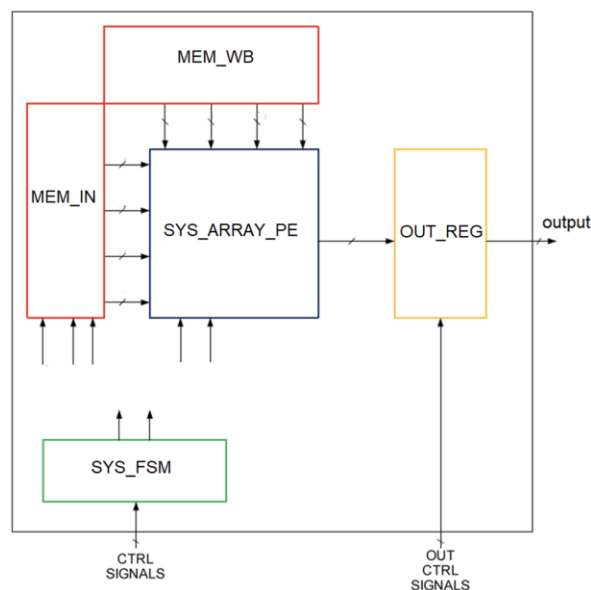
این است که ابعاد تمامی ماتریس‌های مذکور (شامل تعداد سطرها و ستون‌های n ، k و m) برابر با ۴ در نظر گرفته شوند که منجر به تشکیل یک آرایه با قابلیت انجام عملیات پیوسته خواهد شد.

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,k} \\ a_{2,1} & a_{2,2} & \dots & a_{2,k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \dots & a_{n,k} \end{bmatrix} \times \begin{bmatrix} b_{1,1} & b_{1,2} & \dots & b_{1,m} \\ b_{2,1} & b_{2,2} & \dots & b_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{k,1} & b_{k,2} & \dots & b_{k,m} \end{bmatrix} + \begin{bmatrix} c_{1,1} & c_{1,2} & \dots & c_{1,m} \\ c_{2,1} & c_{2,2} & \dots & c_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n,1} & c_{n,2} & \dots & c_{n,m} \end{bmatrix} =$$

$$D = \begin{bmatrix} \sum_{i=1}^k a_{1,i}b_{i,1} + c_{1,1} & \sum_{i=1}^k a_{1,i}b_{i,2} + c_{1,2} & \dots & \sum_{i=1}^k a_{1,i}b_{i,m} + c_{1,m} \\ \sum_{i=1}^k a_{2,i}b_{i,1} + c_{2,1} & \sum_{i=1}^k a_{2,i}b_{i,2} + c_{2,2} & \dots & \sum_{i=1}^k a_{2,i}b_{i,m} + c_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^k a_{n,i}b_{i,1} + c_{n,1} & \sum_{i=1}^k a_{n,i}b_{i,2} + c_{n,2} & \dots & \sum_{i=1}^k a_{n,i}b_{i,m} + c_{n,m} \end{bmatrix}$$

برای اینکه داده‌ها بتوانند بدون ایجاد گلوگاه ارتباطی با سرعت بالا در ساختار حرکت کنند، مشخصات این شتاب‌دهنده سخت‌افزاری در پنج فاز اصلی تفکیک و معماری می‌شود که حضور همگی آن‌ها در ساختار مدار نهایی الزامی است. این ماژول‌های اساسی عبارتند از:

هسته اصلی محاسبه که شامل آرایه‌ای منظم از بلوک‌های عنصر پردازشی (PE) یا به اختصار (SYS_ARRAY_PE) است و مسیر اصلی گذر داده‌ها را تشکیل می‌دهد. در کنار این شبکه پردازشی، دو واحد مستقل حافظه در نظر گرفته شده است: یکی برای بارگذاری داده‌های خام ورودی سیستم (MEM_IN) و دیگری برای نگهداری ضرایب وزن و بایاس شبکه عصبی (MEM_WB). در قلب سیستم نیز یک ماژول کنترلر وضعیت مبتنی بر ماشین حالت متناهی (SYS_FSM) تعبیه می‌گردد تا جریان ارسال، هماهنگی زمان‌بندی خطلوله (پایپ‌لاین) و صحت اجرای دستورات در سطح شبکه را به‌طور دقیق مدیریت کند. در نهایت، رجیسترهایی تحت عنوان خروجی سیستم (OUT_REG) جهت دریافت نتایج نهایی محاسبات پس از خروج از شبکه لحاظ خواهند شد.



شکل ۳-۱) معماری بلوک‌های اصلی سیستم پیشنهادی شتاب‌دهنده شامل مسیرهای داده، کنترلر ماشین حالت و حافظه‌ها

نحوه حرکت داده در ساختار طراحی شده به این شکل خواهد بود که پردازش‌ها دارای روند تناوبی و پلکانی هستند؛ به‌طوری‌که در نخستین کلاک، عملیات پردازشی تنها برای عناصر لبه و در نخستین سطر/ستون شبکه آغاز می‌گردد. با فرا رسیدن کلاک دوم، سایر PE‌های مجاور فعال شده و محاسبات را تحویل گرفته و گسترش می‌دهند تا جایی که سراسر شبکه با حفظ همزمانی، درگیر اجرای این ضرب ماتریسی خواهند شد.

برای اینکه پیاده‌سازی عنصر پردازشی (PE) در بستر شبکه سیستمولیک با نهایت کارایی همراه باشد، طراح سخت‌افزار وظیفه دارد برای انتخاب دو زیربلوک اساسی درون PE یعنی ماشین جمع‌کننده و ماشین ضرب‌کننده گزینه‌های متعددی را ارزیابی کند. مطابق روند تعریف‌شده برای پروژه، می‌بایست عملکرد بلوک ضرب‌کننده از میان حداقل دو ساختار مشخص یعنی ضرب‌کننده آرایه‌ای^۱ و ضرب‌کننده درخت والاس^۲ بررسی گردد. به همین ترتیب برای بلوک جمع‌کننده، باید گزینه‌های محبوبی همچون RCA^۳ در برابر CLA^۴ ارزیابی شوند تا در نهایت بر اساس مصالحه‌های مهندسی نظیر مساحت اشغالی، تاخیر مسیر بحرانی و میزان سازگاری با خط لوله سیستمولیک، مناسب‌ترین جفت طراحی در بلوک PE تثبیت شود.

مهم‌ترین چالش عددی در اجرای یک عملیات شبکه‌عصبی متصل بر سخت‌افزار ثابت، جلوگیری از لبریز شدن^۵ داده‌ها در اثر انباشت ضرب‌های متوالی است. به همین دلیل، ابعاد درگاه‌های سیستم باید به دقت پیکربندی شود. ورودی‌ها (دیتا و ضرایب وزن) همگی در قالب مقادیر ۸ بیت علامت‌دار تعریف شده‌اند. پس از ورود به PE، درگاه ضرب یک محدوده جدید را تولید می‌کند، به همین سبب خروجی ضرب‌کننده به ۱۶ بیت گسترش یافته است. اما زمانی که در عملیات ضرب یک ماتریس 4×4 بخواهیم حاصل چهار عملگر ۱۶ بیتی را به طور پیاپی انباشت کنیم و بایاس ۸ بیتی شبکه را به آن بیافزاییم، نتیجه ممکن است دچار خطای محاسباتی شود، زیرا عدد شانزده بیت دیگر برای مقادیر حاصل که مرز آن حدود [۶۵۱۵۲- تا ۶۵۶۶۳] است، پوشش کافی فراهم نمی‌سازد. بنابراین و بنابر ملاحظات مستندات اولیه پروژه، درگاه انباشت و نهایتاً درگاه خروجی در شبکه می‌بایست با رزولوشن ۱۸ بیت علامت‌دار طراحی و دریافت گردند تا پهنای کامل نتایج بدون تلفات ذخیره گردد.

فاز انتقال داده و زمان‌بندی شبکه وابسته به ماشین حالت FSM است. وظیفه کنترلر این است که با شروع وضعیت آغاز، ابتدا سیستم در حالت بیکار به انتظار سیگنال راه‌انداز باقی مانده و سپس فاز دریافت ضرایب و بایاس از دو ناحیه مشخص در بلوک ذخیره‌سازی محلی وزن‌ها (که ابعادی برابر ۱۶ بلوک حافظه‌ی ۸ بیتی به خود گرفته است) اجرا گردد. در کنار آن نیز دیتا به بلوک MEM_IN ارسال شده و به میزان ۸ بیت اصطلاحاً بسط داده^۶ و برای پردازنده لبه آماده می‌شوند. کنترل‌کننده سیستمولیک با وارد کردن صحیح به پردازنده‌ها وظیفه حفظ سلامت زمان‌بندی شبکه در تمام چرخه‌ی سیستم، شامل توالی‌های نشان‌داده‌شده در زیر را بر عهده خواهد داشت تا زمانی که تمام ۱۶ درایه ماتریس، به دقت در رجیستر پایانی نگارش یافته و بیت پرچم تایید عملیات نشان داده شود.

IDLE → LOAD → COMPUTE → WRITEBACK → DONE

¹ Array Multiplier

² Wallace Tree Multiplier

³ Ripple Carry Adder

⁴ Carry Lookahead Adder

⁵ Overflow

⁶ Sign-Extended

ارزیابی و تست نهایی سیستم مورد مطالعه شامل دو مولفه مهم است؛ معیار اول تأخیر پنهان است که از هنگام ثبت اولین کلاک تا روی خط رفتن نخستین خروجی با اعتبار محاسبه خواهد گردید. پارامتر دومی که از شبیه‌سازها خروجی گرفته می‌شود میزان گذردهی^۱ است که نمایانگر عملکرد مدار برای انجام تولید دیتای سالم در ازای پُر بودن تمام درگاه‌های ورودی PE خواهد بود. لازم به ذکر است که معماری این زیرساخت به صورت کاملاً پارامتری شده^۲ نوشته می‌شود تا متغیر پایه ابعادی ماتریس (m) بتواند با قابلیت پیکربندی بین اندازه یک تا چهار، معماری شبکه را به طور پویا در برگرد و امکان را برای ایجاد پردازش در قالب سایر توپولوژی‌های یادگیری از جمله Fully Connected را برای پردازنده تسهیل سازد.

¹ Throughput

² Parametric Design

فصل دوم: مروری بر معماری‌های مختلف برای پیاده‌سازی Systolic Array

(۲-۱) بررسی منطق نظری آرایه‌های سیستولیک (H.T. Kung – 1982)

برای طراحی یک پردازنده پرقدرت اختصاصی، صرفاً ساخت مدارهای سخت‌افزاری سریع کفایت نمی‌کند؛ بلکه منطق نظری و معماری پیاده‌سازی شده در دل سیستم، باید دقیقاً با ذات الگوریتم ریاضیاتی در حال پردازش هم‌سو باشد. مقاله H.T. Kung در سال ۱۹۸۲ بر روی همین مفاهیم عمیق تئوری دست گذاشت. وی برای نخستین بار توضیح داد که چرا رایانه‌های متداول (سیستم‌های مبتنی بر معماری همه‌منظوره) در اجرای مسائل حجیم به طور ذاتی دچار ناتوانی و اتلاف انرژی می‌شوند. مبنای تئوری بررسی‌های او در این مسیر، جداسازی محاسبات در دو کفه ترازوی منطقی بود: طیف پردازش‌های متکی به ارتباطات داده‌ای^۱ و در سوی دیگر مسائل درگیر شونده با قدرت خالص حسابگری^۲.

به زبان ریاضی، چنانچه شمار کل عملگرهای اعمال شده در یک عملیات سیستمی، با مقدار کل درایه‌ها و داده‌های خروجی و ورودی مورد نیاز هم‌مرز باشد، مسئله صرفاً گرفتار درگیری خط ارتباطی برای ورود/خروج بوده است. از طرفی خانواده محاسبات پایه عصبی که مبتنی بر تکرار ترکیب ماتریس‌ها و یافتن نقاط برخورد الگوریتم‌هایی نظیر کانولوشن روی داده‌ها هستند؛ جز دسته Compute-Bound شناسایی می‌شوند. بنابراین اجرای رویه‌هایی که ذاتاً تکراری می‌باشند، بر پهنای یک رایانه ساده محکوم به ماندن پشت درهای حافظه‌ای آن می‌ماند (عارضه‌ای به اسم باتل گلوگاه خط ارتباط حافظه‌ای سیستم).

مقاله پروفسور Kung، برای نجات از چنین مشکلی نظریه جالبی را داد. منطق وی روی بررسی ساده‌سازی مفهوم ریاضی الگوریتمی که در فرمول کانولوشن و تابع وزنی چندمتغیره به آن دست‌وپنجه نرم می‌کنیم بنیان گذاشته شده است. مطابق آنچه که می‌دانیم، ماهیت رشته‌ای یک سیستم وزن‌دهنده از جنس رابطه

$$y_i = w_1 x_i + w_2 x_{i+1} + \dots + w_k x_{i+k-1}$$

بوده و این مسئله دارای قابلیت‌های بسیار تکرار شونده مقادیری مانند w به تعداد x ‌ها و نهایتاً متغیر پایانی y است. راه‌حلی نظری در مقابله با چالش‌های گلوگاه گذردهی پهنای حافظه، حول استفاده مضاعف از دیتا بعد تنها یکبار لود شدن بنیان‌گذاری شده است. به بیانی بهتر با وارد نمودن منطقی معماری پایپ‌لاین که اطلاعات مانده خون (به شیوه منظم قلب تپنده پردازشگر سیستمی به طول شبکه‌ی همگون) پخش گردد سبب تحول چشمگیر راندمان‌ها می‌گردد، به‌طوریکه گذر اطلاعات مستقیماً بدون اینکه مجدداً وقتشان هدر رفته، خودجوش دست به دست می‌شوند تا این که بلوک نهایی پرچم تایید برای تکمیل چرخه‌ی عملیاتی می‌شود. در نتیجه دلیل مستحکمی پایه‌ریزی می‌شود که از روی کاغذ تا برد الکترونیکی توان محاسبات تا حدی بهینه‌سازی شده و هزینه‌های تلفات خطوط حافظه حذف می‌گردند.

پروفسور H.T. Kung پس از ارائه مفهوم اساسی استفاده چندباره از داده‌ها پس از یکبار فراخوانی برای رفع گلوگاه ارتباطی میان پردازنده و حافظه، منطق خود را با چهار اصل مهم برای طراحی تکمیل نمود. این اصول بنیادین مشخص می‌کنند که چرا معماری سیستولیک نه تنها بر روی کاغذ سریع است، بلکه پیاده‌سازی و ساخت شبکه‌های فیزیکی آن بر روی تراشه‌های مجتمع VLSI بسیار کارآمد و در ابعاد مقیاس‌پذیر مدرن کاملاً امکان‌پذیر می‌باشد.

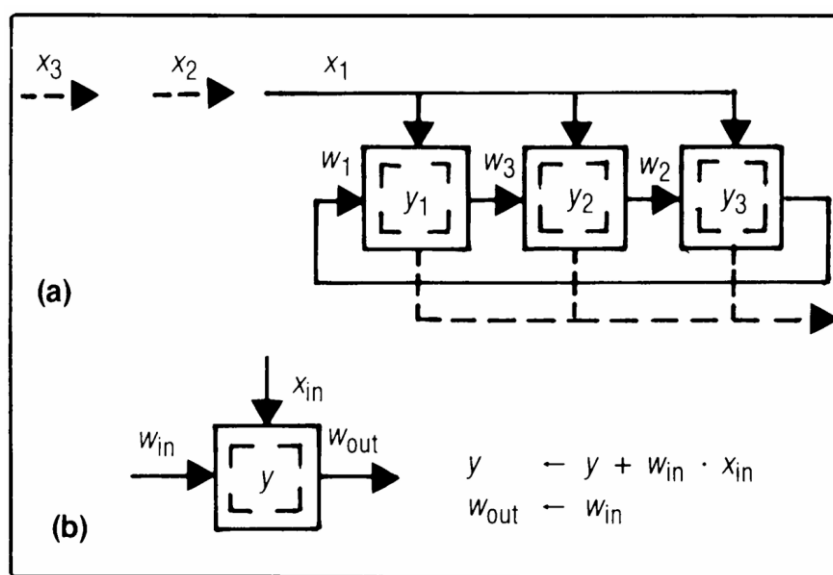
نخستین رویکرد مهم منطقی در این مقاله، تأکید بر مفهومی به نام سادگی طراحی و ماژولار بودن فیزیکی سیستم‌ها است. در پردازنده‌های همه‌منظوره، بالا بردن قدرت محاسبات به بهای طراحی و تعبیه پردازنده‌های فوق‌پچیده و درگیر ساختن آن‌ها

¹ I/O-Bound

² Compute-bound

انجام می‌شود؛ با این حال منطق Kung بیان می‌کند برای پیش‌برد با الگوریتم‌سازی منظم، دیگر هیچ نیازی به ارتقا درجه‌ی پیچیدگی واحدها نمی‌باشد و معماری سخت افزار باید ترکیبی باشد از تکرار پی‌درپی چند عضو سلولار به شدت ساده، در ابعاد ماتریسی گسترده. به زبانی ساده‌تر، وقتی ما بخواهیم مقیاس قدرتمان توسعه‌یابد این آرایه‌ها مقیاس‌پذیرند^۱ و تنها با بسط شبکه‌ی PE‌های کوچک اما یکپارچه، امکان توسعه معماری برای پردازش حجم سنگین در زمان مشابه روی تراشه فراهم می‌شود.

اصل سوم و چهارم در قانون معماری نظری H.T. Kung را باید دلیل اصلی موفقیت وی در مدارات مقیاس‌وسیع به‌شمار آورد؛ او شدیداً درباره مشکل سیم‌کشی‌های بلند، ارسال و کنترل‌های سراسری در کل شبکه‌های پردازنده‌ها همانند سیگنال‌های Global Broadcasting در ارسال از منبع به تک‌تک پردازنده‌ها در مقاله ۱۹۸۲ هشدار می‌دهد و راه‌حل را فقط در تبادل محلی جریان دیتاها با پردازشگران مجاور می‌داند. یک مهندس طراح می‌داند زمانی که سیم اتصالی مسیر ارسال کلاک‌های هدایتی یا دیتا بین عناصر PE فاقد مرز بندی نزدیک باشند تاخیر مسیر RC، پخش حرارت مصرف توان، انباشت پهنای و بروز انحرافات پالس کلاک^۲ گریبانگیر عملکرد قطعات می‌شود. پس به منظور کارایی ماشین محاسبات، این اتصالاتی‌ها از سیستم منع گردید تا بلوکها از روش تبادل فقط با یک‌همسایه مرزی مشغول به کار باشند و سیستم برای ارتباط راه دور دیگر فلج نباشد! به عنوان جمع‌بندی در پیاده سازی معماری سیستمولیک می‌توان قاطعانه بیان نمود تا زمانی که هسته‌ی عناصر، ساده نگه داشته شود (PE) و با این نوع دست‌به‌دست‌شدن خط لوله جریان همگون با شبکه پیوند برقرار داشته باشند، گلوگاهی بر بستر I/O برای دسترسی پدید نیامده و اجرای سنگین یک جریان آموزش/نتیجه‌گیری کانولوشنی با الگوریتمی مقیاس یافته کاملاً در قلب تئوری پردازش پر بازده سیستمولیکی سازگاری خواهند داشت.



شکل (۲-۱) نمایه داخلی چگونگی انتقال جریان مجاور به مجاور دیتا در المان محاسباتی منظم و عبور بدون درنگ سیستم بر اساس مدل پایپ لاینی Kung (۱۹۸۲)

¹ Expandable

² Clock Skews

۲-۲) طبقه‌بندی استراتژی‌های جریان داده (Dataflow) در شبکه‌های سخت‌افزاری

یکی از چالش‌های اصلی پس از تعیین ابعاد سخت‌افزاری یک آرایه سیستمولیک، مدیریت نحوه عبور داده‌ها در میان سلول‌های پردازشی است. در هر چرخه از عملیات محاسباتی مانند ضرب ماتریسی در شبکه‌های عصبی عمیق، ما همیشه با سه گروه مختلف از متغیرها سروکار داریم: داده‌های ورودی خام، مقادیر ضرایب یا همان وزن‌ها و در نهایت نتایج محاسبات میانی یا همان حاصل جمع‌های انباشته. از آنجایی که بازخوانی هر سه گروه این متغیرها به طور همزمان از حافظه مرکزی انرژی زیادی را مصرف کرده و زمان تاخیر را بالا می‌برد، استراتژی هوشمندانه‌ای به نام برنامه‌ریزی جریان داده^۱ در شبکه‌های سخت‌افزاری مطرح می‌شود.

مفهوم استراتژی جریان داده به این موضوع اشاره دارد که طراح سخت‌افزار تصمیم می‌گیرد از بین این سه عنصر محاسباتی (ورودی، وزن و خروجی)، حداقل یکی از آن‌ها را درون رجیستر خود واحد پردازشی (PE) اصطلاحاً ثابت یا میخ‌کوب^۲ کند و در عوض، مقادیر دیگر را درون شبکه سیستمولیک همانند یک خط لوله جاری سازد. این مدیریت هوشمندانه حرکت داده سبب کاهش جابه‌جایی فیزیکی مقادیر روی چیپ شده و توان مصرفی قطعه الکترونیکی را به حداقل می‌رساند. بر همین اساس، استراتژی‌های جریان داده به سه دسته طبقه‌بندی می‌شوند:

۱) استراتژی جریان داده با وزن‌های ثابت یا WS^۳:

در این الگوی معماری هدف ثابت نگه‌داشتن شبکه ارتباطی اعصاب است. در این شیوه، هر وزن یا ضریب تنها یک‌بار از حافظه محلی سیستم خوانده شده، وارد آرایه می‌شود و درون یک رجیستر در سلول محاسباتی ذخیره می‌شود و همان‌جا منتظر می‌ماند. سپس، بسته‌های متوالی از داده‌های ورودی از سویی، و مقادیر انباشت خروجی‌ها از سویی دیگر، با سرعت بالا از طریق شبکه جریان پیدا می‌کنند و ضرب شده و خارج می‌شوند. این معماری یکی از محبوب‌ترین فرم‌های جریان داده در پردازشگرهایی با مدل شبکه بزرگ همچون TPU گوگل است.

۲) استراتژی جریان داده با نتایج ثابت OS^۴:

در رویه دوم این دسته‌بندی که توسط کونگ به نام گروه طراحی R1 و R2 یعنی Results stay, Inputs & Weights Move پایه‌ریزی شده، سناریو کاملاً معکوس است. این‌بار وظیفه سلول پردازنده (PE) آن است که نتایج مربوط به جمع‌های ضرب هر مقطع را، تا تکمیل محاسبات، درون رجیستر حافظه‌دار خودش حفظ (انباشته) کند. به این ترتیب دیگر مقادیر انباشتگر با هربار محاسبه بین بلوک‌ها جابه‌جا نمی‌شوند، بلکه دو جبهه‌ی داده‌ها (ورودی خام) و (ضرایب) همزمان وارد درگاه‌ها شده و ضرب‌های آن‌ها پشت سرهم با مقدار داخل رجیستر در PE افزوده می‌شود. زمانیکه که کل سیکل‌ها طی شدند مقدار رجیستر نتیجه اجازه می‌یابد بر درگاه Output نهایی شبکه جاری شده و رجیستر دوباره ریست شود.

^۱ Dataflow

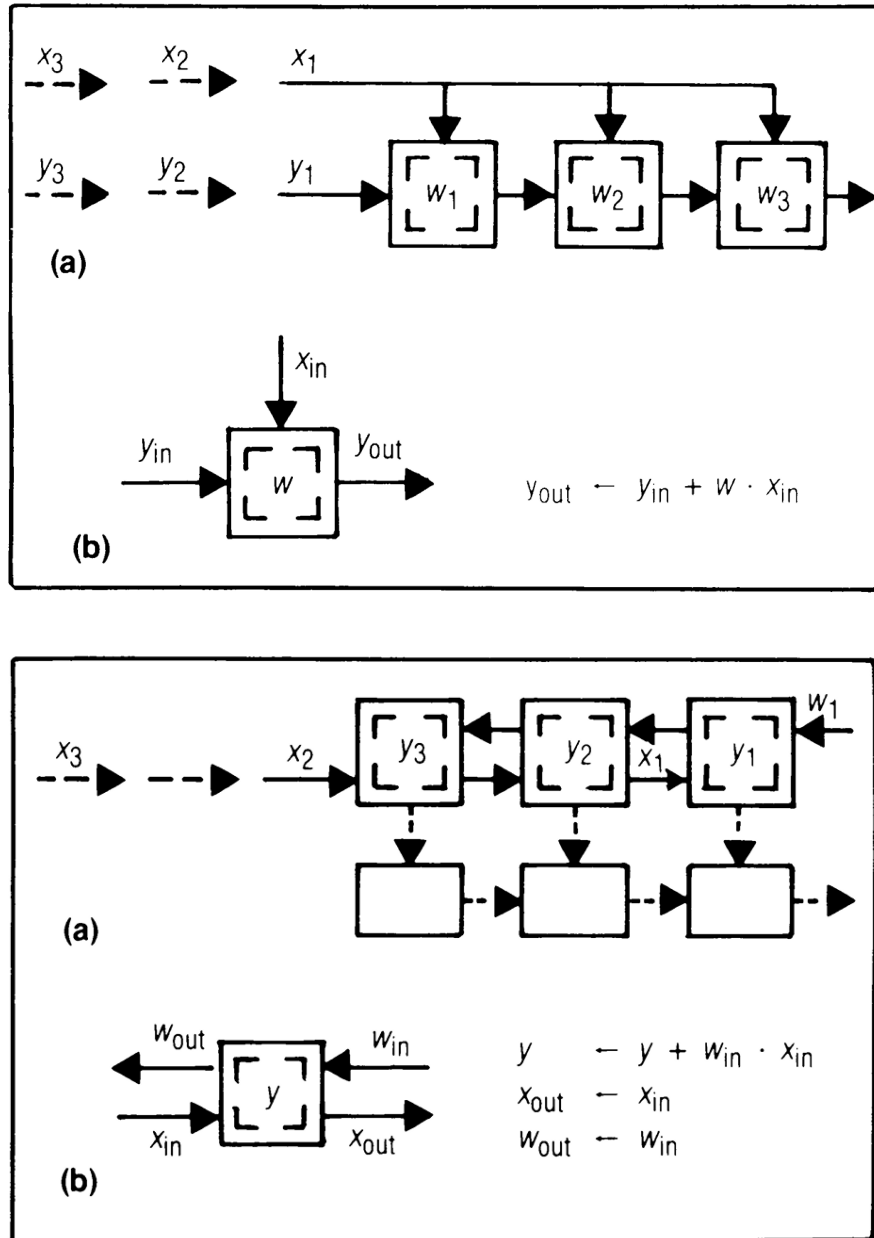
^۲ Stationary

^۳ Weight-Stationary

^۴ Output-Stationary

(۳) استراتژی با داده‌های ثابت^۱ و سایر روش‌ها:

هرچند در پردازنده‌های سیستم‌تولیک رواج و کاربرد معماری WS و OS بخش اصلی صنعت امروز شبکه عصبی را به دست دارند، ولی چنانچه حجم ماتریس مقادیر ورودی بسیار سنگین‌تر از لود کردن وزن‌های حافظه باشند (یا مثلاً نیاز به اعمال مجدد و فوری داشته باشیم)، برخی سخت‌افزارها معماری‌شان را بر اساس قفل کردن ورودی‌ها می‌گذارند. کونگ نیز مدل‌هایی که مبتنی بر Broadcast آنی از بالا، اما روان‌سازی سایر داده‌ها صورت می‌گیرند را مورد اشاره قرار داد.



شکل ۲-۲) تقابل مفهومی طبقه‌بندی جریان-داده روی ساختارهای سلولار پردازنده براساس تحقیقات H.T. Kung در شمای نخست الگوی مبتنی بر نگهداری وزن (WS) و شمای دوم روش متمرکز بر تثبیت داده‌های خروجی در انباشت‌کننده‌ی داخل‌گره (OS) قابل مشاهده‌اند.

¹ Input-Stationary

۳-۲) مقایسه حالت Output-Stationary با حالت Weight-Stationary و بررسی عملکرد در شبکه‌ها

هنگامی که وارد فاز پیاده‌سازی یک آرایه سیستم‌تولیک و قرار دادن عناصر پردازشی (PE) در کنار یکدیگر می‌شویم، مهم‌ترین تصمیمی که مسیر طراحی رجیسترها و اتصالات بین واحدی را مشخص می‌کند، مقایسه و انتخاب نهایی رویکرد حرکت داده‌ها در مدار است. رقابت اصلی در طراحی پردازنده‌های هوش مصنوعی (و به‌خصوص مدارهای انجام دهنده عملیات محاسبه لایه ضرب ماتریس‌ها) عمدتاً میان رویکرد Output-Stationary (رجیستر خروجی) و رویکرد Stationary Weight (رجیستر وزنی) است. هر یک از این استراتژی‌ها تأثیر مستقیمی بر نحوه انجام ریاضی عملگر MAC و در نتیجه تأخیر فیزیکی، حجم سیم‌کشی سیلیکون و چگالی انرژی بلوک محاسباتی خواهند داشت.

برای شروع این بررسی عمیق، در معماری مبتنی بر جریان داده (WS)، اولویت بر روی بازنگه‌داشتن پهنای باند ارتباطی با حافظه وزن‌ها است. همان‌طور که می‌دانید خواندن داده از بلوک‌های استاتیک محلی حافظه توان زیادی مصرف می‌کند. در روش WS به منظور به حداقل رساندن این تلفات خواندن/نوشتن از بلوک‌هایی چون (MEM_WB)، شبکه‌ای تشکیل می‌یابد که یک قطعه وزن (مثلاً هشت بیتی) به سلول تخصیص داده شده ارسال گشته و به عنوان پایه محاسباتی مقطعی PE فیکس و بی‌حرکت می‌گردد. روند انجام کار و توسعه حاصل جمع (مبتنی بر مدل فضایی-خطی الگوریتم‌های پروسسوری) در چنین فرمولی مطابق رابطه

$$PartialSum_{out} = PartialSum_{in} + (X_{flow} \times W_{fixed})$$

رفتار می‌نماید که یعنی جریان خروجی جزئی از انباشته شدن جریان جزئی دریافتی مرز همسایه، روی دستاورد ضربی همان ماشین سلولی محقق می‌یابد و بدین سبب مقادیر خروجی به شکل سیال بین بلوکی سینه به سینه گذر می‌کنند. این رویکرد به معماری مدارها به لحاظ چگالی و سرعت برای کاربردهایی که دسته‌های سنگینی از وزن‌ها فیکس شده‌اند انعطاف زیادی می‌دهد.

حال در جناح مقابل به کالبدشکافی معماری مبتنی بر روش انباشت-ایستای نتیجه یا همای روش نامدار (OS) می‌رسیم؛ منطقی که ساختار ماشین پردازنده را وادار به ذخیره‌نگهداری دستاورد جزئی خود در داخل رجیستری محدود (در بطن ماژول PE) می‌کند و به‌گونه‌ای طراحی شده که قطعات خرد وزن (w) و بسته‌های خرد داده اطلاعات (a)، دائماً جاری‌اند در حالی که متغیر محلی در این آرایش سخت‌افزاری طی هر پالس زمانی تنها تغییر ارزش درون سلولی پیدا کرده (بروزرسانی انباشته)، تا وقتی شمارگر آن به پایان نهایی پردازش سیستم‌تولیکی رسید رجیسترها تخلیه شوند و برای خروج به باس ارسال گردند. روند ریاضی و تجمیع مقطعی زمانی و بروزآوری این رویه پردازشی متناسب با چگونگی ذکر شده در رابطه

$$Accumulator_value(t) = Accumulator_value(t - 1) + (X_{in(t)} \times W_{in(t)})$$

طرح ریزی می‌گردد.

یکی از مشکلات و آسیب‌زایی‌های موجود که از طریق متد WS پدید می‌آید چالش افزایش سایز باس کابل و خطوط پهن‌تر انتقال در مورد تراشه پردازشی خواهد بود. به طور خلاصه چون داده نتیجه همزمان دست به دست می‌گردد نیاز است کابل با پهنایی کامل از بیت خروجی کشیده شود، مثلاً پهنایی شامل انتقال ۱۸ بیت علامتدار؛ بنابراین وقتی رزولوشن به مقدار ۱۶ تا ۱۸ بیت گسترش یابد، شیفتهای دائم این پالت‌ها روی مسیر طولانی پردازنده‌ها می‌تواند عامل اتلاف جریان انرژی گردد. بالعکس نقطه درخشش Output-Stationary در شبکه‌های مدار سخت‌افزاری جایی پررنگ می‌گردد که باس بزرگ فقط در حباب رجیستر یک ماژول PE مدفون گشته‌اند، به‌نوعی توان پراکندگی تلف نمی‌شود و ماحصل معماری تنها انتقال جریان

هشت بیتی بین مداراتی می‌باشد و پس از کامل طی شدن ضرب تمامی ماتریس، اطلاعات یک‌جا فقط خوانش شده است (مستقیم بر خط خروجی پروژه)، که همین نکته مهم سبب گشته معماری‌های OS به جهت صرفه‌جویی پالس کلاک‌ها در معماری و پنهان نمودن پهنای ترافیک نتایج با ابعاد بالا، تبدیل به قهرمان کارآمد پردازنده‌ها گردند و برای توسعه پروژه‌های شتاب‌دهنده کاربرد روزافزونی ارائه کنند.

برای اینکه یک سیستم پردازش عصبی مبتنی بر آرایه‌های سیستولیک (خصوصاً معماری‌هایی که حالت خروجی ایستا یا OS را به کار می‌گیرند) بتواند به راندمان قابل قبولی در دنیای واقعی دست یابد، تنها نحوه عبور داده‌ها مهم نیست، بلکه معماری بلوک‌های داخلی PE که هسته‌ی اصلی این استراتژی را در خود جای داده است نیز به شدت بر مصرف توان تاثیرگذار است. همان‌طور که در یک استراتژی Output-Stationary، رجیستر ۱۸ بیتی نتیجه درون سلول باقی می‌ماند، دفعات تغییر وضعیت گیت‌های منطقی (سوئیچینگ) داخل رجیستر به فرکانس کار پردازنده پیوند می‌خورد؛ بنابراین طراحی فیزیکی ماشین ضرب‌کننده و جمع‌کننده که مدار MAC را می‌سازند، حیاتی است.

در معماری‌های رایج سخت‌افزاری، ساختار سنتی یک پردازنده بدین شکل است که عملیات ضرب به عنوان یک ماشین کاملاً مستقل، مقادیر a و b را می‌گیرد و نتایج را تحویل یک گیت مجزای جمع‌کننده می‌دهد تا با مقدار رجیستر محلی جمع شوند (الگویی که در معماری کاندید پروژه مانند بهره‌گیری از واحد جمع با رویکرد RCA یا CLA مطرح شده است). اما پژوهش‌های جدید نشان می‌دهد با در هم آمیختن این مسیر (ادغام ماشین‌ها به یک سخت افزار پردازشی منسجم که ضرب و انباشت را همزمان محقق سازد)، چگالی توانی شبکه کاهش چشم‌گیری پیدا می‌کند. تحقیقات مدرن و ارائه مقالاتی در جهت معماری PE کارآمد (نظیر رهیافت مبتنی بر بلوک‌های جزئی مثبت (PPC) و بلوک‌های منطقی جزئی منفی (NPPC) برای کنترل اعداد علامت‌دار هشت‌بیتی)، نشان می‌دهد اگر ماشین را به این سلول‌های جمعی-ضربی بهینه مزین سازیم، تلفات توانی شبکه‌های ما به واسطه کاهش همزمان محصولات میانی ریاضی بسیار تنزل می‌یابد.

برای اثبات ارجحیت‌های شبکه در فضای مهندسی الکترونیک تراشه، از دو شاخص قدرتمند به نام مساحت فیزیکی^۱ میزان سیلیسیوم اشغالی واحدها (و شاخص حیاتی توان-تأخیر یعنی PDP^۲ مخفف برای بیان هزینه مصرف ماشین بهره گرفته می‌شود. وقتی متدهای سیال و ثابت معماری‌های نام‌برده را درون شبکه توسعه یافته (نظیر شبکه‌ایی شامل ضرب ماتریس‌ها)، در مقیاس فشرده می‌آزماییم؛ استراتژی OS که بر واحدهای ضربی منسجم خطاییاب-خطی استوار گشته؛ در ترکیب خود تا سقف ۲۴ الی ۶۵ درصد توان چرخه‌ی تولید حاصلضرب نهایی خود (برای نمونه در پیاده‌سازی بلوک ۸ بیت دقیق علامتدار) را در قیاس با شیوه‌های ابتدایی کاهش می‌دهد، که این ارزش والای بهره‌مندی درست در بستر سیستولیکی سخت‌افزارها را نمایش داده و آن را بهترین رویکرد در محاسبات دستگاه‌هایی مثل لبه‌اینترنت و فیلترینگ لبه تصویر بدل نموده است.

همچنین یک بعد اساسی دیگر عملکرد معماری شبکه‌های ضرب سیستولیکی، محاسبه شاخص راندمان کارایی و چرخه‌های تأخیر آن‌ها می‌باشد. وقتی متد حرکت‌ها، معماری شبکه‌ای چند در چند (N در N ماتریس سخت افزاری شبکه PE) ساخته باشد و تغذیه این المانهای یکپارچه به عنوان معماری لوله پایپ-لاینینگ آغاز شود، طراح پارامتر ارزیاب تأخیر مدار را براساس محاسبه چرخه استارت نقطه ارائه خروجی بامعنا به نام Latency فرمول می‌کند. همان‌طور که در معادلات بنیادی شبکه‌های

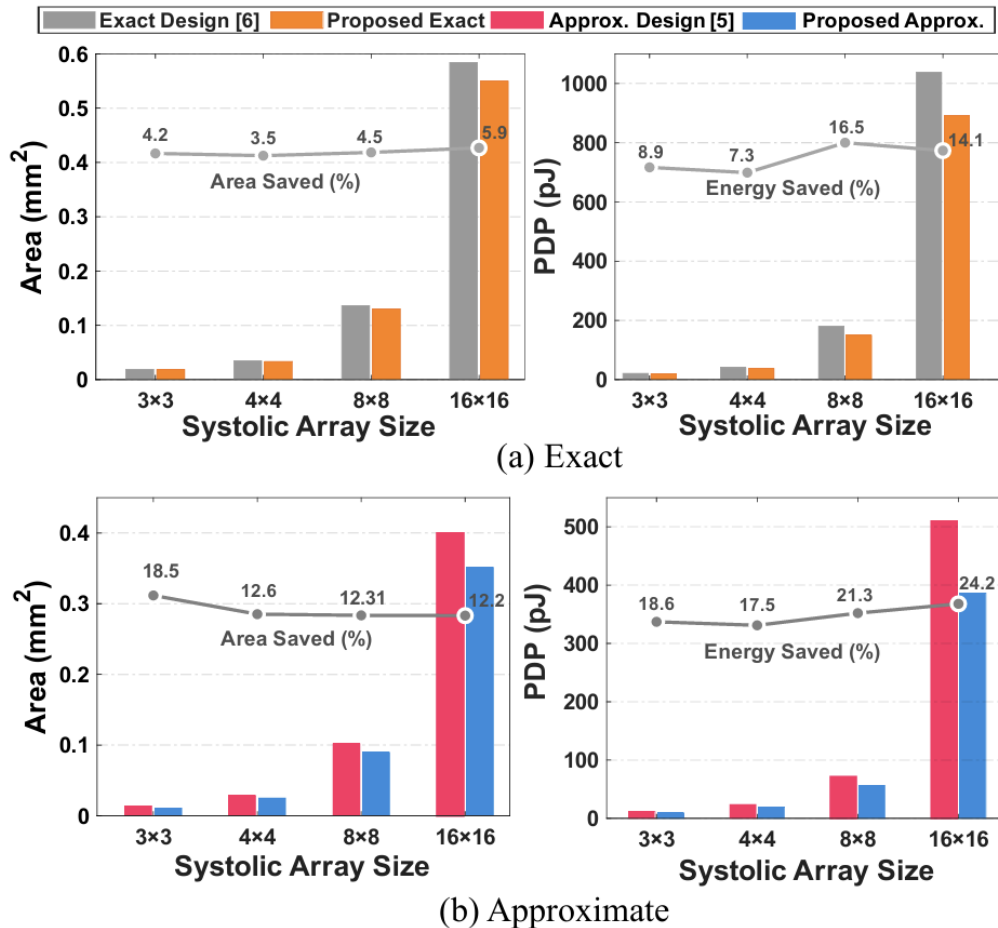
¹ Area

² Power Delay Product

معماری رایج سیستم‌تولیکی استنتاج گشته، زمان لازم برای به ثمر نشستن خروجی با ابعاد ابعاد ماتریسی و شیف‌ت پایپ‌لاینی داده مطابق رابطه کلی

$$Latency = 3N - 2$$

مدل می‌شود. پس از آنکه مسیر شبکه پُر گشت، جریان پیوسته‌ای شروع به سرازیر شدن از دهانه باس خواهد کرد. مقداری که هر کلاک سیستم یک نتیجه معتبر آماده به تحویل در خروجی ارائه می‌کند، در معماری به عنوان مقدار تراکنش پایپ لاین یا مقیاس **Throughput** اندازه‌گیری گشته که عامل پیروزی این مدارهای گران اما پرثمر توجیه خواهد شد.



شکل ۳-۲) مقایسه از دستاورد حفظ توان الکتریکی تراشه (PDP) با توان خط داده‌ها در آرایه‌های گسترش یافته براساس ساختار ۸بیتی ضرب علامتدار

۲-۴) مکانیزم پخش سیگنال کلاک (Clock Distribution) در المان‌های متصل متقارن

همان‌طور که در فصل اول تشریح شد، معماری سیستم‌تولیک تماماً بر پایه سادگی و حذف ارتباطات سراسری طولانی برای انتقال داده بنا شده است. این موضوع به طراح اجازه می‌دهد داده‌ها را صرفاً از یک المان پردازشی به همسایه مجاورش در یک ساختار متقارن هدایت کند. با این حال، حتی در ناب‌ترین شبکه‌های سیستم‌تولیکی، همچنان به یک بستر ارتباطی سراسری روی بورد الکترونیکی نیاز داریم؛ تمام عناصر فیزیکی در یک شبکه پردازشی باید دقیقاً در یک ضرب‌آهنگ واحد و منظم هماهنگ شوند که این قلب تپنده، همان سیگنال جهانی کلاک یا سیستم کلاک است.

بنابراین، تنها خط انتقالی (به جز کابل‌های منبع تغذیه و اتصال زمین) که در کل آرایه کشیده می‌شود و در تک‌تک گره‌ها حضور پیدا می‌کند، شبکه پخش سیگنال کلاک^۱ می‌باشد. هدف مکانیزم پخش سیگنال کلاک این است که تمامی عناصر پردازشی شبکه در یک شبکه کاملاً متقارن (مثلاً شبکه N در N که به شکل لوله جریان در حال محاسبه لایه‌های عصبی هستند)، با دریافت لبه‌های بالارونده این سیگنال از یک ماشین کنترل مرکزی اقدام به جابه‌جا کردن مقادیر میانی و پیش‌برد جریان ضرب کنند. اما هنگامی که با شبکه‌های فیزیکی متصل وسیع (مانند یک تراشه متشکل از هزاران بلوک PE) سروکار داریم، مدیریت این همگام‌سازی ابداً ساده نخواهد بود.

یکی از جدی‌ترین تهدیداتی که شبکه‌های یکپارچه سیلیکونی با آن مواجه می‌شوند، چالشی موسوم به انحراف پالس کلاک است. وقتی پالس الکتریکی کنترلر قرار است روی یک شبکه پهناور فلزی حرکت کرده و به گره‌ها برسد، تأخیرهای ذاتی فیزیکی کابل‌ها (مدل‌سازی شده بر اثر خازن‌ها و مقاومت مسیر باعث می‌شوند که گره‌های قرار گرفته در فواصل دورتر نسبت به مرکز فرمان، این لبه‌ها را اندکی دیرتر از المان‌های نزدیک به کنترلر دریافت کنند. به این ترتیب، احتمال عدم همگام‌سازی و خطای محاسباتی میان دو المانی که متقارن هستند، به وجود می‌آید.

بر اساس مشاهدات تحلیلی پروفیسور Kung، شبکه‌های سیستمیک رفتار کاملاً متفاوتی در برابر خطای Skew بسته به ساختار هندسی معماری خود نشان می‌دهند:

اگر چیدمان شبکه کاملاً به صورت خطی (آرایه متصل یک‌بعدی) تعبیه شده باشد، معماری سیستمیک حتی با بروز پدیده انحراف زمان، در سراسر این امتداد طولی مقاوم خواهد بود و به شکلی شگفت‌آور این تأخیر انباشته باعث اختلال ارتباطی مابین انتهای قطعات مجاور این المان‌های یک‌بعدی نخواهد گردید؛ و کل جریان کار خود را بدون وقفه توسعه می‌دهد. اما مسئله پیچیده جایی پدیدار می‌شود که شبکه‌ی سیستمیک دارای ساختار عریض و ماتریس‌گونه باشد (همانند شبکه دوبعدی که قرار است در این پروژه‌ی شتاب‌دهنده برای ضرب شبکه‌های عمیق به مقیاس پیاده در آیند). پخش نامتوازن پالس‌ها در شبکه بزرگ دوبعدی به شدت چالش‌برانگیز شده، چنانچه تیک‌تاک این آرایه به طور نابرابر منتشر گردد، خطر بهم‌ریختگی جمعی اطلاعات برای شبکه وجود دارد.

برای مدیریت انحراف سیگنال کلاک در این هندسه آرایه دوبعدی متقارن، طراح سیستم می‌تواند در رویه کلان شبکه دو شیوه فنی اتخاذ نماید؛ راه حل نخست (آسان و سنتی)؛ اجباراً کاستن از راندمان با افزایش تأخیر بین‌پالس یا به عبارتی کاستن از سرعت و ساماندهی جریان سیستم برای غلبه قطعی با این عقب‌ماندنی‌هاست. و راهبرد مهندسی دیگر برای حل کلاک اسکیوها، چشم‌پوشی موقتی روی خط کلاک‌های فراگیر (جهانی) از مسیر اصلی سیستم؛ و رفتن سراغ استقرار شیوه مداری اصطلاحاً مستقل-و-هماهنگ-ساز دست‌تکان‌دانی در همسایگی همان پردازنده که به نام مدارهای زمان‌سنج خودی^۲ نام نهاده می‌شود. که اینگونه بدون اختلال کلاکی پردازش بینهایت عظیم دوبعدی گشته اما در اجرا مستلزم حجم مدار پیاده‌ساز پرهزینه می‌باشد. به واسطه وجود تمامی همین پدیده‌های توزیع، هنگامی که قصد استخراج معیاری نظیر تأخیر فرکانس در قطعات توسعه‌یافته سیستمیک با ابزار Delay Tools Synthesis در مقالات)، داریم اهمیت کنترل سیستم نمایان شده تا از گلوگاه‌های ناخواسته مدارهای پردازش جلوگیری گردد.

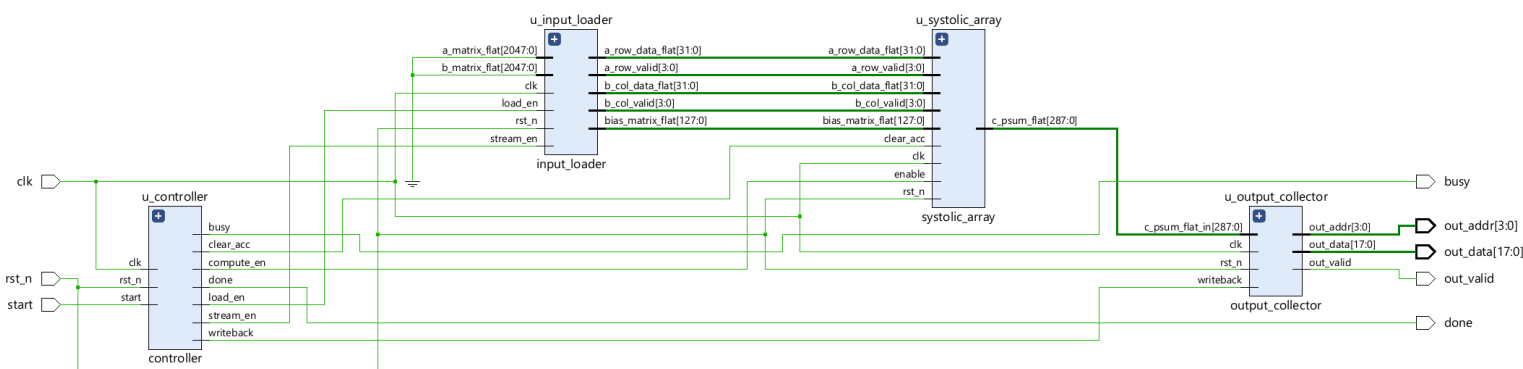
¹ Clock Distribution Network

² Self-timed synchronization protocols

فصل سوم: روش بررسی معماری کلی سیستم پیاده شده

۳-۱ معرفی ماژول سطح بالای کنترل کننده

در این پروژه، ماژول `systolic_array_top` به عنوان بالاترین لایه بیرونی سیستم عمل کرده و ارتباط میان پردازنده شبکه عصبی و دنیای بیرون را ایجاد می‌کند. طراحی این قسمت بر اساس الزامات اعلام شده در تعریف پروژه برای نام‌گذاری و پیکربندی درگاه‌های کنترلی سیستم (مانند سیگنال‌های `Start`، `Done`، `Busy` و خروجی‌های معتبر) توسعه داده شده است. از آنجا که سیستم طراحی شده بر پایه ساختار آرایه سیستولیک باید به شکل یک جریان منظم عمل کند، درون ماژول اصلی چهار بلوک مهم کنار هم قرار گرفته‌اند. این بلوک‌ها شامل واحد ماشین حالت سیستم یا کنترلر^۱، واحد مدیریت خوانش حافظه ورودی^۲، بلوک محاسباتی هسته یا همان شبکه‌ی سیستولیک^۳ شامل مجموعه‌ای از المان‌های پردازشی و نهایتاً جمع‌کننده‌ی ترتیبی خروجی‌ها^۴ هستند. به این ترتیب سیگنال کنترل با شروع پردازش از واحد حافظه داده‌ها را می‌خواند، داده‌های واکنشی شده روانه‌ی آرایه‌ی پردازشی می‌شوند و پس از اتمام محاسبه ماتریسی، داده‌های نهایی وارد بخش رجیسترهای خروجی خواهند شد.



شکل ۳-۱) دیاگرام ماژول سطح بالا و ارتباط میان بلوک‌های داخلی کنترلر، حافظه، شبکه سیستولیک و رابط خروجی

از نکات مهم پیاده‌سازی شده این سیستم، طراحی کاملاً پارامتریک و انعطاف‌پذیر^۵ سخت‌افزار آن است. با بهره‌گیری از پارامترهای m (تعداد ردیف‌های پردازشی ماتریس) و n (تعداد ستون‌ها)، و با اعمال پیکربندی ریاضی نشان داده شده در

$$OutputSize \propto (m \times n), \quad m \in \{1, 2, 4\}$$

سیستم می‌تواند هم برای حالت یک نورون در ساختار تمام متصل و هم حالت شبکه (شبکه کامل چهار نورونی) پیکربندی و به راحتی سنتز گردد، بدون اینکه به تغییری در منطق کنترلرهای داخلی احتیاج داشته باشیم.

¹ controller

² input_loader

³ systolic_array

⁴ output_collector

⁵ Scalable

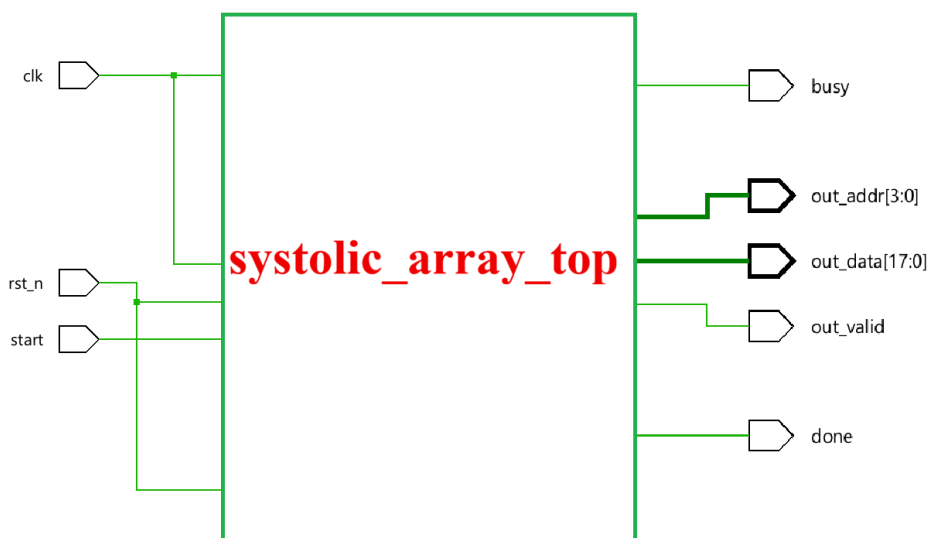
۳-۲) تشریح سیگنال‌ها، پورت‌ها و پین‌های دقیق رابط کاربری

درگاه‌های ورودی و خروجی ماژول بالاترین سطح، مبتنی بر داکيومنت الزامات پروژه طراحی شده‌اند. همانطور که در بخش قبل اشاره شد، پارامتر ردیف‌های شبکه به عنوان M و پارامتر مربوط به ستون‌ها با N مشخص شده‌اند. اگر قرار باشد شتاب‌دهنده برای ماتریس‌های با ابعاد چهار در چهار کار کند، هر دو پارامتر بر روی عدد چهار تنظیم می‌شوند.

System_Configuration: $M = 4, N = 4$, (Neurons/Matrix Dimension)

در کنار تنظیم ابعاد شبکه، پارامتر اندازه مقدار ۸ بیت (نشان‌گر ۸ بیت ورودی‌های علامت‌دار) تنظیم گردیده است. اندازه گذرگاه‌های خروجی نهایی پس از مراحل ضرب و تجمع در قالب پارامتری ۱۸ بیتی تعریف شده است. این پهنای ۱۸ بیتی در واقع منطبق با بدترین شرایط تولید اعداد بزرگ پس از جمع کردن بلوک‌های ضرب انجام گرفته در یک سطر شبکه تنظیم شده است تا پورت خروجی هیچ‌گاه دچار خطای سرریز^۱ در نمایش اطلاعات نگردد.

$$\text{Maximum Result} \in [-65152, 65663] \Rightarrow \text{Total Width} \geq 18 - \text{Bits}$$



شکل ۳-۲) نمایش پورت‌های اصلی ماژول `systolic_array_top`

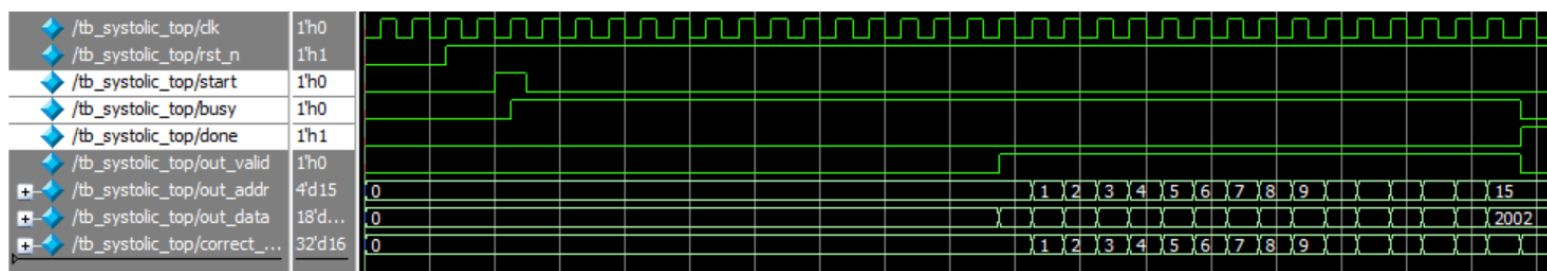
پایه‌ی تامین سیگنال کلاک برای پیاده‌سازی طراحی‌های مبتنی بر زمان‌بندی سنکرون است و لبه‌ی بالارونده آن تمام منطق‌ها از واحد انتقال حافظه تا پردازش شبکه‌ی سیستمولیک را به صورت قدم به قدم به انجام می‌رساند. پورت بعدی، پایه مربوط به سیگنال ریست یا بازنشانی است که در اینجا از نوع فعال پایین طراحی شده است. از آنجا که سیستم حاوی رجیسترها و المان‌های حافظه‌دار بی‌شماری چه در کنترلر وضعیت‌های ماشین و چه در بین رجیسترهای بافر درگاه داده است، هنگامی که سیگنال ذکر شده صفر گردد، سخت‌افزار فوراً ریست می‌شود و تمام فلیپ‌فلاپ‌های حاوی جمع‌جزئی، پورت‌های استخراج شده‌ی داده و مراحل طی شده کنترل‌گر به صورت آنی پاکسازی^۲ می‌شوند تا شتاب‌دهنده منتظر سیکل پردازشی جدید خود در حالت بی‌کاری قرار گیرد. این عملکرد در تراشه‌ها اهمیت ویژه‌ای دارد تا داده‌های باقی‌مانده از محاسبه پیشین، روی محاسبات جدید تداخل ایجاد نکنند.

^۱ Overflow

^۲ Clear

در هر ماژول شتاب‌دهنده، لازم است تا وضعیت فعلی تراشه به پردازنده اصلی گزارش شود و سیستم بر اساس یک پروتکل هماهنگی بداند که چه زمانی داده‌ها معتبر هستند و چه زمانی امکان آغاز پردازش جدید وجود دارد. برای پاسخ به این نیاز سیستم، سه پین کنترلی **start**، **busy** و **done** به ماژول **systolic_array_top** اضافه شده است.

پین **start** یک سیگنال ورودی تک‌بیتی است که به نوعی تریگر آغاز پردازش به شمار می‌رود. به محض اعمال یک پالس روی این پورت، واحد کنترل‌گر داخلی سیستم متوجه می‌شود که مقادیر اولیه برای ورودی و ماتریس وزن‌ها و بایاس بر روی حافظه مهیا شده و باید مراحل محاسباتی استارت بخورد. از سویی دیگر، خروجی وضعیت با عنوان پین **busy** تعبیه شده است. این سیگنال زمانی که کنترل‌گر مرکزی سیستم از فاز بیکاری و استراحت (**IDLE**) خارج شده و وارد فرآیندهای لود داده و پردازش سنگین در شبکه‌ی آرایه سیستمولیک شود، به سطح بالا (یک منطقی) می‌رود. بالا بودن ارزش این سیگنال، پردازنده‌های بیرونی را متوجه می‌کند که مسیرهای دیتای سیستم مسدود هستند و شتاب‌دهنده نمی‌تواند دستور جدیدی بپذیرد. در نهایت، با تکمیل کل جریان عملیاتی آرایه پردازشی و ارسال تمامی اطلاعات، سیگنال تک‌بیتی خروجی یعنی **done** برای یک کلاک فعال می‌شود که خبر اتمام موفقیت‌آمیز فاز قبلی پردازش روی ضرب کامل ماتریس شبکه را می‌دهد و سیستم مجدد آماده اجرای گام‌های پردازشی جدید خواهد بود.



شکل ۳-۳) شکل موج تغییرات سیگنال‌های کنترلی سیستم؛ در این شماتیک فعال شدن پایه **start** و نگهداشت خط **busy** توسط کنترل‌گر داخلی و نهایتاً پالس اعلام اتمام عملیات (**done**) قابل رویت است.

بخش آخر درگاه‌ها به روش ارتباط سیستم با حافظه مقصد (آنجا که حاصل ضرب و محاسبه نوروں‌ها ارسال میشوند) بازمی‌گردد. خروجی در شبکه به شکل دسته‌ای و تمام‌موازی بیرون داده نمی‌شود، بلکه این انتقال به صورت ارسال ترتیبی طراحی شده است تا نیاز به پین‌های مازاد به شدت کاهش یابد و استانداردهای طراحی بلوک‌های سخت‌افزاری روی پروتکل رابط رعایت شود.

این عملیات خوانش داده روی سه درگاه اساسی قرار گرفته است. اولین آن‌ها پایه **out_valid** است؛ یک سیگنال با یک شدنش خبر از وجود دیتایی پایدار روی پایه‌ی گذرگاه‌ها می‌دهد. هم‌زمان با یک‌شدن پایه اعتبار، خروجی چهار بیتی به نام **out_addr** با تغییرات خود از ۰۰۰۰ تا ۱۱۱۱، مشخص می‌کند که هر یک از جواب‌های محاسبه شده مربوط به کدام درایه از ماتریس هستند.

$Memory_Addresses (4bits) \Rightarrow Address_Range: [0, (m \times n) - 1] \xrightarrow{\text{for } m=4, n=4} Address \in \{0, \dots, 15\}$

نهایتاً، همراه این آدرس ایجاد شده، پایه برداری ۱۸ بیتی **out_data** به محیط بیرونی شتاب‌دهنده مسیر می‌دهد. دیتایی که حاصل ضرب و پردازش است و بصورت جواب نهایی بدست آمده است.

۳-۳) پارتیشن‌بندی میان المان هسته محاسباتی، رابط حافظه و...

همانطور که در بخش ۳-۱ توضیح داده شد، برای جلوگیری از تداخل مسیرهای ارتباطی، کاهش پیچیدگی‌های زمان‌بندی و دیباگ آسان‌تر، سیستم طراحی شده به چهار زیرسیستم کاملاً مجزا تقسیم شده است. این قطعات مداری به ترتیب شامل: (۱) واحد کنترل‌کننده^۱، (۲) رابط بارگذاری و آماده‌سازی اطلاعات از حافظه^۲، (۳) شبکه‌ی پردازش‌گر و موتور محاسباتی سیستم^۳، و (۴) بازنویسی ترتیبی پاسخ‌ها در خروجی^۴ هستند. هر بلوک، سیگنال‌های مشخص خود را داشته و اطلاعات از طریق این گذرگاه‌های سیگنال جریان پیدا می‌کنند تا عملکرد نهایی تراشه مانند یک خط لوله^۵ منظم رو به جلو برود.

ساختار بخش کنترلی بر پایه یک مدار ماشین حالت بنا نهاده شده که سنکرون با کلاک اعمالی به سیستم است. حالت‌های این سیستم در پنج وضعیت طراحی شده‌اند: حالت آماده‌باش یا بی‌کاری (IDLE)، واکنشی یا لود حافظه (LOAD)، محاسبه یا پردازش (COMPUTE)، خوانش و ثبت‌کردن نتایج (WRITEBACK) و حالت نهایی (DONE). در وضعیت IDLE، شبکه کمترین درگیری را تجربه می‌کند؛ به محض دریافت پالس استارت، سیستم وضعیت جدید خود یعنی LOAD را به خود گرفته و همزمان سیگنال‌های ریستارت اکومولاتورها (بازنشانی جمع‌کننده‌ها در شبکه) و مجوز خواندن حافظه را فعال می‌نماید. وضعیت LOAD بدین شکل طراحی شده است که تراشه **دو** سیکل را صرف ثبت مطمئن دیتا روی رجیسترهای ورودی می‌کند.

با انتقال بلوک منطقی وضعیت ماشین به حالت COMPUTE، سخت‌افزار با درنظر گرفتن داده‌های ورودی و همچنین مسیر خروجی آرایه‌های سیستولیک، پردازش را شروع می‌کند. در طول فرایند پردازش، آرایه‌های سیستولیک به بازه زمانی تاخیردار نیازمند است تا جریان اولیه داده را به داخل واحدهای پردازشی وارد کند و همچنین داده‌ها را تا گرفتن نتیجه کامل، درون مدار هدایت کند. پس از سپری‌شدن دقیق چرخه عملیاتی پردازش، وضعیت عوض شده و پروسه به سمت جمع‌آوری نتیجه (Writeback) تغییر می‌یابد.

عملیات لایه‌ی شبکه عصبی به سه گروه اصلی داده نیازمند است: ماتریس مقادیر ورودی خام (Feature Map/A)، ماتریس وزن‌های آموزش‌دیده (Weights/B) و بردار بایاس‌ها (Biases/C). در معماری توسعه‌یافته برای این تراشه، ماژول input_loader به‌طور انحصاری نقش واحد حافظه محلی کش^۶ را بازی می‌کند. برای شبیه‌سازی، داده‌های پیش‌فرض ابتدا از طریق فایل‌های مجزا (IN.txt برای دیتا و WB.txt برای مجموعه بایاس و وزن) خوانده می‌شوند. پس از فرمان FSM برای خواندن (Load enable)، فوراً تک‌تک بلوک‌های ۸ بیتی ذخیره‌شده به صورت موازی بر روی یک درگاه بسیار پهن تحت عنوان باس مسطح^۷ گسترانده می‌شوند. در همین چرخه، زیرمجموعه درایه‌های مربوط به ماتریس بایاس (خانه‌های ۱۶ الی ۳۱) کاملاً جدا شده و به صورت برداری در رجیسترهای انباشت‌گر در هسته‌های نهایی قرار می‌گیرند.

با آماده‌سازی اعداد در ورودی شبکه اصلی پردازش، وارد مهم‌ترین فرآیند زمان‌بندی انتقال در سیستم‌های خط‌لوله می‌شویم. به دلیل ذات دوبعدی شبکه و تبادل همسایه‌محور عناصر آن، داده‌ها نباید به صورت موازی و با هم وارد بخش پردازشی شوند.

¹ controller

² input_loader

³ systolic_array

⁴ output_collector

⁵ Pipeline

⁶ Cache Interface

⁷ Flat Bus

المان‌ها تنها مجازند اطلاعات خود را در سیکل بعد به خانه همسایه بدهند. جهت حل سخت‌افزاری این فرایند، سیستم به نحوی طراحی شده که داده‌ها بصورت زمان‌بندی مورب یا کجی داده‌ها^۱ بر روی خروجی‌های خود قرار می‌گیرند.

$$Delay_{(Data_Arrival)}^{i,j} = Offset_i(Clock) + Offset_j(Clock) \propto (i + j)$$

بدین صورت که هرکدام از مسیرها منتظر شماره شمارنده سیستم می‌شوند تا از مقدار آدرس فضایی آن‌ها بزرگ‌تر یا مساوی شود؛

$$Start_Shift_{(i)}: if(Cycles \geq i \&\& (Cycles - i) < N) \rightarrow Gate = Valid$$

به این مفهوم که در ثانیه و کلاک اول اجرای Stream صرفاً خانه ابتدایی از ردیف صفرم و ستون صفرم اجازه جریان‌یافتن پیدا کرده، و باقی ورودی‌ها صفر تزریق می‌کنند و در کلاک دوم یک موج جدید شروع می‌شود و اجازه ورود به سطر اول جدید (خانه پایین دستش) صادر می‌شود. این پلکان تأخیر فیزیکی مثل یک مکانیزم پله‌ای چیده شده تا اطلاعات هرکدام به گونه‌ای سرازیر شوند که پس از رسیدن به PE درون کلاک متقاطع، در یک گره متناسب ظاهر شوند و همزمانی از بین نرود. هنگامی که جریان تنظیم‌شده و پلکانی داده‌ها جدا شد، سرانجام به شبکه سیستمولیک سرازیر می‌گردد. توصیف سخت‌افزار شبکه با بهره‌گیری از یک منطق تکرارشونده فضایی^۲ بنا شده تا آرایه‌ای دقیق با عرض پارامتری M در طول پارامتری N بسازد. در معماری مدرن بلوک پردازشی برای رفع تأخیر طول مسیر، از به‌کارگیری بلوک گند ضرب‌کننده‌ی زنجیره‌ای، اجتناب شده است. سیستم از ساختار سریع و پیشرفته‌ی ضرب درختی^۳ استفاده می‌کند.

البته خروجی به‌دست آمده از مرحله ضرب درخت‌والاس، ۱۶ بیتی است و برای محافظت از سرریز در تجمیع، از فیلتر افزایش عرض باند و گسترش امن علامت عبور کرده و وارد فرمت ایمن ۱۸ بیتی می‌گردد. حالا پاسخ ضرب در هر خانه آماده نشستن درون بلوک حافظه‌ی حاصل جمع است؛ اینجا نیز جهت پرهیز از اتلاف پالس زمان از جمع‌کننده‌ی زنجیره‌ای عادی پرهیز گردیده و واحد محاسباتی پیش‌بین نقلی یا مدار انباشت (CLA-18Bit) طراحی شده تا محاسبات مسیر بدون تأخیر انجام پذیرد.

$$Accumulate_{(cycle\ T+1)} = (CLA_{Adder_18bit}(PE_{psum[T]}, \{2^{bit_ext}, Wallace_{mult}(A, B)\}))$$

سرانجام آخرین وضعیت مدار کنترلی سخت‌افزار، ماژول دریافت‌گر جریان سریال خروجی (output_collector) است. بعد از اتمام پردازش برای یک استنتاج در شبکه‌ی PE، خروجی‌ها در آکومولاتورهای سیستمولیک تولید شده است؛ منتها دریافت همزمان همه‌ی آن‌ها یعنی لزوم قرار دادن صدها درگاه باس موازی و فیزیکی! این ماژول، به‌سادگی و منطبق بر نیاز خط لوله یک راهکار حافظه‌ی عکس‌بردار(!) ساخته است؛ هنگامی که وضعیت سیگنال منطقی (WRITEBACK) بالا رود ماژول مقادیر آرایه دوبعدی ۱۸ بیتی را همانند گرفتن عکسی ناگهانی یک پارچه درون رجیستری مسدود می‌گرداند و تا چرخه‌ی جدید محاسبه دیگر، داده‌ها را روی ۱۸ بیت رشته به همراه آدرس درایه‌ها در درون ماتریس، به محیط پردازنده ارسال میکند تا فرآیند با موفقیت خاتمه یابد.

¹ Data Skewing

² Generate-Loop

³ Wallace Tree Multiplier

فصل چهارم: بررسی معماری بلوک‌های پایه: ضرب کننده و جمع کننده

۴-۱) ضرورت طراحی تفکیک یافته مدارهای سنتزپذیر ضرب و جمع

بخش اصلی در هر شبکه عصبی سخت‌افزاری و آرایه‌های سیستمولیک، واحد پردازشی (PE) است. وظیفه اصلی هر عنصر پردازشی در معماری پیاده‌سازی شده، محاسبه‌ی یک عملیات پایه به نام ضرب و انباشت (MAC) می‌باشد.

$$Accumulation(y_t) = y_{t-1} + (A_{input} \times B_{weight})$$

در کدنویسی با زبان‌های توصیف سخت‌افزار مانند Verilog، می‌توان یک عملیات MAC را به صورت رفتاری^۱ و تنها با یک خط کد(!) پیاده‌سازی کرد؛ با این حال، استفاده از این روش یکپارچه در سطح انتقال رجیستر (RTL) به هیچ وجه مطلوب مهندسان طراح مدار مجتمع نیست!

دلیل اول این موضوع، عدم تسلط طراح بر ساختار نهایی مدار پس از فرآیند سنتز (Synthesis) است. ابزارهای سنتز برای تفسیر عملگرهای ریاضی پیوسته، از کتابخانه‌های پیش فرض خود استفاده می‌کنند که ممکن است از نظر مصرف توان، مساحت و به ویژه زمان‌بندی مسیر بحرانی، در حالت بهینه قرار نداشته باشند. از آنجایی که در این پروژه یک ماتریس پردازشی با ابعاد متغیر ایجاد شده است و بلوک‌های PE به تعداد زیاد تکثیر می‌شوند، حتی وجود یک گیت منطقی اضافه در عملیات جمع یا ضرب، منجر به افزایش چشمگیر توان مصرفی کل تراشه خواهد شد.

به همین جهت روش تعیین شده در این طراحی، تفکیک کامل واحدهای محاسباتی به ماژول‌های مستقل است. در معماری ارائه شده، به جای انجام جمع و ضرب هم‌زمان درون کلاک پردازنده، دو بلوک پایه یعنی ضرب کننده‌ی علامت دار ۸ بیتی و جمع کننده‌ی ۱۸ بیتی به صورت کاملاً ماژولار و تفکیک یافته طراحی و سپس درون هر واحد PE فراخوانی^۲ شده‌اند. مزیت بزرگ دیگر این طراحی تفکیک یافته، فراهم کردن بستر مناسب برای بررسی معماری‌های مختلف در مراحل طراحی مدار است. این رویکرد به ما اجازه می‌دهد تا عملکرد بلوک‌های مختلف را (همانند مقایسه‌ی معماری ضرب کننده و جمع کننده‌های مختلف که در الزامات طراحی مطرح شده است) در سطح زیرماژول ارزیابی و شبیه‌سازی کنیم، بدون آن که نیاز باشد منطق کلی کنترلر یا زمان‌بندی کل سیستم در سطح پردازنده تغییر یابد. بدین ترتیب، ضمن توسعه‌پذیری پروژه، صحت‌سنجی ماژول‌ها پیش از یکپارچه‌سازی در سطح آرایه نیز با اطمینان بسیار بالایی صورت می‌گیرد.

۴-۲) تحلیل مهندسی معماری (Adder Architecture)

با توجه به محاسبات و تحلیل‌های صورت گرفته در تعیین طول داده (دیتا)، مشخص شد که دامنه مقادیر خروجی بلوک جمع و ضرب به حداقل ۱۸ بیت پهنای باند برای ذخیره‌سازی نیاز دارد. از این رو، وظیفه‌ی اصلی بخش جمع کننده در این پردازنده، انباشت حاصل ضرب ۱۶ بیتی گسترش یافته با مقادیر بایاس و یا مقادیر انباشت شده‌ی قبلی ۱۸ بیتی در طول چرخه‌های کلاک است. در طراحی بلوک جمع کننده‌ی یک آرایه سیستمولیک، باید این موضوع را در نظر داشت که به دلیل عملیات خط‌لوله‌ای و تکثیر گسترده این واحدها در کل مدار، بلوک جمع کننده نقش تعیین کننده‌ای در تاخیر مسیر بحرانی^۳ تراشه خواهد داشت. به همین دلیل بررسی و مهندسی معماری جمع کننده‌ها یک ضرورت جدانشدنی برای بهینه‌سازی سرعت مدار است.

¹ Behavioral

² Instantiate

³ Critical Path Delay

۱-۲-۴) مقایسه ساختار و تاخیر مدارهای جمع کننده RCA در برابر CLA

برای طراحی مدار جمع کننده ۱۸ بیتی در سطح رجیستر (RTL)، رویکردهای ساختاری متنوعی وجود دارد. ساده ترین و ابتدایی ترین روش پیاده سازی سخت افزاری این بخش، استفاده از جمع کننده های نقلی موجی^۱ است. در ساختار RCA، جمع کننده های کامل تک بیتی به صورت آبشاری و پشت سر هم قرار می گیرند. در این ساختار، تولید خروجی در هر بیت کاملاً وابسته به دریافت رقم نقلی^۲ از بیت قبل تر از خود می باشد. همین وابستگی ساختاری باعث می شود که مدار دارای یک تاخیر زمانی نسبتاً زیادی باشد. از منظر روابط زمانی

$$T_{RCA} = (N - 1) \cdot t_{carry} + t_{sum} \cong O(N)$$

تاخیر کل مدار RCA به طور مستقیم و خطی با تعداد بیت ها (اندازه ورودی) افزایش می یابد؛ به گونه ای که برای یک عدد ۱۸ بیتی، رقم نقلی باید ۱۸ طبقه تاخیر گیت را پشت سر بگذارد تا خروجی نهایی سیستم به حالت پایدار و معتبر خود برسد. وجود چنین تاخیر بالایی به صورت خطی باعث کاهش چشمگیر ماکزیمم فرکانس قابل تحمل پردازنده می شود، چراکه مسیر طی شده ی پالس نقلی از پایین ترین بیت (LSB) به بالاترین بیت (MSB)، طولانی ترین زمان محاسباتی را اشغال می کند و با طبیعت پردازش های موازی و پرسرعت شبکه های عصبی هم خوانی ندارد. به همین جهت، در طراحی خود از معماری بهینه تری به نام مدار نقلی مسبق^۳ بهره می جویم (!).

برخلاف رویکرد موجی و سریالی در مدار RCA، جمع کننده CLA با استفاده از مفاهیم پیش بینی منطقی، تاخیر ناشی از زنجیره ی نقلی ها را به طور کلی درهم می شکند. ساختار این جمع کننده، دو سیگنال بنیادی و تولید کننده ی پایه تحت عنوان سیگنال تولید نقلی و سیگنال انتقال نقلی ایجاد می کند

$$\begin{cases} P_i = A_i \oplus B_i \\ G_i = A_i \cdot B_i \end{cases}$$

در واقع هر یک از این سیگنال ها بر اساس مقدار دو ورودی مدار و صرفاً با گذار از تنها یک طبقه گیت منطقی محاسبه می شوند. بدین طریق به جای آنکه هر بیت منتظر جواب نقلی بیت ماقبل خود بماند، با استفاده از جبر بولی بسط داده شده، محاسبات پایه ی سیگنال نقلی برای تمام طبقه ها به صورت کاملاً موازی

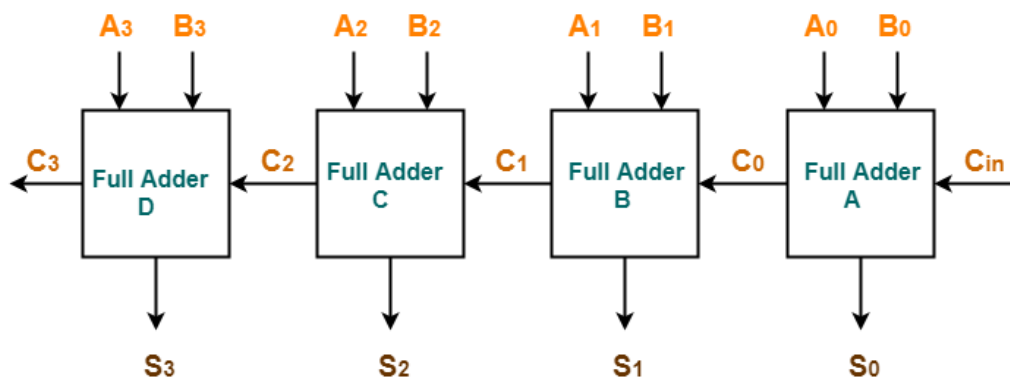
$$C_{i+1} = G_i + (P_i \cdot C_i)$$

تعیین می گردد. استفاده از این ساختار درخت وار و پیش بینی کننده باعث می شود که وابستگی شدید تاخیر محاسبات به طول بیت از حالت خطی (RCA) خارج شده و مدار با کمترین میزان تلفات زمانی عمل انباشت را انجام دهد. از منظر تاخیر و فرکانس کاری، طراحی با ساختار منطقی CLA برتری قطعی و عملکرد بدون بحثی نسبت به شبکه ی نقلی کلاسیک RCA به دنبال دارد. (البته با در نظر گرفتن این مسئله که سرعت مهم تر از مساحت در این زمینه است!)

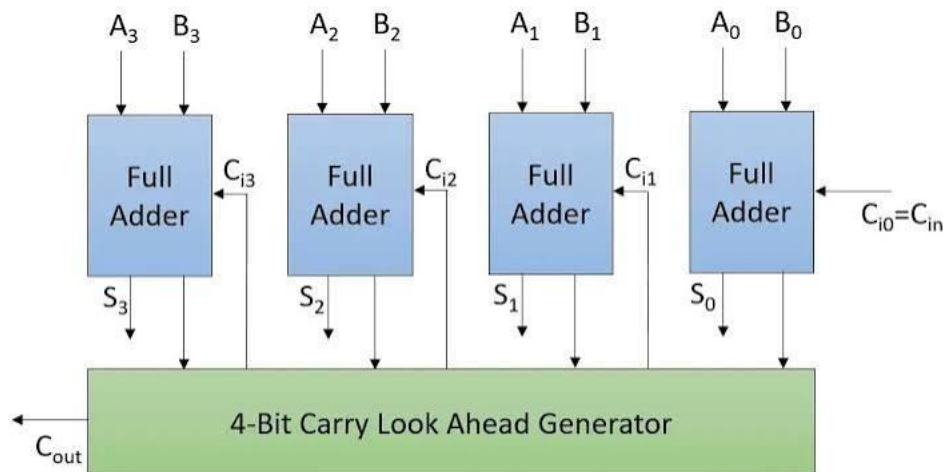
¹ Ripple Carry Adder - RCA

² Carry

³ Carry Lookahead Adder - CLA



4-bit Ripple Carry Adder



4-bit Carry Look Ahead Adder

شکل ۴-۱) مقایسه نمودار ساختاری اتصال زنجیروار و سریالی جمع کننده نقلی موجی (RCA) با شبکه‌ی منطقی و موازی جمع کننده نقلی مسبق (CLA) و کاهش طول مسیر رقم نقلی

۴-۲-۲) پیاده‌سازی سخت افزاری و اثبات دلایل مهندسی برای استفاده از ساختار CLA

طراحی یک شتاب‌دهنده مبتنی بر آرایه سیستمولیک، به خصوص در مواردی که پایلین کردن از معیارهای مهم طراحی محسوب می‌شود، نیازمند حداکثر دقت در انتخاب ساختارهای درون هسته‌ی محاسباتی (PE) هستیم. دلیل اصلی انتخاب ساختار جمع کننده پیش‌بینی کننده رقم نقلی (CLA) به جای معماری زنجیره‌ای موجی (RCA) در این پروژه، ماهیت زمان‌بر بودن انباشت داده‌های ۱۸ بیتی در پایان چرخه‌ی پردازشی است.

زمانی که پهنای باند جمع کننده پایین باشد (مثلاً ۴ بیت)، اختلاف زمانی عملکردی بین دو مدار نام‌برده تا حدود زیادی نامحسوس است. اما زمانی که برای پیشگیری از سرریز حاصل ضرب‌ها، داده‌ها به یک فضای ۱۸ بیتی انتقال داده می‌شوند، گسترش آبخاری تاخیرها در RCA فاجعه‌برانگیز (!) می‌شود. به همین منظور برای سرعت پردازش بالا، بهینه‌سازی منطقی سیستم به دسترسی آنی تمام بیت‌ها به سیگنال نقلی وابسته می‌شود... این موضوع ثابت می‌کند که استفاده از یک مدار کاملاً ماژولار و سطح-رجیستر از خانواده CLA برای ارتقاء پارامتر سرعت عملیاتی تراشه یک تصمیم مهندسی به شمار می‌رود.

در ماژول توصیف سخت‌افزاری cla_18bit، روش از حالت توصیف رفتاری (استفاده‌ی ساده از عملگر +) کاملاً به سمت طراحی ساختاری^۱ جبر بولی پیش برده شده است. این روند به گونه‌ای طراحی شده که توزیع سیگنال‌ها بر پایه روابط گیت‌های پایه صورت گیرد.

¹ Structural Definition

ابتدا بر اساس مقدار هر کدام از ۱۸ بیت دیتای ۱۸ بیتی مقادیر A و مقادیر (B)، دو گذرگاه سیگنال تحت عناوین تولید و انتقال، ارزیابی می‌شود. به عبارت مهندسی، سیگنال انتقال^۱ از طریق ترکیب موازی دو مقدار بر روی گیت $XOR (a \oplus b)$ به طول کامل ۱۸ بیت تعریف می‌شود، برای ارزیابی سیگنال تولید^۲ هر درایه صرفاً از یک اتصال سطح بیت گیت منطقی پایه و یا $AND (a \& b)$ بهره برداری شده است.

$$\forall i \in \{0,1, \dots, 17\} \rightarrow \begin{cases} \text{prop}[i] = a[i] \oplus b[i] \\ \text{gen}[i] = a[i] \cdot b[i] \end{cases}$$

بدیهی است که عملیات فوق برای ۱۸ عنصر یک ساختار موازی و پرسرعت مهیا می‌کند. پس از آماده شدن تمامی بیت‌های مستقل ارزیابی (تولید و انتقال)، مدار نیازمند توسعه بیت‌های نقلی خواهد بود. بدین منظور کد نوشته شده، عملیات تولید سیگنال حامل مقادیر تولیدی از یک بیت تا گسترش شبکه ۱۸ بیتی موازی را تضمین می‌کند.

$$\forall i \in \{0,1, \dots, 17\}: C[i+1] = \text{gen}[i] + (\text{prop}[i] \cdot C[i])$$

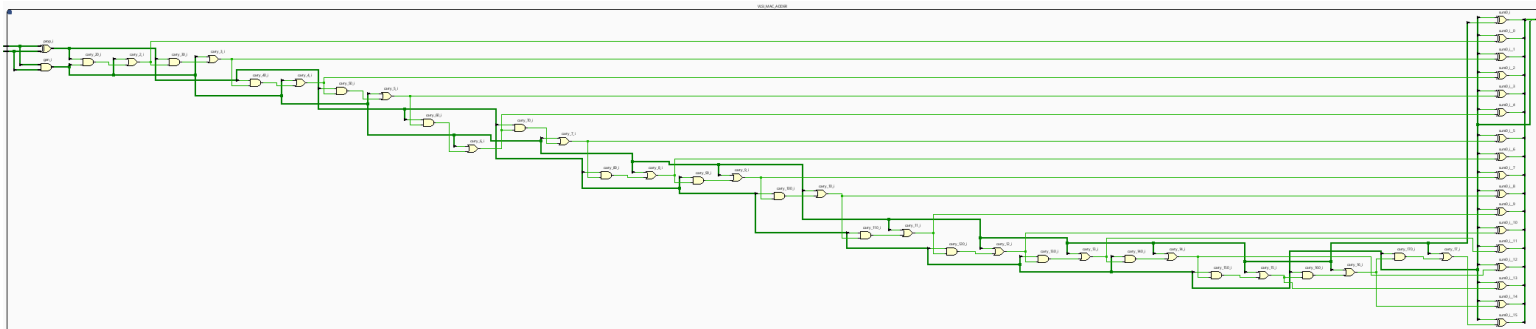
درون این حلقه سخت افزاری generate مقدار هر نقلی به فرم بسط داده شده ترکیب و نتیجه مستقیماً بدون طی کردن دروازه‌های منطقی قبلی فقط متکی بر جبر دودویی خروجی مدار را تعیین می‌کند.

$$\forall i \in \{0,1, \dots, 17\}: \text{sum}[i] = \text{prop}[i] \oplus C[i]$$

```
module cla_18bit (
    input [17:0] a,
    input [17:0] b,
    output [17:0] sum
);
    wire [17:0] gen = a & b;
    wire [17:0] prop = a ^ b;
    wire [18:0] carry;
    assign carry[0] = 1'b0;

    genvar i;
    generate
        for(i = 0; i < 18; i = i + 1) begin: carry_lookahead
            assign carry[i+1] = gen[i] | (prop[i] & carry[i]);
            assign sum[i] = prop[i] ^ carry[i];
        end
    endgenerate
endmodule
```

شکل ۲-۴) کد توصیف معماری جمع کننده ۱۸ بیتی نقلی مسبق (CLA)



شکل ۳-۴) شماتیک سطح پایین جمع کننده ۱۸ بیتی نقلی مسبق (CLA)

¹ Propagate

² Generate

۳-۴) تحلیل مهندسی معماری ضرب کننده (Multiplier Architecture)

سنگین ترین، توان مصرف کننده ترین و البته پیچیده ترین ماژول سخت افزاری در هسته پردازشی یک آرایه (PE)، عملگر ضرب می باشد. در این پروژه سیستم نیازمند اعمال ضرب وزن ها و داده های ورودی ۸ بیتی به صورت علامت دار است که نتیجه آن یک عدد ۱۶ بیتی است و مستقیماً وارد بلوک جمع کننده خواهد شد. چالش اصلی طراح مدارهای دیجیتال در پردازش های سنگین ریاضی (مانند شتاب دهنده سخت افزاری ماتریسی)، مدیریت هم زمان دو فاکتور اصلی در طراحی مدار مجتمع، یعنی تاخیر مسیر بحرانی و هزینه مساحت/مصرف توان است. فرآیند ضرب در مدارهای دیجیتال به طور کلی از دو مرحله اساسی تشکیل می شود: ابتدا تولید حاصل ضرب های جزئی^۱ بین تمام بیت های ورودی و دوم، جمع کردن و فشرده سازی این حاصل ضرب های جزئی برای دستیابی به خروجی نهایی. در ادامه، جهت رسیدن به بهترین راندمان از نقطه نظر طراحی سطح انتقال رجیستر، دو ساختار متداول ضرب کنند سخت افزاری یعنی آرایه ای و درختی بررسی شده است.

۳-۴-۱) مقایسه ساختار ضرب کننده آرایه ای (Array) نسبت به درخت ضرب والاس (Wallace Tree)

ساختار ضرب کننده آرایه ای (Array Multiplier):

این معماری یکی از شناخته شده ترین و پرکاربردترین الگوهای طراحی برای انجام ضرب سخت افزاری است. نحوه محاسبه در ضرب آرایه ای شباهت بسیار زیادی به روش محاسبات ریاضی جمع بر روی کاغذ در روش دستی دارد. خروجی در یک ساختار تماماً شبکه وار تولید می شود. بدین ترتیب که تولید و جمع شدن تمامی حاصل ضرب های جزئی درون بلوک هایی از گیت های AND منطقی و سلول های جمع کننده ی کامل و نیمه (Full Adder و Half Adder) انجام می پذیرد. مهم ترین نقطه قوت معماری آرایه ای برای مهندسان سیلیکون در جانمایی و ترک کشی (!) مدارهای مجتمع نمایان می شود؛ چرا که کابل کشی^۲ داخل تراشه و ساختار بلوکی این الگو، به شدت منظم است. از نظر پیاده سازی بلوک های ساختمانی پایه روی تراشه ی واقعی بسیار به هم پیوسته، منظم و با تراکم بسیار بالا در کنار هم قرار می گیرند و بنابراین جانمایی بهینه ای از خود نشان می دهند. البته عیب اصلی در ساختار خطی و ساده گرایانه این مدار قرار دارد... چون از نظر پردازشی، تولید جواب دارای زمان و تاخیر نسبتاً بالایی می باشد. به دلیل اینکه مدار کاملاً آرایه ای طراحی شده است تاخیر سیستم دارای رشدی تقریباً متناسب به صورت خطی (خطی متناسب با رشد بیت ورودی) است.

$$Delay_{Array} \propto O(N) \rightarrow \cong [(N-1) \cdot T_C + N \cdot T_S]$$

و بنابراین تاخیر پردازنده کاملاً بستگی به عبور موج نقلی (سیگنال حاصل شده) از گیت ها دارد.

ساختار درخت والاس (Wallace Tree Multiplier):

طراحی درختی روشی پیشرفته و منحصراً تمرکز شده روی زمان محاسباتی و فرکانس های بالای کاری است. به این نحو که فاز تجمیع مقادیر درهم ریخته نتایج با تعداد گسترده ای از جمع کامل استفاده می شود بدون آنکه مقادیر خروجی نیازی داشته باشند زنجیر نقلی یکپارچه ایجاد نمایند.

$$stages(h) \rightarrow \text{for } n > 2: N_{k+1} = 2 \cdot \left\lceil \frac{N_k}{3} \right\rceil + (N_k \bmod 3)$$

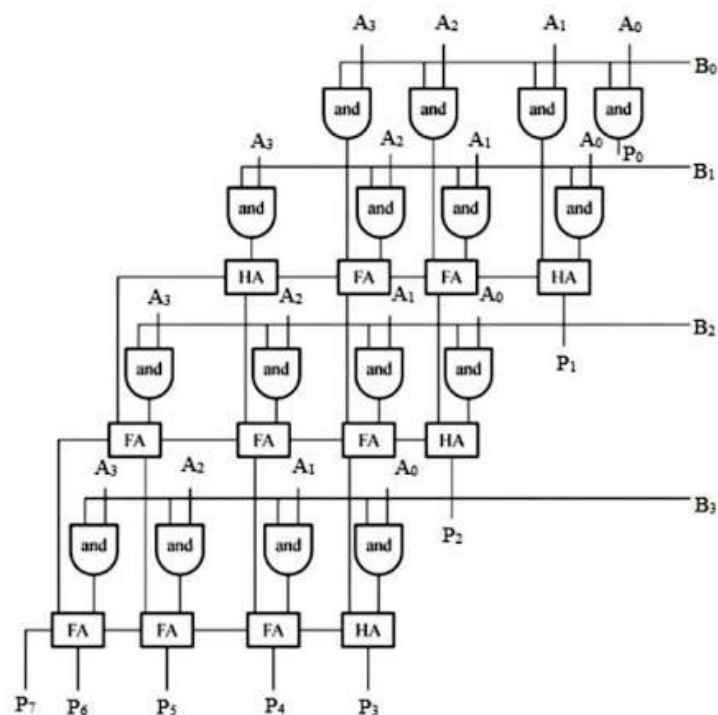
مزیت بزرگ روش ضرب درختی والاس، تاخیر به شدت پایین در مقیاس رشد متناسب لگاریتمی را است.

$$Delay_{Wallace} \propto O(\log_{3/2}(N))$$

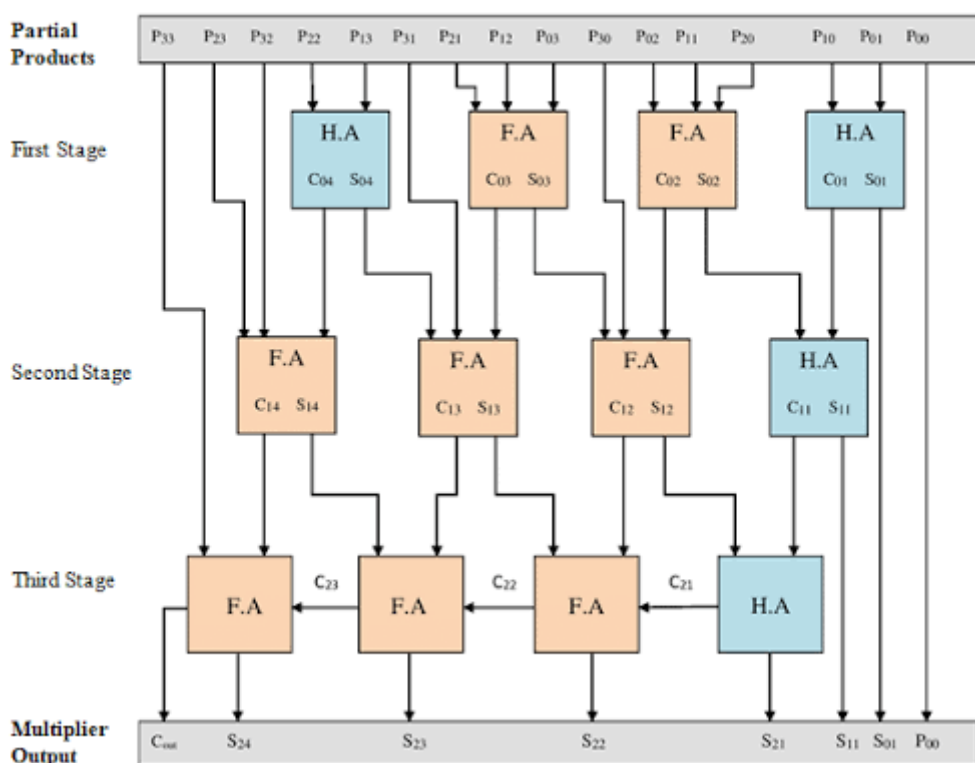
^۱ Partial Products

^۲ Routing

اما نقطه منفی این معماری، کابل‌کشی به شدت بی‌نظم در طرح سخت افزاری بلوک است. ماهیت درختی بودن اتصال رجیسترهای مدار نیازمند حجم به شدت زیادی ترک(!) است، این مساله سبب می‌گردد در زمان قرارگیری کامپایل شده روی چیدمان چیپ یک تراکم اتصال غیر قابل تصور در سیلیکون رخ بدهد که همواره سبب پدیده‌های ناخواسته چون ازدحام ارتباطی در پروسه Layout می‌شود که مساحت غیر مفیدی به بار خواهد آورد.



شکل ۴-۴ طراحی ضرب کننده با ساختار آرایه‌ای (Array Multiplier)



شکل ۴-۵ طراحی ضرب کننده با ساختار درخت ضرب والاس (Wallace Tree Multiplier)

۲-۳-۴) استدلال سازگاری بالا برای انتخاب ساختار درخت والاس و بررسی پیاده‌سازی سخت‌افزاری آن

با بررسی و مقایسه‌های تئوری انجام شده میان دو معماری مختلف، متوجه این موضوع شدیم (!) که پردازنده‌ی به کار رفته در سیستم‌های سیستولیک و عصبی، ذاتاً در هر کلاک پردازشی، مقادیر سنگینی از اعمال جبری را به رجیسترها تزریق می‌کنند. در چنین محیط‌هایی که عملیات روی هر هسته نیازمند تکرار سریع و همگام است، گلوگاه اصلی چیزی جز طول مسیر بحرانی نخواهد بود. به همین دلیل ساختار معماری درخت ضرب والاس با دید مهندسی به پروژه اضافه گردید تا محدودیت افزایش خطی زمان تاخیر را در برطرف کند و سازگاری زمانی بیشتری در خط لوله آرایه‌ها بوجود بیاورد.

به منظور جلوگیری از ابهام ابزار سنتز، بلوک پایه‌ای به نام جمع‌کننده‌ی حفظ نقلی (csa_16bit) توسط توصیف ساختاری ایجاد شد. سلول‌های منطقی پایه CSA به نحوی هستند که سه ورودی مستقل می‌پذیرند و به جای محاسبه سریال و توالی‌دار ارقام نقلی در طبقه‌ها، بدون ذره‌ای درنگ جواب را صرفاً درون دو متغیر مجموع مستقل (Sum) و بردار نقلی (Carry) ذخیره کرده و به سطح بعد انتقال می‌دهند. با استخراج دو رابطه‌ی مستقل

$$\begin{cases} \text{Sum}_i = x_i \oplus y_i \oplus z_i \\ \text{Carry}_{i+1} = (x_i \cdot y_i) \vee (x_i \cdot z_i) \vee (y_i \cdot z_i) \end{cases}$$

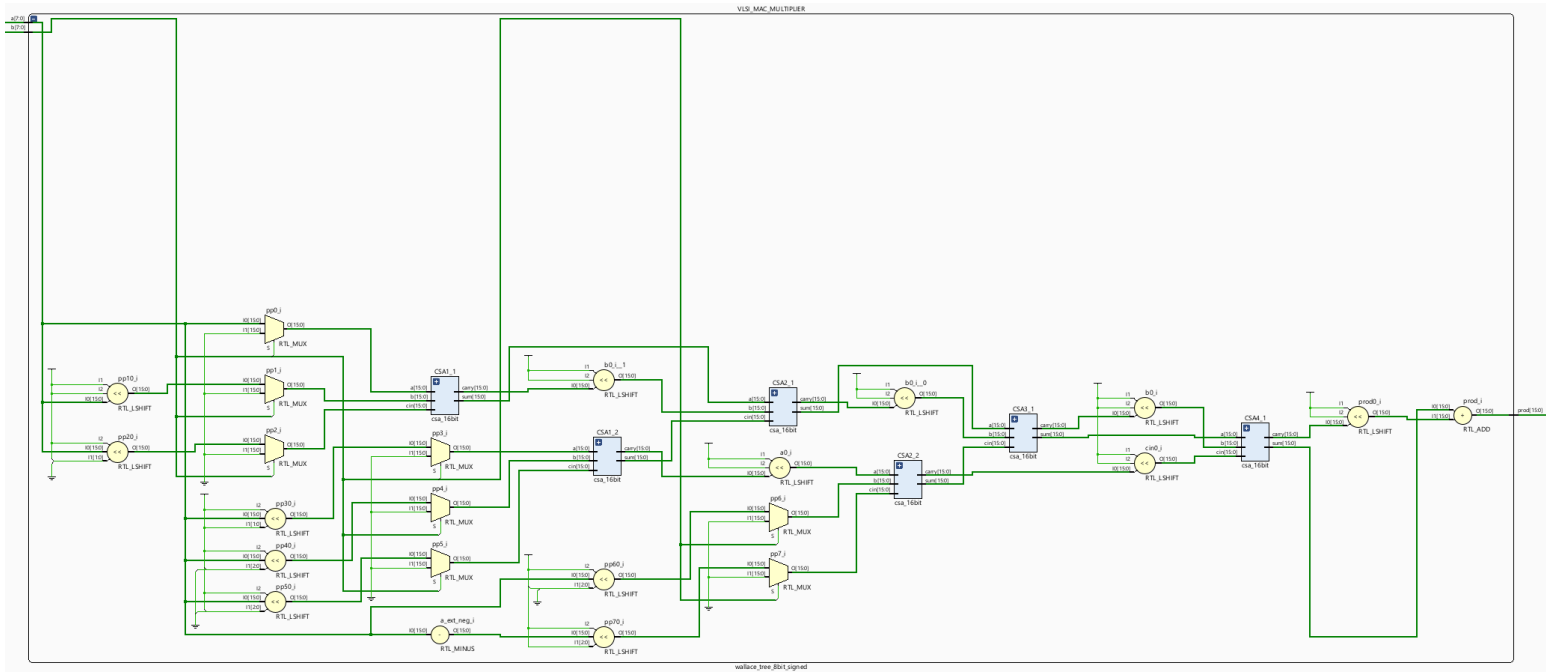
و عدم وابستگی آن‌ها در مسیر طی‌شده برای بیت‌های همسایه، تاخیر هر طبقه‌ی شانزده بیتی در فشرده‌سازی برابر با تاخیر گذر فقط یک گیت منطقی ثابت خواهد بود.

$$\text{Delay}(CSA_{\text{stage}}) = \mathcal{O}(1) \Rightarrow (T_{\text{XOR}}) + (T_{\text{AND}} \vee T_{\text{OR}})$$

برای طراحی ماژول هسته یعنی ضرب ۸ بیتی علامت‌دار (wallace_tree_8bit_signed)، اولین بخش پیاده‌سازی شده، بخش استنتاج حاصل ضرب‌های جزئی می‌باشد. برای پشتیبانی از اعداد علامت‌دار، هفت درایه ابتدایی (b[0] تا b[6]) به عنوان شیفت و تولید جزئی ساده رفتار می‌کنند. اما استخراج آخرین درایه جزئی یا شیفت هفتم که نمایانگر وزن منفی‌کننده‌ی متغیر در ضرب مقادیر علامت‌دار در نظر گرفته می‌شود، با ترکیب مفهوم عملگر مکمل دو محاسبه می‌شود. خروجی در آن سطر ابتدا نسبت داده شده، سپس برای جبران اختلاف جبری قرینه می‌گردد.

$$P_7 = (-A_{\text{extended}} \times 2^7) \cdot B[7]$$

این هشت سطر متوالی که آماده فشرده شدن در مرحله تقاطع هستند، مستقیماً توسط طبقات CSA تحت الگوهای مرحله‌ای شیفت‌داده‌شونده به چپ، کوچک می‌شوند.



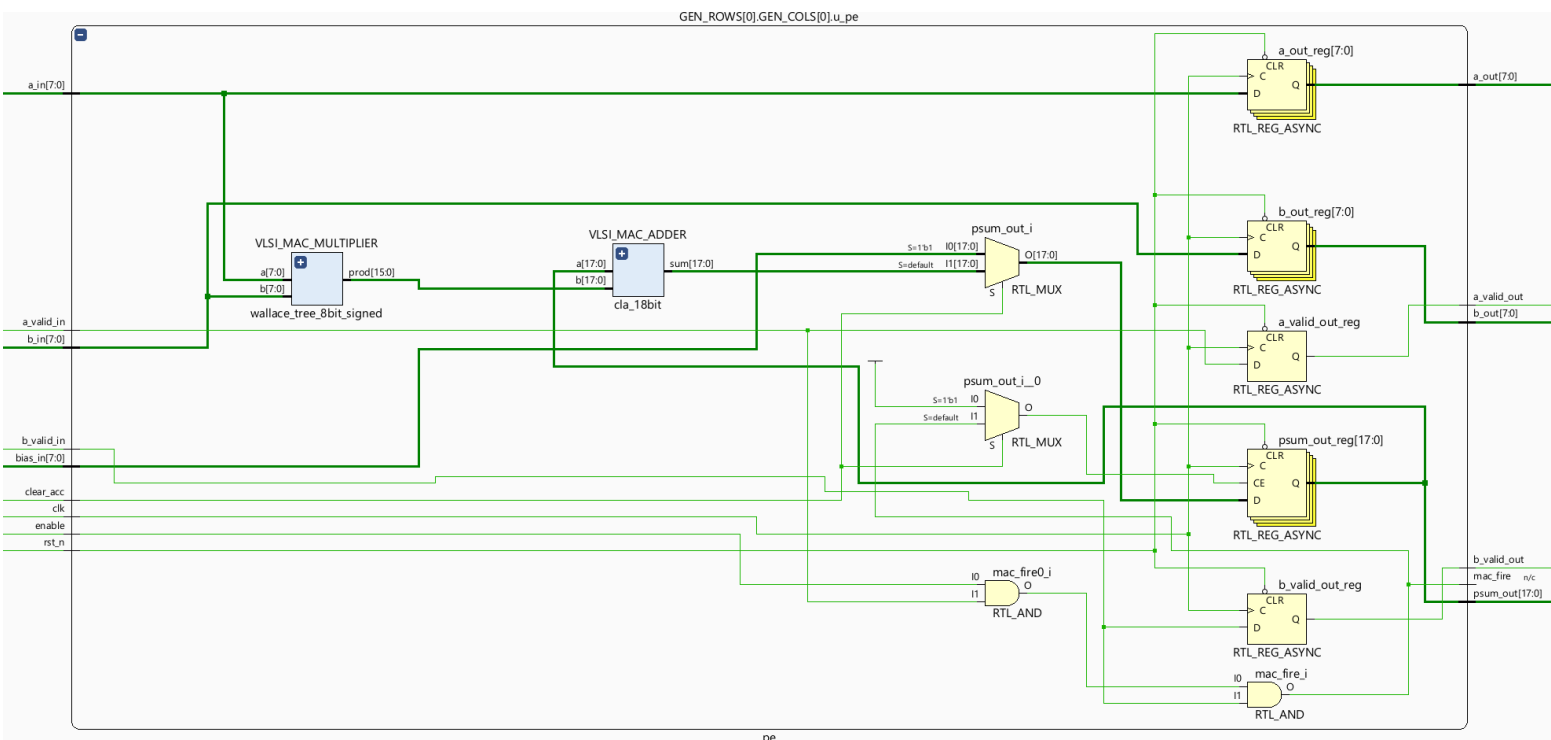
شکل ۴-۶) شماتیک طراحی شده ضرب کننده با ساختار درخت ضرب والاس (Wallace Tree Multiplier) و فراخوانی ماژول پایه (CSA) با متد درخت وارونه جهت جمع جزئی اعداد

فصل پنجم: معماری عنصر پردازشی (PE) و تحلیل ابعاد داده

۵-۱) فلوچارت انتقال مسیر داده و اتصال دهی بلوک‌های محاسبه داخل عنصر محاسباتی

عنصر پردازشی (PE) به عنوان واحد پایه محاسباتی در ساختار آرایه سیستمولیک شناخته می‌شود. وظیفه اصلی این المان، انجام همزمان عمل‌های ضرب و انباشت جمع (MAC) روی داده‌های ورودی و سپس انتقال مقادیر برای سیکل‌های بعدی است. در این پروژه، طراحی مسیر داده^۱ درون این المان به گونه‌ای انجام شده است که بیشترین توان عملیاتی در کنار کمترین تأخیر انتشار مسیر بحرانی^۲ بدست آید.

جهت ورود و خروج سیگنال‌ها در المان پردازشی کاملاً منظم و بر اساس حالت پردازشی سیستمولیک تنظیم شده است. به این صورت که داده‌های ماتریس اول (a_{in}) به شکل افقی و از چپ به راست حرکت کرده و داده‌های ماتریس دوم (b_{in}) به شکل عمودی و از بالا به پایین منتشر می‌شوند.



شکل ۵-۱) شماتیک پین‌های ورودی/خروجی و مسیر انتشار موج‌داده در یک عنصر پردازشی (PE).

روند جریان داده درون ماژول به محض اعمال ورودی‌ها به صورت بلوک‌های متوالی (آبشاری) انجام می‌پذیرد. فلوچارت و اتصال دهی منطق درون این المان از دو بخش منطق ترکیبی (برای محاسبات) و منطق ترتیبی (برای ذخیره‌سازی نتایج) تشکیل شده است.

ابتدا داده‌های ۸ بیت علامت‌دار وارد بلوک ضرب‌کننده درختی والاس می‌شوند. این بلوک برخلاف ضرب‌کننده‌های ساده، نیازی به کلاک برای پردازش‌های میانی ندارد و نتیجه‌ی ضرب را به صورت ۱۶ بیت و تماماً به فرم ترکیبی(!) در اختیار سیستم قرار می‌دهد.

^۱ Datapath

^۲ Critical Path

برای حفظ دقت در زمان انباشت داده‌ها و جلوگیری از تداخل ابعاد، حاصل ضرب ۱۶ بیتی مستقیماً به یک بلوک تطبیق ابعاد یا گسترش علامت^۱ متصل می‌شود تا به پهنای باند ۱۸ بیت ارتقا یابد. در مرحله بعدی شبکه مسیر داده، این مقدار ۱۸ بیتی وارد بلوک جمع‌کننده پیش‌بین نقلی (CLA) می‌شود. جمع‌کننده در این مقطع مقدار حاصل ضرب سیکل فعلی را با مقدار ذخیره شده از محاسبات مراحل پیشین انباشت می‌کند. از نظر ریاضی، مسیر طی شده در داخل این المان سخت‌افزاری بر اساس رابطه محاسباتی عملگر انباشت ضرب

$$psum_{new} = psum_{old} + (A_{in} \times B_{in})$$

تعریف می‌گردد.

اگر دستور پاکسازی و مقداردهی اولیه (clear_acc) صادر شود، مقدار بایاس اولیه ماتریس متصل به این المان مستقیماً (به عنوان عدد ۱۸ بیت با اعمال گسترش علامت) درون انباشت‌گر مقداردهی می‌گردد. از طرف دیگر برای اجرای سیکل به سیکل جمع و ضرب، در صورت فعال بودن شبکه (enable) و معتبر بودن داده‌ها (a_valid و b_valid)، یک سیگنال کنترلی به نام آتش‌موج‌داده (MAC Fire) ایجاد می‌گردد. تا زمانی که این شرط محقق باشد، خروجی‌ها از مسیر جمع‌کننده مجدداً داخل انباشت‌گر قرار گرفته و همزمان، داده‌های اصلی با یک سیکل تأخیر، بدون هیچ تغییری، روانه‌ی المان‌های مجاور برای ادامه‌ی محاسبات سیستمولیک می‌شوند.

۵-۲) بررسی الزامات سخت‌گیرانه فرمت ابعاد متغیرها برای عملیات MAC

در طراحی سیستم‌های دیجیتال، تعریف دقیق و مهندسی‌شده عرض بیت‌ها^۲ یا همان ابعاد داده، یکی از مهمترین بخش‌های طراحی است. اگر ابعاد باس‌های ارتباطی بیش از حد نیاز در نظر گرفته شود، منابع سخت‌افزاری مانند المان‌های منطقی (LUTs) و رجیسترها به هدر می‌روند که در نهایت باعث افزایش مصرف توان و اشغال مساحت اضافی در تراشه می‌شود. از طرف دیگر، اگر پهنای بیت‌ها کوچک‌تر از میزان لازم انتخاب شود، سیستم دچار از دست دادن دقت اعشار و پدیده‌هایی نظیر سرریز عددی خواهد شد.

```
module systolic_array #(
    parameter M = 4,
    parameter N = 4,
    parameter DATA_WIDTH = 8,
    parameter ACC_WIDTH = 18
```

شکل ۵-۲) استفاده از رویکرد توصیف پارامتریک در وریلاگ جهت اعمال دقیق فرمت متغیرهای محاسباتی در سیستم.

ساختار پردازشی در این سیستم بر اساس محاسبات ممیز ثابت علامت‌دار^۳ تعیین شده است. داده‌های اولیه تولید شده توسط حافظه‌های سیستم (a_in و b_in)، شامل مقادیر عناصر ماتریس وزن و مقادیر داده‌ی ورودی است. همانطور که قبلاً در فصل سوم اشاره شد، طول کلمه پایه بر اساس الزامات این پروژه، هشت بیت تنظیم شده است. به منظور بهره‌گیری سیستم در

^۱ Sign-Extension

^۲ Bit-Width

^۳ Signed Fixed-Point Format

پردازش مقادیر مثبت و منفی، ساختار نمایش متمم دو^۱ در سرتاسر کد الزامی گشته و ورودی‌ها بصورت علامتدار تعریف میگردند.

$$Range_{Two'sComplement} = [-(2^{n-1}), (2^{n-1} - 1)]$$

ورود داده‌ها به لایه‌ی بعدی یعنی عملگر ضرب هسته، یک تغییر ساختاری بزرگ ایجاد می‌کند. مدار ترکیبی طراحی شده موظف است تا ضرب ماتریسی را بدون فدا کردن دقت اجرا کند. یک اصل در دیجیتال ایجاب می‌کند که برای عدم مواجه با کاهش دقت، عرض رجیستر حافظ خروجی ضرب‌کننده باید دقیقاً معادل حاصل جمع عرض بیت‌های ورودی در نظر گرفته شود.

$$Width_{Multiplier-out} = Width_A + Width_B$$

بنابراین، رنج ابعادی ماژول پردازش با دقت فیزیکی ۱۶ بیت طراحی می‌گردد.

برای امکان پذیر شدن تجمیع سخت‌افزاری و ترکیب مقدار هشت بیتی اولیه شبکه به عنوان پایه، ارزش ۱۶ بیتی ضرب‌کننده و نهایتاً جمع شدن همه آن‌ها روی انباشتگر، مسیر انتقال بین بلوکی باید فرمت ۱۸ بیتی داشته باشد. در ساختار توسعه داده شده برای این شتاب‌دهنده، قبل از برخورد داده‌های میانی ۱۶ بیتی با مقادیر انباشتگر ۱۸ بیتی، با استخراج کردن آخرین بیت معنی‌دار داده (ارزش مکانی تعیین‌کننده‌ی منفی یا مثبت بودن داده یا همان بیت MSB)، عرض باس سخت‌افزاری را بدون تغییر عدد اولیه پهن‌تر(!) می‌سازیم:

$$Extended_{Result}[17:0] = \{2'bSign, 16'bMult_{Data}\}$$

```
// Dynamic scale extension keeping precision accuracy via standard Sign bit-repeating
assign mult_ext = {{(ACC_WIDTH-16){mult_result[15]}}, mult_result};
```

>  mult_result[15:0]	3f01	3f01
>  mult_ext[17:0]	03f01	03f01

شکل ۳-۵) گسترش بیت علامت. عرض ۱۶ بیت فرمت قبلی با گسترش منطقی عدد تبدیل به ۱۸ بیت برای پرهیز از تداخل سبایز ورودی و خروجی ماژول می‌شود.

۳-۵) استدلال نظری و ریاضی محدودیت‌های عددی، بیت‌ها و جلوگیری از پدیده سرریز

یکی از چالش‌برانگیزترین دغدغه‌ها در معماری واحدهای محاسبه‌گر سخت‌افزاری، پدیده سرریز عددی^۲ است. این اتفاق زمانی به وقوع می‌پیوندد که مقدار پردازش‌شده یا تجمیع‌یافته به لحاظ ریاضیاتی از سقف ظرفیت تعبیه شده برای المان ذخیره‌کننده آن تجاوز نماید؛ اتفاقی که سبب تغییر مقادیر محاسبه شده، اشتباه در محاسبه بیت علامت و در نهایت بهم خوردن فرآیند شبکه‌ی عصبی در زمان اجرای فاز استنتاج^۳ می‌گردد. در این بخش دلایل بنیادی انتخاب این سطوح از رجیسترها را در طراحی پایه ماژول به کمک روابط ریاضیات باینری (ممیز ثابت) اثبات می‌کنیم تا از هرگونه بروز سرریز و از دست رفتن اعشار آسوده(!) باشیم.

^۱ Two's Complement

^۲ Overflow

^۳ Inference

۱-۳-۵ تحلیل اثبات کفایت رجیستر ۱۶-بیت علامت‌دار برای حفظ دقت ضرب دو مقدار ۸ بیت

چنانچه پیش‌تر نیز بررسی شد، در مازول عنصر پردازشی هر دوی مقادیر موج داده‌ی ماتریس ورودی و پارامتر وزن ماتریسی با قالب اعداد صحیح علامت‌دار ۸ بیت وارد شبکه‌ی المان می‌گردند. در استاندارد منطق دودویی متمم دو، بزرگ‌ترین عدد مثبت ۸ بیتی برابر $127 = (2^8 - 1)$ و کوچکترین عدد منفی (بیشترین ارزش در ناحیه منفی)، معادل -128 ارزیابی می‌شود.

$$Range_{8-bit} \in [-128, 127]$$

عمل ضرب این مقادیر مجاز به منظور تولید کوچکترین ارزش (در سناریوی بدترین حالت ناحیه‌ی منفی) هنگامی صورت می‌گیرد که یک سیگنال منفی (مثلاً -128) در بالاترین مقدار مثبت (مانند 127) ضرب گردد که به معنای ایجاد عدد -16384 در پایانه خروجی است. به شکل عکس نیز چنانچه دو متغیر شامل بالاترین مرز قدر مطلق باشند (مقدار منفی 128 برای هر دو درگاه متصور باشد)، ضرب آنها ایجادگر عدد مثبت 16384 می‌گردد.

$$\begin{cases} Min: (127) \times (-128) = -16,256 \\ Max: (-128) \times (-128) = 16,384 \end{cases} \Rightarrow P_{out} \in [-16,384, +16,384]$$

نتیجه‌ی بدست آمده بازه‌ای محاسباتی را نشان می‌دهد که برای نمایش عددی مقادیر به صورت صحیح در قالب باینری، متغیر با فرمت ۱۵ بیتی قادر به تامین ظرفیت مورد نیاز آن نخواهد بود، زیرا مرز ثبت مقادیر مثبت برای رجیستر ۱۵ بیتی تنها به ارزش 16383 محدود شده و قادر به در برگرفتن پیک 16384 نیست! بر همین اساس و برای تضمین خروجی معتبر، سائز حداقلی گذرگاه‌ها، مستلزم عرض و ظرفیت فیزیکی معادل رجیسترهای ۱۶ بیت است که فضای کافی دربرگیرنده در هر دو سوی مثبت و منفی ایجاد می‌کند.

۲-۳-۵) تعمیم رجیستر خروجی (Sign-Extension) به رجیستر Accumulator ۱۸ بیت علامت‌دار

ساختار پیش‌فرض پروژه (طبق الزامات) به صورت ابعادی تنظیم شده که به منظور پرشدن و تحویل هر سلول جدید به پردازنده (مقدار $M=N=4$)، قرار است عملیات خروجی شامل مجموع جمع تکرارشونده چهار دوره عمل ضرب ۱۶ بیتی به‌اضافه‌ی بایاس سخت‌افزار باشد که معادل عملگر ریاضی زیر می‌باشد

$$ACC_{sum} = Bias + \sum_{i=1}^{n=4} (A_{in_i} \times B_{in_i})$$

برای بررسی بدترین حالتی که سرریز روی خواهد داد نیاز است محدوده ظرفیت این تجمیع محاسبه گردد. مجموع دامنه برای چهار انباشت متوالی ضرب و اعمال نهایی مقدار افزایشی بایاس (بازه مقداری -128 تا 127)، حد فاصلی از ارزش عددی در نقطه ماکزیمم منفی تا قله‌ی نقطه مثبت می‌سازد.

$$\begin{cases} Range_{min}: (4 \times -16,256) - 128 = -65,152 \\ Range_{max}: (4 \times 16,384) + 127 = 65,663 \end{cases}$$

بسیار واضح است که یک بلوک با ظرفیت معمول ۱۶ بیتی توان نمایش حد نهایی خود (تجاوز از حد $32,767$) را نخواهد داشت! جالب اینجاست که حتی توسعه دادن به عرض رجیستر ۱۷ بیت باز هم کافی نبوده است و از پشتیبانی ارقام بزرگتر از 65535 ناکام می‌ماند و منجر به بروز اختلال داده خواهد شد. لذا اثبات گردید برای پردازش بدون اختلال، رجیسترهای خروجی باید از ۱۸ بیت کامل بهره‌مند گردند.

$$Capacity_{18-bit} \in [-131072, 131071]$$

که هرگونه ناپایداری منطقی را بر اساس روابط مذکور برای داده‌های آرایه و عملگر MAC برطرف می‌کند.

۴-۵) بررسی لچ کردن سیگنال داده و زمان بندی چرخه به چرخه در مسیر (Pipelining)

اساس کارایی ساختار آرایه‌ی سیستمولیک نسبت به پردازنده‌های سنتی، متکی به زمان بندی فضایی و مفهوم پایپ لاین (خط لوله کردن محاسبات) است. در پیاده سازی این سیستم سخت افزاری، هر یک از عناصر پردازشی در عمل نقش یک ایستگاه توقف (لچ) برای انتشار اطلاعات را ایفا می کنند. به عبارت مهندسی، معماری توصیف شده برای این پروژه مسیر ترکیبی طولانی و در مقیاس بزرگ را در بر ندارد که خود مانع از اُفت فرکانس کلاک می گردد؛ بلکه سیگنال های مقادیر داده بین ستون ها و ردیف های ماتریس ها، سیکل به سیکل با لبه های بالارونده پالس کلاک قفل و تثبیت^۱ می شوند.

بررسی عمیق توصیف سخت افزار نشان می دهد که بلوک توزیع کننده داده (input_loader) به جای قرار دادن تمام داده های ماتریس ورودی روی باس های المان در یک چرخه مشخص، مقادیر را به طور شیفت یافته و در حالت اوریب تزریق می کند. دلیل این الزام آن است که مقادیر پردازشی تنها در صورتی معتبر میشوند که سیگنال جریان افقی داده های آرایه نخست با سیگنال عمودی داده های ماتریس دوم به طور متقاطع روی یک ایستگاه به همدیگر برسند.

¹ Latch

فصل ششم: طراحی ماشین حالت (کنترلر سیستم FSM) و مدیریت تاخیرات

۶-۱) ساختار روند انتقال منطق حالت برای ماشین حالت اصلی کنترل کننده سیستم

در طراحی سخت افزارهای پردازشی پیچیده نظیر این پروژه (آرایه های سیستمولیک)، وجود یک واحد کنترل کننده مرکزی برای مدیریت زمان بندی دقیق و هماهنگی میان ماژول های مختلف سیستم (از جمله حافظه ها، واحد پردازش و رجیسترهای خروجی) امری کاملاً ضروری است. در این پروژه، واحد کنترل کننده بر مبنای یک ماشین حالت متناهی (FSM) طراحی شده است تا وظایف تعریف شده در مستندات پروژه را به صورت ترتیبی به انجام برساند.

همانطور که بطور مختصر در فصل سوم بحث شد، این ماشین حالت دارای پنج وضعیت^۱ اصلی است که روند انتقال بین آنها، چرخه پردازش یک عملیات ضرب ماتریسی را از لحظه دریافت فرمان شروع تا تولید خروجی نهایی شکل می دهد. این وضعیت ها عبارتند از: IDLE، LOAD، COMPUTE، WRITEBACK و DONE.

روند انتقال منطق حالت به این صورت است که سیستم در ابتدا پس از اعمال سیگنال بازنشانی (Reset)، در حالت IDLE قرار می گیرد و منتظر دریافت سیگنال start می ماند. با یک شدن سیگنال start، سیستم به حالت LOAD منتقل می شود. در حالت LOAD، کنترل کننده وظیفه دارد تا فرمان خواندن از حافظه های ورودی را صادر کند. در این بازه زمانی، سیگنال load_en برای بارگذاری داده ها و سیگنال clear_acc برای پاکسازی مقادیر انباشت گرهای^۲ در واحدهای پردازشی فعال می شوند.

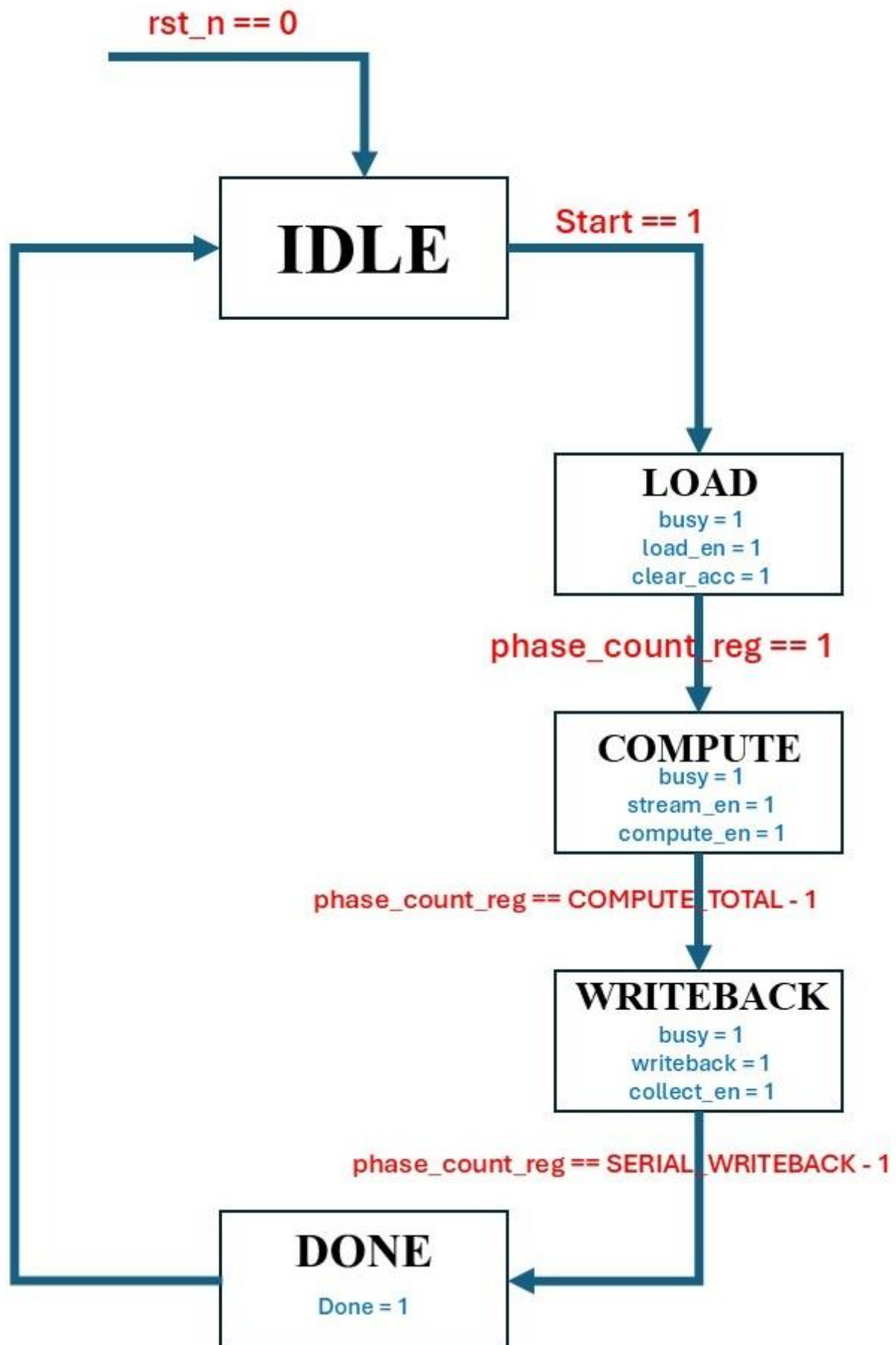
پس از پایان فاز بارگذاری، ماشین حالت به وضعیت COMPUTE (محاسبه) انتقال می یابد. این وضعیت، قلب تپنده سیستم است که در آن عملیات ضرب و جمع ماتریسی انجام می پذیرد. مدت زمان ماندگاری در این حالت کاملاً وابسته به ابعاد ماتریس ها (پارامترهای M و N) است. سیستم برای تزریق داده ها به درون آرایه به تعدادی پالس کلاک نیاز دارد و پس از آن نیز باید منتظر بماند تا آخرین داده ها از انتهای آرایه سیستمولیک خارج شوند. مجموع این پالس های کلاک در معادله

$$COMPUTE_{TOTAL} = (M + N - 1) + N$$

فرمول بندی شده است.

¹ State

² Accumulators



شکل ۱-۶) دیاگرام انتقال حالت برای ماشین حالت کنترل کننده سیستم (FSM)

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state_reg          <= IDLE;
        phase_count_reg <= 16'd0;
    end else begin
        case (state_reg)
            IDLE: begin
                phase_count_reg <= 16'd0;
                if (start) state_reg <= LOAD;
            end

            LOAD: begin
                if (phase_count_reg == 1) begin
                    state_reg          <= COMPUTE;
                    phase_count_reg <= 16'd0;
                end else begin
                    phase_count_reg <= phase_count_reg + 16'd1;
                end
            end

            COMPUTE: begin
                if (phase_count_reg == (COMPUTE_TOTAL - 1)) begin
                    state_reg          <= WRITEBACK;
                    phase_count_reg <= 16'd0;
                end else begin
                    phase_count_reg <= phase_count_reg + 16'd1;
                end
            end

            WRITEBACK: begin
                if (phase_count_reg == (SERIAL_WRITEBACK - 1)) begin
                    state_reg          <= DONE;
                    phase_count_reg <= 16'd0;
                end else begin
                    phase_count_reg <= phase_count_reg + 16'd1;
                end
            end

            DONE: begin
                state_reg          <= IDLE;
                phase_count_reg <= 16'd0;
            end

            default: begin
                state_reg <= IDLE;
                phase_count_reg <= 16'd0;
            end
        endcase
    end
end
end

```

```

always @(*) begin
    busy          = 1'b0;
    done          = 1'b0;
    load_en       = 1'b0;
    stream_en     = 1'b0;
    compute_en    = 1'b0;
    clear_acc     = 1'b0;
    collect_en    = 1'b0;
    writeback     = 1'b0;
    state_dbg     = state_reg;

    case (state_reg)
        IDLE:      busy          = 1'b0;
        LOAD:      begin
                    busy          = 1'b1;
                    load_en       = 1'b1;
                    clear_acc     = 1'b1;
                end
        COMPUTE:   begin
                    busy          = 1'b1;
                    compute_en    = 1'b1;
                    if (phase_count_reg < STREAM_CYCLES) stream_en = 1'b1;
                end
        WRITEBACK: begin
                    busy          = 1'b1;
                    collect_en    = 1'b1;
                    writeback     = 1'b1;
                end
        DONE:      done          = 1'b1;
    endcase
end

```

شکل ۲-۶) پیاده‌سازی سخت‌افزاری سیگنال‌های کنترلی خروجی بر اساس وضعیت فعلی ماشین حالت

پس از اتمام کامل فاز محاسبات، ماشین حالت وارد وضعیت WRITEBACK می‌شود. در این فاز، نتایج موازی تولید شده توسط آرایه سیستمولیک باید به صورت سریال در پورت‌های خروجی قرار گیرند. از آنجایی که ماتریس خروجی دارای ابعاد M در N است، ماشین حالت باید دقیقاً به اندازه تعداد عناصر ماتریس خروجی به علاوه یک سیکل اضافه (جهت همگام‌سازی) در این وضعیت باقی بماند که رابطه آن در معادله

$$SERIAL_{WRITEBACK} = (M \times N) + 1$$

نشان داده شده است. سیگنال‌های `collect_en` و `writeback` در این فاز برای هدایت داده‌ها به سمت ماژول جمع‌آوری خروجی فعال می‌گردند.

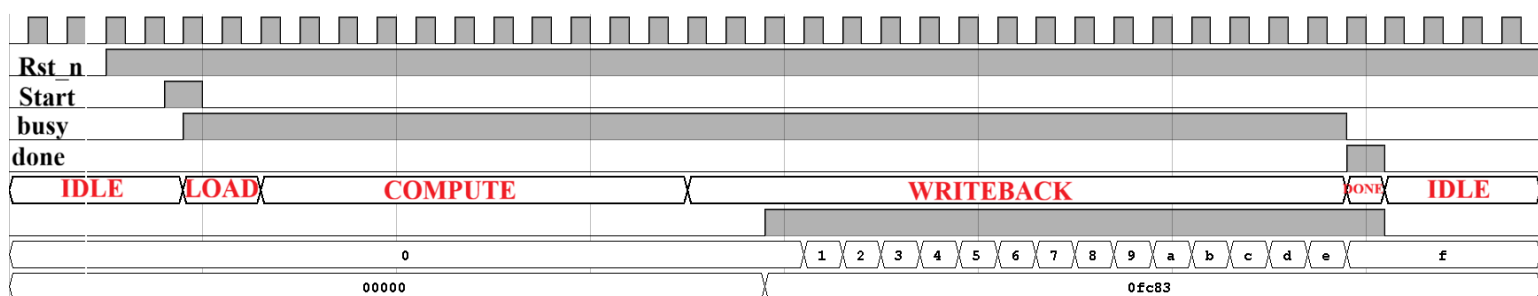
در نهایت، با اتمام فرآیند انتقال داده‌ها، FSM به حالت DONE منتقل شده و سیگنال خروجی `done` را به نشانه موفقیت‌آمیز بودن کل عملیات فعال می‌کند. در پالس کلاک بعدی، سیستم مجدداً به حالت IDLE باز می‌گردد تا برای دریافت دستور پردازش بعدی آماده باشد. این ساختار انتقال حالت، تضمین می‌کند که هیچ یک از فازهای بارگذاری، محاسبه و استخراج نتیجه با یکدیگر تداخل نداشته و سیستم دارای قابلیت اطمینان^۱ بالایی در سطح ثبت انتقال (RTL) باشد.

^۱ Reliability

۲-۶) تشریح حالت‌های پیاده‌سازی شده در ماشین حالت

در پیاده‌سازی ماشین حالت سیستم، برای پیگیری وضعیت فعلی و همچنین شمارش دقیق پالس‌های کلاک در هر وضعیت، از دو رجیستر استفاده شده است. رجیستر اول مشخص‌کننده وضعیت فعلی سیستم است و رجیستر دوم به عنوان یک شمارنده زمان‌سنج^۱ عمل می‌کند تا مدت زمان ماندگاری در هر حالت را با دقت یک پالس کلاک مدیریت کند. در ادامه، عملکرد و سیگنال‌های کنترلی مربوط به فازهای این سیستم تشریح می‌شوند.

وضعیت اول در ماشین حالت، وضعیت استراحت یا IDLE نامگذاری شده است. این وضعیت نقطه شروع به کار سیستم پس از روشن شدن یا اعمال سیگنال بازنشانی (Reset) محسوب می‌شود. در این حالت، ماشین حالت در یک حلقه انتظار قرار می‌گیرد و هیچ‌گونه فعالیت پردازشی یا انتقال داده‌ای انجام نمی‌دهد. در نتیجه، سیگنال خروجی مشغول بودن (busy) غیرفعال باقی می‌ماند تا به ماژول‌ها نشان دهد که آرایه سیستمولیک آماده دریافت فرمان جدید است. در طول این وضعیت، رجیستر شمارنده فاز به طور پیوسته صفر نگه داشته می‌شود. کنترل‌کننده در هر کلاک، پورت ورودی فرمان شروع (start) را بررسی می‌کند و به محض دریافت مقدار یک منطقی در این پورت، سیستم از وضعیت استراحت خارج شده و وارد فاز آماده‌سازی می‌شود.



شکل ۳-۶) شکل موج شبیه‌سازی نشان‌دهنده خروج FSM از حالت استراحت و آغاز چرخه کاری با دریافت فرمان شروع

وضعیت دوم، حالت بارگذاری یا **LOAD** است. با ورود به این فاز، بلافاصله سیگنال مشغول بودن سیستم فعال می‌شود که نشان‌دهنده آغاز به کار رسمی شتاب‌دهنده سخت‌افزاری است. هدف اصلی در این فاز، خواندن داده‌های ورودی شامل ماتریس‌های عملوند و بایاس از حافظه‌های جانبی و آماده‌سازی آن‌ها برای تزریق به درون آرایه سیستمولیک است. برای تحقق این امر، کنترل‌کننده سیگنالی را جهت فعال‌سازی ماژول بارگذار ورودی صادر می‌کند تا این ماژول بتواند مقادیر را از فایل‌های حافظه استخراج کرده و روی گذرگاه‌های داخلی قرار دهد.

به صورت همزمان در همین فاز، سیگنال پاکسازی انباشت‌گرها نیز به سمت آرایه پردازشی ارسال می‌گردد. این سیگنال باعث می‌شود تا تمامی واحدهای پردازشی، مقادیر محاسباتی قبلی خود را پاک کرده و رجیسترهای انباشت‌گر، خود را با مقادیر اولیه بایاس (که مختص به همان واحد پردازشی است) مقداردهی کنند.

¹ Timer

یکی از نکات بسیار مهم در طراحی فاز بارگذاری، مدیریت تاخیرهای انتشار و دسترسی به حافظه است. از آنجایی که استخراج داده‌ها از حافظه و تثبیت آن‌ها بر روی کابل‌های ارتباطی داخلی به زمان نیاز دارد، ماشین حالت به گونه‌ای طراحی شده است که یک تاخیر عمدی ایجاد کند. بر این اساس، سیستم دقیقاً به مدت دو پالس کلاک در وضعیت بارگذاری متوقف می‌ماند. شمارنده فاز در پالس کلاک اول مقدار یک را به خود می‌گیرد و در پالس کلاک دوم، با برآورده شدن شرط توقف، فاز بارگذاری به اتمام رسیده و شمارنده مجدداً صفر می‌شود تا سیستم با داده‌های تثبیت شده و پایدار، با اطمینان کامل وارد فاز محاسبات گردد.

```
LOAD: begin
    if (phase_count_reg == 1) begin
        state_reg      <= COMPUTE;
        phase_count_reg <= 16'd0;
    end else begin
        phase_count_reg <= phase_count_reg + 16'd1;
    end
end
```

شکل ۴-۶) پیاده‌سازی منطق توقف دو سیکلی در فاز بارگذاری جهت رفع عدم تطابق زمانی حافظه

پس از تثبیت داده‌ها روی گذرگاه‌ها در فاز بارگذاری، ماشین حالت وارد حساس‌ترین و اصلی‌ترین بخش عملکردی خود یعنی وضعیت محاسبه یا COMPUTE می‌شود. در این فاز، سیگنال مشغول بودن سیستم همچنان فعال می‌ماند. کنترل‌کننده با فعال کردن سیگنال مجوز محاسبه، به تمامی واحدهای پردازشی (PE) درون شبکه سیستمولیک فرمان می‌دهد تا عملیات ضرب و انباشت (MAC) را آغاز کنند. ویژگی بارز آرایه‌های سیستمولیک، ورود داده‌ها به صورت مورب و پله‌ای^۱ است. به همین دلیل، در طول این فاز سیگنال مجوز جریان داده نیز فعال می‌شود تا مازول بارگذار، داده‌ها را به صورت زمان‌بندی شده و پایپ‌لاین به لبه‌های آرایه تزریق کند.

مدت زمان حضور در وضعیت محاسبه به دو زیربخش زمانی تقسیم می‌شود. بخش اول، زمان لازم برای تزریق کامل تمامی عناصر ماتریس‌های ورودی به درون آرایه است که به ابعاد ماتریس‌ها بستگی داشته و بر اساس معادله

$$STREAM_CYCLES = M + N - 1$$

محاسبه می‌گردد. بخش دوم، زمان تخلیه^۲ نام دارد. پس از آنکه آخرین داده وارد لبه‌های آرایه شد، این داده‌ها باید در طول شبکه‌ی پردازشی حرکت کرده و تا آخرین واحد پردازشی در گوشه پایین سمت راست آرایه پیش بروند تا محاسبات آن واحد نیز تکمیل شود. این زمان تاخیر انتشار داخلی، بر اساس معادله

$$DRAIN_CYCLES = N$$

فرمول‌بندی شده است.

ماشین حالت با استفاده از رجیستر شمارنده خود، مجموع این دو زمان را اندازه می‌گیرد و سیگنال مجوز جریان داده را تنها در زمان تزریق فعال نگه می‌دارد، اما سیستم را تا پایان زمان تخلیه در وضعیت محاسبه حفظ می‌کند تا از صحت و تکمیل تمامی ضرب‌ها و جمع‌ها اطمینان حاصل شود.

¹ Skewed

² Drain



شکل ۵-۶ رفتار سیگنال‌های کنترلی در وضعیت COMPUTE و زمان‌بندی دقیق پایپ‌لاین آرایه سیستمولیک

با پایان یافتن فاز محاسبات و تکمیل شدن مقادیر نهایی درون رجیستر انباشت‌گرهای هر واحد پردازشی، سیستم به وضعیت بازنویسی یا WRITEBACK منتقل می‌گردد. در این لحظه، نتایج کل ماتریس خروجی به صورت یک آرایه موازی بسیار عریض در دسترس است، اما طبق نیازمندی‌های پروژه، این داده‌ها باید به صورت سریال و تک‌به‌تک همراه با آدرس اختصاصی به خروجی ارسال شوند.

در این فاز، کنترل‌کننده سیگنال‌های مربوط به جمع‌آوری و بازنویسی داده را فعال می‌کند تا ماژول جمع‌آوری خروجی Output_Collector وارد عمل شود. این ماژول در اولین پالس کلاک از این فاز، یک عکس‌برداری^۱ لحظه‌ای از تمامی نتایج موازی انجام داده و آن‌ها را در یک بافر داخلی قفل می‌کند تا در طول فرآیند ارسال سریال، داده‌ها تغییر نکنند. ماشین حالت باید به تعداد کل عناصر ماتریس خروجی به علاوه یک سیکل اضافی جهت همگام‌سازی اولیه و تثبیت بافر، در این وضعیت باقی بماند که این زمان در معادله

$$SERIAL_WRITEBACK = (M \times N) + 1$$

نشان داده شده است. شمارنده فاز به طور مداوم کار می‌کند تا تک‌تک داده‌های ۱۸ بیتی به همراه آدرس ۴ بیتی مربوطه، چرخه به چرخه به سمت پورت‌های خروجی هدایت شوند.

```
WRITEBACK: begin
    if (phase_count_reg == (SERIAL_WRITEBACK - 1)) begin
        state_reg      <= DONE;
        phase_count_reg <= 16'd0;
    end else begin
        phase_count_reg <= phase_count_reg + 16'd1;
    end
end

WRITEBACK: begin
    busy      = 1'b1;
    collect_en = 1'b1;
    writeback  = 1'b1;
end
```

شکل ۶-۶ منطق شمارش و شرط خروج از فاز WRITEBACK جهت سریال‌سازی نتایج موازی

سرانجام پس از ارسال موفقیت‌آمیز آخرین عنصر ماتریس خروجی، FSM به آخرین وضعیت خود یعنی DONE منتقل می‌شود. این وضعیت تنها به مدت یک پالس کلاک به طول می‌انجامد. در این تک پالس، کنترل‌کننده سیگنال خروجی انجام کار را یک می‌کند تا به پردازنده‌ی اصلی یا محیط تست‌بنچ اتمام موفقیت‌آمیز کل عملیات ضرب ماتریسی را اعلام نماید. بلافاصله در لبه بالارونده بعدی پالس کلاک، ماشین حالت به صورت خودکار به وضعیت استراحت بازگشته و تمامی شمارنده‌ها

¹ Snapshot

و سیگنال‌های کنترلی بازنشانی^۱ می‌شوند تا سیستم بدون هیچ‌گونه تاخیر اضافی، آماده دریافت فرمان جدید برای پردازش بعدی باشد. این معماری حلقوی و خودکار، پایداری جریان کنترل را در کاربردهای پردازش مستمر شبکه‌های عصبی تضمین می‌کند.

۳-۶) معرفی سیستم ردگیری تاخیرها برای اثبات کارایی پایپ‌لاین سیستم

همان‌طور که در نیازمندی‌های این پروژه ذکر شده است، گزارش دو پارامتر اساسی یعنی زمان تاخیر و توان عملیاتی برای ارزیابی عملکرد مدار الزامی است. برای پاسخ‌گویی دقیق به این نیازمندی و به حداقل رساندن خطاهای محاسباتی در زمان شبیه‌سازی، یک سیستم مانیتورینگ و ردگیری چرخه^۲ به صورت سخت‌افزاری درون ماژول کنترل‌کننده (FSM) تعبیه شده است. این سیستم ردگیری بر پایه یک رجیستر شمارنده ۱۶ بیتی با نام `cycle_count_dbg` پیاده‌سازی شده است. پسوند `dbg` (مخفف Debug) نشان‌دهنده این است که این رجیستر وظیفه نظارتی مدار را بر عهده دارد و کمک می‌کند تا رفتار زمانی سیستم را پایش کنیم. منطق عملکرد این شمارنده بسیار هوشمندانه و در عین حال ساده طراحی شده است تا بار پردازشی اضافی به سیستم تحمیل نکند.

روند کار این سیستم مانیتورینگ به این شکل است که شمارنده به سیگنال‌های تنظیم مجدد (Reset) و فرمان شروع (start) حساس است. به محض روشن شدن سیستم یا دریافت فرمان جدید برای محاسبه یک ماتریس جدید، این شمارنده بلافاصله صفر می‌شود. این ویژگی تضمین می‌کند که اندازه‌گیری زمان برای هر عملیات ضرب ماتریسی به صورت کاملاً مستقل انجام شده و تاخیرهای مربوط به عملیات‌های قبلی در محاسبات جدید تداخل ایجاد نکنند.

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)          cycle_count_dbg <= 16'd0;
    else if (start)      cycle_count_dbg <= 16'd0;
    else if (busy)       cycle_count_dbg <= cycle_count_dbg + 16'd1;
end
```

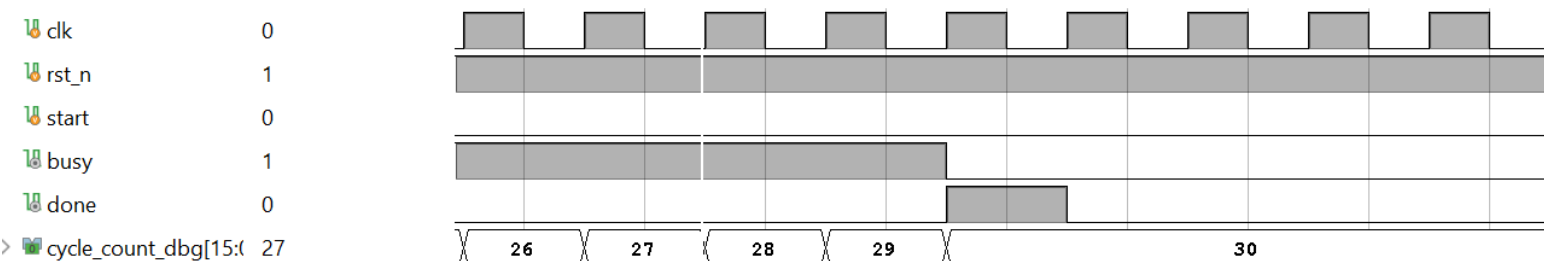
شکل ۷-۶) پیاده‌سازی سخت‌افزاری رجیستر مانیتورینگ جهت ردگیری تعداد پالس‌های کلاک و اثبات کارایی پایپ‌لاین

پس از صفر شدن شمارنده با دریافت سیگنال شروع، شرط افزایش این شمارنده تنها به یک متغیر وابسته است: سیگنال مشغول بودن یا `busy`. تا زمانی که ماشین حالت در یکی از وضعیت‌های کاری خود (یعنی فاز بارگذاری، فاز محاسبات و فاز بازنویسی) قرار داشته باشد، سیگنال `busy` در وضعیت یک منطقی باقی می‌ماند. در این بازه زمانی، رجیستر ردگیری تاخیر در هر لبه بالارونده پالس کلاک، یک واحد افزایش می‌یابد. به محض اتمام کامل فرآیند و ورود ماشین حالت به وضعیت `DONE`، سیگنال `busy` صفر شده و شمارنده دقیقاً در همان لحظه متوقف می‌گردد. اهمیت این سیستم در اثبات کارایی پایپ‌لاین سیستم نهفته است. همان‌طور که پیش‌تر بحث شد، در یک آرایه سیستمولیک، داده‌ها به صورت موجی و با تاخیرهای مورب وارد واحدهای پردازشی (PE) می‌شوند و واحدهای مختلف در سیکل‌های متفاوت شروع به کار می‌کنند. در یک طراحی بدون نقص، نباید هیچ‌گونه توقف^۳ در طول مسیر پایپ‌لاین وجود داشته باشد. با مشاهده مقدار نهایی ذخیره شده در رجیستر `cycle_count_dbg` پس از پایان عملیات و مقایسه آن با مدل‌های ریاضی و تئوری، می‌توان به صورت قطعی اثبات کرد که سخت‌افزار طراحی شده دقیقاً در همان تعداد پالس کلاکی که از نظر ریاضی پیش‌بینی شده بود، عملیات را به پایان رسانده است.

^۱ Reset

^۲ Cycle Tracking System

^۳ Stall



شکل ۸-۶) خروجی شکل موج شبیه‌سازی نشان‌دهنده مقدار نهایی شمارنده تاخیر در پایان عملیات، تاییدکننده عدم وجود توقف در پایپ‌لاین

در واقع، این رجیستر به عنوان یک کرنومتر داخلی عمل می‌کند. اگر طراحی آرایه سیستمولیک یا مکانیزم دسترسی به حافظه دارای نقص زمان‌بندی بود، داده‌ها در زمان مقرر به واحدهای پردازشی نمی‌رسیدند و مجبور می‌شدیم چرخه‌های انتظار به ماشین حالت اضافه کند که این امر مقدار شمارنده نهایی را افزایش می‌داد. اما مطابقت مقدار این رجیستر در شبیه‌سازی با روابط ریاضی، اثبات عملکرد صحیح معماری پایپ‌لاین در این پروژه است.

۴-۶) مدل‌سازی تاخیر پردازشی اولیه مدار تا ظهور اولین پاسخ برحسب شمارش پالس کلاک

یکی از مهم‌ترین شاخص‌های ارزیابی عملکرد در معماری‌های مبتنی بر پایپ‌لاین و شتاب‌دهنده‌های سخت‌افزاری، پارامتر تاخیر پردازشی^۱ است. همان‌طور که در نیازمندی‌های این پروژه به وضوح ذکر شده است، تاخیر پردازشی به معنای دقیق تعداد سیکل‌های کلاک از لحظه دریافت فرمان شروع (start) تا ظاهر شدن اولین خروجی معتبر تعریف می‌گردد. در مدارهای مجتمع دیجیتال، این زمان نشان‌دهنده مدت زمانی است که طول می‌کشد تا خط لوله پردازشی به طور کامل پر شده و اولین نتیجه صحیح از انتهای آن خارج گردد. برای اثبات و محاسبه دقیق این پارامتر در معماری طراحی شده، باید مسیر جریان داده^۲ و ماشین حالت را چرخه به چرخه ردیابی کرده و زمان‌های سپری شده در هر فاز را با یکدیگر جمع کنیم. روند محاسبه تاخیر پردازشی در این سیستم به سه مرحله زمانی اصلی تقسیم می‌شود که در ادامه به تفصیل مورد بررسی قرار می‌گیرند:

مرحله اول: تاخیر فاز بارگذاری و دسترسی به حافظه. هنگامی که سیگنال start به ماشین حالت اعمال می‌شود، سیستم از حالت استراحت خارج شده و وارد وضعیت بارگذاری (LOAD) می‌گردد. وظیفه اصلی در این فاز، خواندن داده‌ها از فایل‌های حافظه و نگاشت آن‌ها بر روی رجیسترهای داخلی ماژول input_loader است. به دلیل نیاز به همگام‌سازی و جلوگیری از عدم تطابق زمانی^۳ بین خواندن داده و تزریق آن، ماشین حالت به گونه‌ای طراحی شده است که دقیقاً به مدت دو پالس کلاک در این وضعیت متوقف بماند. در پالس اول، مقادیر خوانده شده، و در پالس دوم، این مقادیر بر روی گذرگاه‌های متصل به آرایه سیستمولیک کاملاً تثبیت می‌شوند. این تاخیر اولیه در معادله

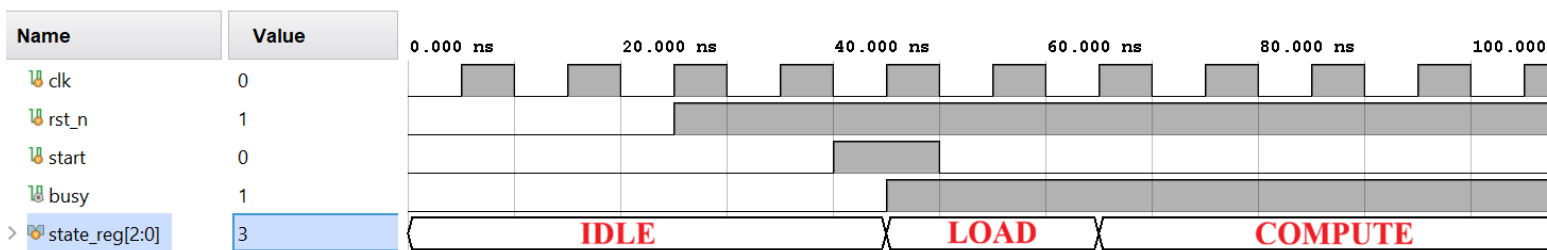
$$Latency_{LOAD} = 2 \text{ cycles}$$

به عنوان تاخیر دسترسی به حافظه تعریف شده است.

¹ Latency

² Data Path

³ Timing Mismatch



شکل ۹-۶) بررسی زمان‌بندی فاز بارگذاری و اختصاص دو سیکل ساعت جهت تثبیت داده‌های حافظه ورودی

مرحله دوم: تاخیر فاز محاسبات و پایپ‌لاین سیستمیک. پس از تثبیت داده‌ها، ماشین حالت وارد وضعیت محاسبه (COMPUTE) می‌شود و همزمان سیگنال‌های تزریق داده و مجوز محاسبه فعال می‌گردند. در معماری آرایه سیستمیک، داده‌ها به صورت همزمان به تمام هسته‌ها نمی‌رسند، بلکه به صورت مورب و پله‌ای با تاخیرهای یک سیکلی وارد لایه‌های بعدی می‌شوند. مدت زمانی که طول می‌کشد تا تمام داده‌های ماتریس‌های ورودی به درون آرایه جریان یابند^۱، وابسته به تعداد سطرها (M) و ستون‌ها (N) است. پس از ورود آخرین داده‌ها، سیستم باید زمان مشخصی را منتظر بماند تا این داده‌ها در طول آرایه پردازش شده و به آخرین واحد پردازشی (PE) در گوشه انتهایی برسند^۲.

بر اساس پیاده‌سازی انجام شده در FSM، مجموع این دو زمان مشخص‌کننده کل زمان ماندگاری در وضعیت محاسبات است. ماشین حالت دقیقاً تا پایان این زمان در وضعیت COMPUTE باقی می‌ماند تا تمامی نتایج موازی در خروجی آرایه کامل شوند. مدل‌سازی ریاضی این تاخیر عمیق پردازشی در معادله زیر نشان داده شده است.

$$Latency_{COMPUTE} = (M + N - 1) + N = M + 2N - 1 \text{ cycles}$$

مرحله سوم: تاخیر آماده‌سازی و عکس‌برداری در فاز خروجی. با تکمیل پردازش موازی، FSM به وضعیت بازنویسی (WRITEBACK) منتقل می‌گردد. در این لحظه، نتایج کل ماتریس (شامل ۱۶ داده ۱۸ بیتی برای ابعاد ۴ در ۴) آماده است، اما برای ارسال آن‌ها به پورت خروجی باید به صورت سریال عمل شود. ماژول جمع‌آوری خروجی (output_collector) به گونه‌ای طراحی شده است که در اولین پالس کلاک از فاز بازنویسی، کل دیتای موازی را در یک بافر داخلی به نام buffered_results قفل^۳ می‌کند. در این سیکل اول، پرچم flag_buffered_once یک می‌شود اما هنوز سیگنال out_valid صفر است. هدف از این یک سیکل تاخیر، ایجاد یک عکس‌برداری امن از نتایج است تا در حین ارسال سریال داده‌ها، هیچ تغییری در مقادیر خروجی آرایه به وجود نیاید. بلافاصله در پالس کلاک بعدی، اولین داده معتبر به همراه آدرس خود در پورت‌های خروجی قرار گرفته و سیگنال out_valid فعال می‌گردد. بنابراین، تاخیر ماژول خروجی تا ظهور اولین پاسخ برابر با دو سیکل کلاک خواهد بود که در معادله

$$Latency_{OUTPUT_SETUP} = 2 \text{ cycle}$$

آورده شده است.

^۱ Stream Cycles

^۲ Drain Cycles

^۳ Latch

```

// Re-arm state tracking flags logically dynamically on load cycle completion
if (!writeback) begin
    flag_buffered_once <= 1'b0;
    flat_counter <= 0;
    out_valid <= 1'b0;
end

// Cycle-by-cycle Memory mapped extraction pipeline operation logic:
if (writeback) begin
    // Lock the accumulated hardware results on the exact moment collection window operates cycle 1
    if (!flag_buffered_once) begin
        buffered_results <= c_psum_flat_in;
        flag_buffered_once <= 1'b1;
        out_valid <= 1'b0;
    end
    else if (flat_counter < (M * N)) begin
        out_valid <= 1'b1;
        out_addr <= flat_counter[3:0]; // Addresses Output buffer bounds! Address format exactly as [3:0] Spec Limits
        // Dynamically mapping internal Mux indexing to the flattened bus Array results to sequential output pins
        out_data <= buffered_results[(flat_counter*ACC_WIDTH) +: ACC_WIDTH];
        flat_counter <= flat_counter + 1;
    end
    else begin
        // Finish transmission logic closure cycle pulse bounds
        out_valid <= 1'b0;
    end
end
end
end

```

شکل ۱۰-۶) منطق سخت‌افزاری ایجاد بافر میانی و مصرف یک سیکل ساعت برای تثبیت موازی داده‌ها پیش از سریال‌سازی خروجی

استنتاج تاخیر کل (Total Latency)

اکنون با در اختیار داشتن تاخیرهای مربوط به هر یک از بلوک‌های مسیر داده، می‌توان تاخیر کل پردازشی مدار را به صورت پارامتری محاسبه کرد. تاخیر کل برابر است با مجموع تاخیر فاز بارگذاری، تاخیر فاز محاسبات و تاخیر آماده‌سازی بافر خروجی. به عنوان یک مثال عملی، با توجه به مقادیر پیش‌فرض پروژه (ماتریس‌های ۴ در ۴ یعنی $M = 4$ و $N = 4$)، فاز بارگذاری ۲ سیکل، فاز محاسبات ۱۱ سیکل (یعنی $1 - 8 + 4$) و فاز آماده‌سازی خروجی ۲ سیکل زمان می‌برند. در نتیجه، از لحظه اعمال فرمان شروع، دقیقاً پس از گذشت ۱۵ پالس کلاک، اولین پاسخ معتبر در پورت خروجی پدیدار خواهد شد. این مدل‌سازی ریاضی که به طور کامل با نتایج شبیه‌سازی شکل موج تطابق دارد، صحت طراحی خط لوله و معماری کنترل‌کننده را به اثبات می‌رساند.

۵-۶) استنتاج محاسباتی مدل توان عملیاتی متوالی (Throughput Report Computation)

پس از محاسبه زمان تاخیر اولیه، پارامتر توان عملیاتی^۱ به عنوان دومین شاخص کلیدی مورد بررسی قرار می‌گیرد. بر اساس تعاریف مستندات پروژه، توان عملیاتی به معنای تعداد خروجی‌های معتبر تولید شده در هر پالس کلاک، پس از پر شدن کامل خط لوله و ظهور اولین پاسخ می‌باشد. به عبارت دیگر، این پارامتر نشان می‌دهد که سیستم با چه سرعتی می‌تواند نتایج متوالی را پس از راه‌اندازی اولیه در اختیار کاربر یا ماژول‌های بعدی قرار دهد.

برای محاسبه و اثبات دقیق این پارامتر، باید رفتار ماژول جمع‌آوری خروجی (output_collector) را در فاز بازنویسی (WRITEBACK) مورد تحلیل قرار دهیم. همان‌طور که در بخش‌های قبلی اشاره شد، نتایج موازی حاصل از آرایه سیستمولیک در اولین سیکل از فاز بازنویسی درون یک رجیستر عریض (!) به نام buffered_results ذخیره و قفل می‌شوند. پس از این سیکل راه‌اندازی، ماشین حالت سیستم در یک حلقه شمارشی قرار می‌گیرد.

¹ Throughput

در پیاده‌سازی سخت‌افزاری این بخش، از یک شمارنده داخلی به نام flat_counter استفاده شده است. این شمارنده وظیفه دارد تا از آدرس صفر تا آخرین آدرس ماتریس خروجی (یعنی $M \times N - 1$) را در هر پالس کلاک طی کند. منطق طراحی به گونه‌ای است که در هر لبه بالارونده از پالس کلاک، مقدار flat_counter یک واحد افزایش یافته و همزمان سیگنال اعتبار خروجی (out_valid) در وضعیت یک منطقی (High) قرار می‌گیرد. در همین لحظه، داده ۱۸ بیتی متناظر با آن آدرس، از کابل عریض buffered_results جدا شده و بر روی پورت out_data قرار می‌گیرد.

با توجه به این ساختار بدون وقفه، از لحظه‌ای که اولین داده معتبر در خروجی ظاهر می‌شود، در تمامی پالس‌های کلاک بعدی (بدون وجود هیچ‌گونه چرخه انتظار)، دقیقاً یک داده معتبر جدید به همراه آدرس اختصاصی آن تولید می‌گردد. بنابراین، توان عملیاتی لحظه‌ای این معماری در فاز خروجی، برابر با یک خروجی معتبر در هر سیکل کلاک است که این موضوع در معادله

$$Throughput = 1 \text{ Valid Output} / \text{Clock Cycle}$$

بیان شده است. این بالاترین نرخ ممکن برای یک گذرگاه سریال است و نشان از استفاده کامل از پهنای باند پورت خروجی دارد.

برای درک بهتر عملکرد کلی سیستم، می‌توان زمان کل مورد نیاز برای استخراج تمامی عناصر ماتریس را نیز محاسبه کرد. از آنجا که سیستم باید ماتریسی با ابعاد M سطر و N ستون را به صورت سریال خارج کند و در هر سیکل یک عنصر استخراج می‌شود، کل زمان لازم برای تخلیه کامل خروجی‌ها دقیقاً برابر با حاصل ضرب ابعاد ماتریس خروجی خواهد بود. این رابطه در معادله

$$Total_{Readout\ Time} = M \times N \text{ cycles}$$

فرمول‌بندی شده است. در نهایت، با ترکیب نتایج به دست آمده از مدل‌سازی تاخیر و توان عملیاتی، می‌توان کل زمان چرخه پردازشی^۱ از لحظه دریافت فرمان start تا پایان کامل عملیات و فعال شدن سیگنال done را محاسبه نمود. این زمان شامل تاخیر اولیه به علاوه زمان لازم برای ارسال تمام داده‌های خروجی متوالی است که رابطه نهایی آن در معادله

$$Total_{Operation\ Time} = Latency + (M \times N) = (M + 2N + 2) + (M \times N) \text{ cycles}$$

آورده شده است. به عنوان مثال، برای پیکربندی پیش‌فرض پروژه (ماتریس‌های ۴ در ۴)، پس از ۱۵ سیکل تاخیر اولیه، سیستم وارد فاز سریال‌سازی شده و ۱۶ داده خروجی را دقیقاً در ۱۶ سیکل متوالی (با توان عملیاتی ۱ داده بر سیکل) تولید می‌کند. بنابراین پس از گذشت مجموعاً ۳۱ پالس کلاک، پردازش یک بلوک کامل ماتریسی به پایان می‌رسد.

¹ Total Operation Time

فصل هفتم: رابط ورودی و خروجی، ارتباط با حافظه و پارامتری کردن ابعاد

۷-۱) مدار طراحی شده جهت پاسخ به استانداردسازی برگرداندن آدرس‌ها به خروجی

در طراحی سیستم‌های پردازشی مبتنی بر آرایه‌های سیستم‌لیک، یکی از چالش‌های اساسی در بخش رابط خروجی^۱، نحوه انتقال نتایج محاسبه شده به دنیای خارج از تراشه یا ماژول‌های بالادستی است. آرایه سیستم‌لیک به دلیل ماهیت موازی خود، در انتهای فاز محاسبات، تمامی نتایج را به صورت یکباره و در قالب یک گذرگاه داده بسیار عریض تولید می‌کند. با این حال، مطابق با نیازمندی‌های ذکر شده در صفحه پنجم مستندات این پروژه، خروجی سیستم باید به صورت سریال، استاندارد و همراه با آدرس‌دهی دقیق (پین‌های out_valid، out_addr و out_data) در دسترس قرار گیرد. برای پاسخ‌گویی به این نیازمندی، ماژولی اختصاصی تحت عنوان output_collector طراحی و پیاده‌سازی شده است.

این مدار خروجی به گونه‌ای طراحی شده است که به عنوان یک پل ارتباطی میان پهنای باند عظیم داخلی آرایه و پورت‌های محدود خروجی عمل کند. روند کار این مدار کاملاً وابسته به سیگنال writeback است که از سوی ماشین حالت صادر می‌شود. به محض دریافت این سیگنال، سیستم نیازمند یک مکانیزم قفل‌کننده^۲ است تا از تغییرات ناخواسته داده‌ها در حین ارسال سریال جلوگیری کند. بدین منظور، در اولین پالس کلاک از فاز بازنویسی، مدار تمام داده‌های موازی آرایه را که از طریق گذرگاه c_psum_flat_in دریافت می‌شوند، در یک رجیستر بافر عریض به نام buffered_results ذخیره می‌کند. برای اطمینان از اینکه این عمل تنها یک بار انجام می‌شود، از یک پرچم کنترلی به نام flag_buffered_once استفاده شده است. این پرچم پس از اولین عکس‌برداری از نتایج، یک می‌شود و از بازنویسی مجدد بافر در سیکل‌های بعدی جلوگیری می‌کند. پس از تثبیت داده‌ها در بافر، فرآیند استانداردسازی و برگرداندن آدرس‌ها آغاز می‌گردد. هسته مرکزی این فرآیند، یک شمارنده داخلی به نام flat_counter است که نقش اشاره‌گر^۳ حافظه را ایفا می‌کند. این شمارنده در هر پالس کلاک یک واحد افزایش می‌یابد و وظیفه پیمایش داده‌های قفل شده را بر عهده دارد. برای تولید آدرس استاندارد خروجی، چهار بیت ارزش پایین (LSB) این شمارنده مستقیماً به پورت out_addr متصل می‌گردند. این تکنیک باعث می‌شود که سیستم به صورت خودکار آدرس‌های 0 تا 15 (برای حالت پیش‌فرض 4 در 4) را تولید نماید و نیازی به یک واحد آدرس‌ساز مجزا نداشته باشیم! بازه تغییرات این شمارنده و تولید آدرس متناظر با آن در معادله

$$0 \leq out_addr \leq (M \times N) - 1$$

نشان داده شده است. همزمان با تولید آدرس، مدار باید داده 18 بیتی متناظر با آن آدرس را نیز از درون بافر عریض استخراج کند. این عمل از طریق یک مالتی‌پلکسر در کد وریلاگ انجام می‌پذیرد؛ جایی که مدار با استفاده از مقدار فعلی flat_counter، آفست دقیق داده مورد نظر را محاسبه کرده و آن بخش 18 بیتی را بر روی پورت out_data قرار می‌دهد. رابطه ریاضی مربوط به استخراج مکان دقیق داده‌ها از داخل کابل عریض بر اساس آدرس شمارنده، در معادله

$$Data_Offset_Start = flat_counter \times ACC_WIDTH$$

فرمول‌بندی شده است.

¹ Output Interface

² Latching

³ Pointer

done	0																	
cycle_count_dbg[15:]	17	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
out_valid	1																	
out_addr[3:0]	2	0		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
out_data[17:0]	927	0	1265	-1279	927	-1912	-854	721	-1114	1659	1630	-1193	2698	-3303	-1369	1455	-1202	2002

شکل ۱-۷) شکل موج شبیه‌سازی خروجی سیستم، تایید‌کننده همگامی کامل بین out_valid با تولید آدرس‌های استاندارد و داده‌های معتبر

در تمام طول این فرآیند استخراج و آدرس‌دهی، پورت `out_valid` در وضعیت یک منطقی (High) نگه داشته می‌شود تا به سیستم گیرنده اعلام کند که داده و آدرس حاضر بر روی گذرگاه معتبر هستند. به محض اینکه شمارنده به سقف ابعاد ماتریس (یعنی مقدار $M \times N$) برسد، مدار فرآیند ارسال را خاتمه داده و پورت `out_valid` را صفر می‌کند.

۷-۲) فرآیند همسان سازی مدارهای حافظه‌ای، MEM_IN و MEM_WB

در معماری شتاب‌دهنده‌های سخت‌افزاری که بر پایه پردازش موازی استوار هستند، گلوگاه اصلی سیستم غالباً در بخش پردازش نیست، بلکه در نحوه تامین داده‌ها و ارتباط با حافظه نهفته است! آرایه‌های سیستمولیک برای حفظ توان عملیاتی بالای خود، نیازمند تزریق داده‌ها با پهنای باند بسیار زیاد در لبه‌های آرایه هستند. با این حال، حافظه‌های استاندارد (مانند SRAMها) معمولاً دارای پورت‌های دسترسی محدود (تک‌پورت یا دوپورت) می‌باشند که خواندن تمام عناصر یک ماتریس را در یک پالس کلاک غیرممکن می‌سازد. برای رفع این چالش و برآورده ساختن نیازمندی‌های بخش 6.1 و 6.2 از مستندات پروژه، ماژول `input_loader` با هدف همسان‌سازی و تطبیق پهنای باند طراحی شده است.

بر اساس ساختار تعریف شده در پروژه، سیستم دارای دو بلوک حافظه اصلی است: حافظه MEM_IN برای نگهداری داده‌های ورودی (ماتریس A) و حافظه MEM_WB برای نگهداری وزن‌ها (ماتریس B) و مقادیر بایاس (ماتریس C). در پیاده‌سازی سخت‌افزاری این پروژه، حافظه MEM_IN به صورت یک آرایه 16 خانه‌ای با عرض 8 بیت تعریف شده است تا 16 عنصر ماتریس ورودی را در خود جای دهد. از سوی دیگر، حافظه MEM_WB به صورت یک آرایه 32 خانه‌ای با عرض 8 بیت تعریف گردیده است؛ به طوری که 16 خانه اول (آدرس‌های 0 تا 15) به وزن‌های ماتریس B و 16 خانه دوم (آدرس‌های 16 تا 31) به مقادیر بایاس اختصاص یافته‌اند.

```

// Load Memory Array elements generated into Test Folders
initial begin
    // -----
    // Ensure "IN.txt" & "WB.txt" are inside Simulation Run Folders
    // -----
    $readmemh("IN.txt", MEM_IN);
    $readmemh("WB.txt", MEM_WB);
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        a_matrix_reg      <= 0;
        b_matrix_reg      <= 0;
        bias_matrix_flat  <= 0;

        a_row_data_flat   <= 0;
        a_row_valid        <= 0;
        b_col_data_flat    <= 0;
        b_col_valid        <= 0;
        stream_cycle_dbg   <= 16'd0;
    end else begin

        // MAP DATA from Read Buffers logically back across to Core Array registers dynamically mapped Row Major Style:
        if (load_en) begin
            for(load_idx = 0; load_idx < 16; load_idx = load_idx + 1) begin
                a_matrix_reg[(load_idx*8) +: 8] <= MEM_IN[load_idx];
                b_matrix_reg[(load_idx*8) +: 8] <= MEM_WB[load_idx];
                bias_matrix_flat[(load_idx*8) +: 8] <= MEM_WB[16+load_idx];
            end
            stream_cycle_dbg <= 16'd0;
        end
    end
end

```

شکل ۲-۷) مقداردهی اولیه مدارهای حافظه در محیط شبیه‌سازی با استخراج داده‌ها از فایل‌های استاندارد IN.txt و WB.txt

فرآیند همسان‌سازی^۱ دقیقاً در فاز بارگذاری (LOAD) اتفاق می‌افتد. زمانی که FSM سیگنال load_en را فعال می‌کند، سیستم به جای آنکه در طول فاز محاسبات به صورت مکرر به حافظه مراجعه کند، یک تکنیک تخت‌سازی فضایی^۲ را اجرا می‌نماید. در این فرآیند، یک حلقه تکرار سخت‌افزاری در مدار پیاده‌سازی شده است که در یک پالس کلاک، تمامی داده‌های مورد نیاز را از بلوک‌های حافظه استخراج کرده و آن‌ها را به هم متصل^۳ می‌کند تا به صورت یک کابل عریض و یکپارچه درآیند. این نگاشت فیزیکی باعث می‌شود که آرایه‌های حافظه‌ای که ذاتاً ساختاری متوالی دارند، به رجیسترهای بسیار عریض و موازی تبدیل شوند. به عنوان مثال، در این حلقه، متغیر load_idx از 0 تا 15 می‌شمارد و در هر تکرار، داده 8 بیتی مربوط به آن اندیس را در جایگاه مناسبی از رجیسترهای a_matrix_reg و b_matrix_reg قرار می‌دهد. رابطه ریاضی این نگاشت بیتی و تخصیص پهنای باند برای همسان‌سازی حافظه با رجیسترهای داخلی در معادله

$$Register_Mapping_Range [Bits] = [(load_idx \times 8) + 7 : (load_idx \times 8)]$$

استخراج شده است. علاوه بر ماتریس‌های عملوند، مقادیر بایاس نیز نیازمند همسان‌سازی هستند. از آنجا که مقادیر بایاس در آدرس‌های 16 تا 31 حافظه MEM_WB قرار دارند، مدار با استفاده از یک شیفت آدرسی (افزودن عدد 16 به اندیس فعلی)، این داده‌ها را خوانده و درون گذرگاه عریض bias_matrix_flat جانمایی می‌کند. این شیفت آدرس‌دهی و انتقال به گذرگاه عریض بایاس در معادله

$$Bias_Flat_Data = MEM_WB[16 + load_idx]$$

نشان داده شده است.

¹ Homogenization

² Spatial Flattening

³ Concatenate

سپس بلوک اصلی با نام GEN_ROWS و GEN_COLS یک حلقه تودرتو^۱ ایجاد می‌کند که تعداد $M \times N$ بار، ماژول pe را درون سخت‌افزار کپی کرده و سیم‌کشی‌های بین آن‌ها را به صورت خودکار انجام می‌دهد. یکی از تکنیک‌های به کار رفته در این بخش، نحوه مدیریت درگاه‌های ورودی و خروجی است. برای استخراج داده‌های مربوط به هر واحد پردازشی، از یک منطق آدرس‌دهی مبتنی بر موقعیت سطر و ستون آن واحد پردازشی در داخل حلقه generate استفاده شده است. پهنای باند و آفست دقیق برای تخصیص بایاس 8 بیتی به هر هسته، بر اساس معادله

$$Bit_Offset = (row \times N + col) \times DATA_WIDTH$$

محاسبه و در کد پیاده‌سازی شده است.

```
generate
for (row = 0; row < M; row = row + 1) begin : GEN_A_EDGE
    assign a_bus[row][0] = a_row_data_flat[(row*DATA_WIDTH) +: DATA_WIDTH];
    assign a_valid_bus[row][0] = a_row_valid[row];
end

for (col = 0; col < N; col = col + 1) begin : GEN_B_EDGE
    assign b_bus[0][col] = b_col_data_flat[(col*DATA_WIDTH) +: DATA_WIDTH];
    assign b_valid_bus[0][col] = b_col_valid[col];
end

for (row = 0; row < M; row = row + 1) begin : GEN_ROWS
    for (col = 0; col < N; col = col + 1) begin : GEN_COLS

        pe #(
            .DATA_WIDTH(DATA_WIDTH),
            .ACC_WIDTH(ACC_WIDTH)
        ) u_pe (
            .clk(clk),
            .rst_n(rst_n),
            .clear_acc(clear_acc),
            .enable(enable),
            .a_in(a_bus[row][col]),
            .b_in(b_bus[row][col]),

            .bias_in(bias_matrix_flat[((row*N + col)*DATA_WIDTH) +: DATA_WIDTH]),

            .a_valid_in(a_valid_bus[row][col]),
            .b_valid_in(b_valid_bus[row][col]),
            .a_out(a_bus[row][col+1]),
            .b_out(b_bus[row+1][col]),
            .a_valid_out(a_valid_bus[row][col+1]),
            .b_valid_out(b_valid_bus[row+1][col]),

            .psum_out(pe_psum[row][col])
        );

        assign c_psum_flat[((row*N + col)*ACC_WIDTH) +: ACC_WIDTH] = pe_psum[row][col];
    end
end
endgenerate
```

شکل ۴-۷) استفاده از ساختار Generate Block جهت تولید خودکار شبکه سیستمولیک و توزیع ماتریس تخت‌شده بایاس‌ها

این معماری و استفاده اصولی از Generics باعث می‌شود که مدار طراحی شده به یک سخت‌افزار کاملاً منعطف^۲ تبدیل شود. چه بخواهیم مدار را برای $M=1$ سنتز کنیم و چه برای $M=4$ ، ابزار سنتز بدون نیاز به حتی یک خط تغییر در کدهای منطقی، سخت‌افزار متناسب را تولید خواهد کرد.

¹ Nested Loop

² Flexible Hardware

۴-۷) تحلیل عملیات تغییر رفتار سائز سیستم و وابسته شدن چرخه‌های زمان به مقدار ورودی پارامتر m

در طراحی معماری‌های پارامتریک و مقیاس‌پذیر، تغییر ابعاد فیزیکی مدار (مانند تعداد سطرها و ستون‌های یک آرایه سیستمولیک) تنها به تغییرات فضایی و مصرف منابع سخت‌افزاری محدود نمی‌شود؛ بلکه یکی از چالش‌برانگیزترین موضوعات این نوع طراحی، مدیریت زمان‌بندی^۱ و وابستگی مستقیم چرخه‌های کنترلی به متغیرهای ابعادی است. در این پروژه، با تغییر مقدار پارامتر ورودی m ، رفتار زمانی کل سیستم به صورت کاملاً پویا و خودکار تغییر می‌کند و ماشین حالت متناهی (FSM) خود را با شرایط جدید وفق می‌دهد.

هسته اصلی این تطبیق‌پذیری زمانی در ماژول کنترل‌کننده پیاده‌سازی شده است. برخلاف طراحی‌های مبتنی بر ابعاد ثابت^۲ که در آن‌ها زمان ماندگاری در هر وضعیت با اعداد ثابت مشخص می‌شود، در این کنترل‌کننده تمامی زمان‌ها به صورت توابع ریاضیاتی وابسته به متغیرهای m و n توصیف شده‌اند. به عنوان مثال، در فاز محاسبات (COMPUTE)، ماشین حالت باید تا زمانی که تمامی داده‌های ورودی به درون شبکه تزریق شده و تا آخرین هسته پردازشی جریان یابند، در این وضعیت باقی بماند. تعداد چرخه‌های لازم برای تزریق داده‌ها^۳ وابستگی مستقیمی به طول مسیر (یعنی پارامتر m) دارد. رابطه ریاضی این وابستگی که دقیقاً در کدهای کنترل‌کننده نیز پیاده‌سازی شده است، در معادله

$$STREAM_CYCLES = m + N - 1$$

مشاهده می‌شود.

```
localparam [15:0] STREAM_CYCLES = M + N - 1;
localparam [15:0] DRAIN_CYCLES = N;
localparam [15:0] COMPUTE_TOTAL = STREAM_CYCLES + DRAIN_CYCLES;

localparam [15:0] SERIAL_WRITEBACK = M * N + 2;
```

شکل ۵-۷) پیاده‌سازی محاسبات پویا برای استخراج چرخه‌های زمانی در ماشین حالت بر اساس مقادیر پارامتریک

علاوه بر فاز محاسبات، فاز بازنویسی خروجی (WRITEBACK) نیز به شدت تحت تاثیر تغییرات پارامتر m قرار دارد. ماژول جمع‌آوری خروجی (output_collector) باید تمام نتایج تولید شده را به صورت سریال به پورت خروجی ارسال کند. تعداد کل این نتایج دقیقاً برابر با حاصل ضرب تعداد سطرها در ستون‌ها است. بنابراین، ماشین حالت باید بداند که چه زمانی عملیات ارسال پایان یافته است تا سیستم را به وضعیت اتمام کار (DONE) منتقل کند. این چرخه زمانی نیز به صورت یک رابطه پارامتری در منطق کنترل‌کننده تعبیه شده است که در معادله

$$SERIAL_WRITEBACK_CYCLES = (m \times N) + 1$$

نشان داده می‌شود.

وابسته شدن چرخه‌های زمان به پارامتر m ، مزیت بسیار بزرگی در تحلیل کارایی^۴ سیستم ایجاد می‌کند. به عنوان مثال، اگر طراح تصمیم بگیرد سیستم را از $m=4$ به حالت $m=1$ تغییر دهد، زمان تاخیر اولیه به شدت کاهش می‌یابد. در حالت $m=4$ ،

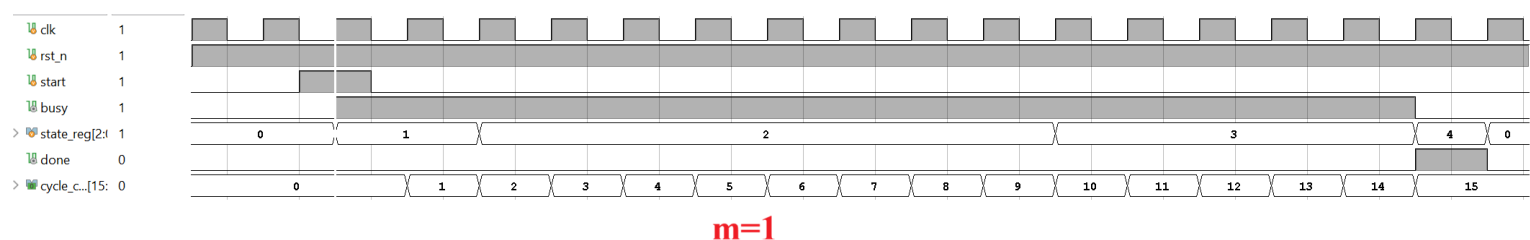
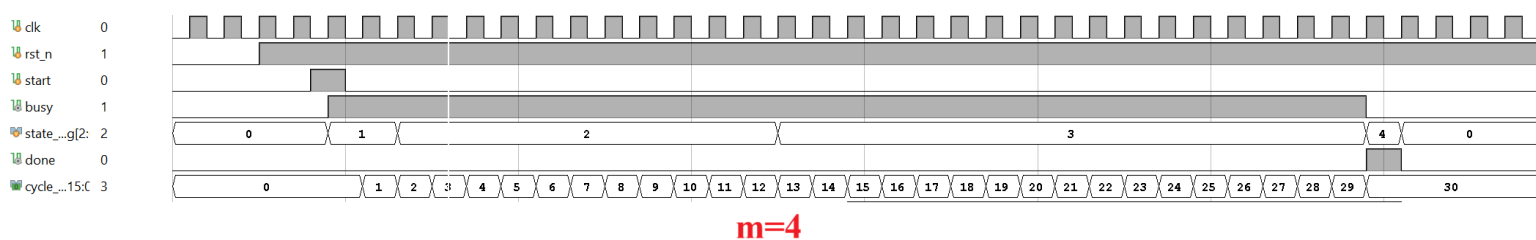
¹ Timing Management

² Hardcoded

³ Stream Cycles

⁴ Performance Analysis

چرخه‌های فاز محاسبات برابر با 11 پالس کلاک است، در حالی که با تغییر به $m=1$ ، این چرخه‌ها به صورت خودکار به 8 پالس کلاک کاهش می‌یابند. همچنین زمان استخراج داده‌ها از 17 سیکل به 5 سیکل تقلیل می‌یابد.



شکل ۶-۷) مقایسه رفتار زمانی سیستم؛ کاهش خودکار طول دوره‌های محاسباتی و بازنویسی با کاهش مقدار پارامتر

ماژول بارگذار ورودی (input_loader) نیز از این قاعده مستثنی نیست و در تولید سیگنال‌های اعتبار ردیف‌ها (a_row_valid) از پارامتر m در شروط حلقه‌های خود استفاده می‌کند تا در صورت کاهش ابعاد، پالس‌های اضافی تولید نکرده و توان مصرفی مدار را بیهوده افزایش ندهد.

۷-۵) چگونگی پیاده‌سازی مفهوم سیستم نورونی لایه اتصال کامل بوسیله مقدار دهی ($M = 1$)

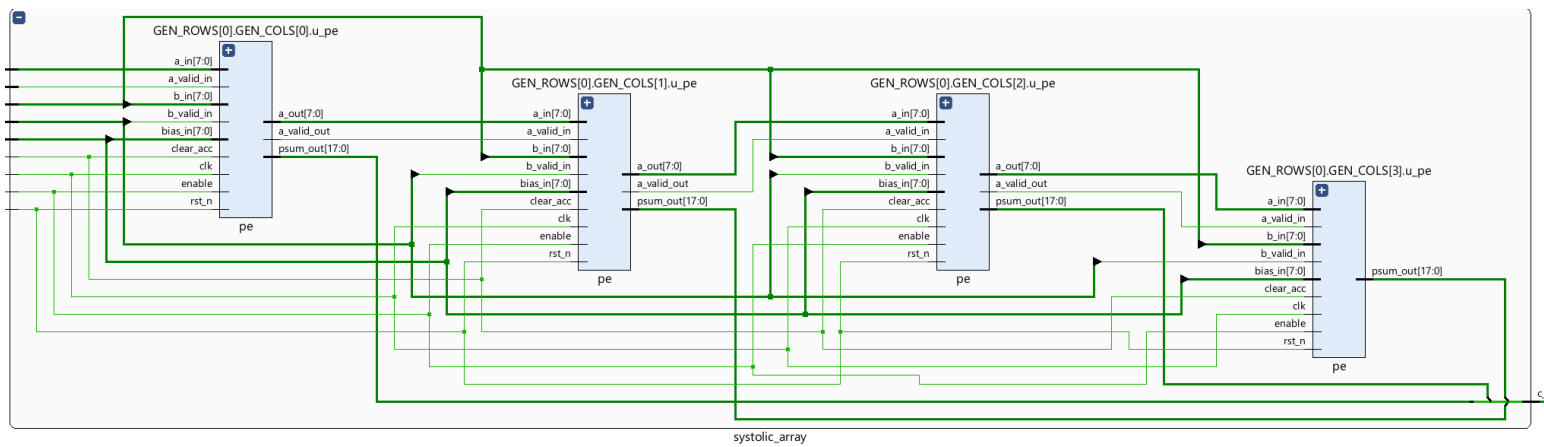
در معماری شبکه‌های عصبی عمیق، لایه‌های اتصال کامل یا تمام‌متصل^۱ نقش تصمیم‌گیرنده نهایی را ایفا می‌کنند. عملیات ریاضی در این لایه‌ها، برخلاف لایه‌های کانولوشنال که مبتنی بر فیلترهای محلی هستند، بر پایه ضرب یک بردار ورودی یک‌بعدی در یک ماتریس وزن دوبعدی و در نهایت جمع با یک مقدار بایاس استوار است. یکی از برجسته‌ترین دستاوردهای معماری پارامتریک در این پروژه، قابلیت تبدیل یک شتاب‌دهنده ضرب ماتریسی (که ذاتاً برای پردازش‌های دوبعدی طراحی شده) به یک موتور پردازش برداری برای لایه‌های اتصال کامل، تنها از طریق تغییر یک پارامتر ساده است.

با قرار دادن مقدار پارامتر ورودی به صورت ($M = 1$) در بالاترین سطح ماژول، ماهیت فضایی آرایه سیستمولیک به طور کامل دگرگون می‌شود. در این حالت، بلوک‌های تولید کد^۲ به جای ساخت یک شبکه مربعی متقاطع (مثلاً 4 در 4)، تنها یک سطر واحد متشکل از واحدهای پردازشی (PE) را در سطح سخت‌افزار نمونه‌سازی^۳ می‌کنند. این تغییر ساختاری باعث می‌شود که آرایه دوبعدی به یک آرایه خطی و یک‌بعدی تقلیل یابد.

^۱ Fully Connected Layers

^۲ generate blocks

^۳ Instantiate



شکل ۷-۷) شماتیک به ازای پارامتر $M=1$ ، نشان‌دهنده تقلیل آرایه دوبعدی به یک ساختار خطی جهت پردازش برداری نوروها

از منظر جریان داده^۱ و مفاهیم ریاضی شبکه عصبی، در حالت ($M = 1$)، ماتریس ورودی A دیگر یک ماتریس نیست، بلکه به یک بردار خطی^۲ تبدیل می‌شود که نمایانگر ویژگی‌های استخراج‌شده^۳ یا مقادیر فعال‌سازی^۴ لایه قبل است. ماتریس B نیز در این حالت نقش ماتریس وزن‌های سیناپسی^۵ را ایفا می‌کند و ماتریس بایاس تنها شامل یک ردیف از مقادیر ثابت برای تنظیم آستانه تحریک نوروها خواهد بود.

هنگامی که این بردار ورودی وارد تنها سطر موجود در آرایه سیستمولیک می‌شود، در طول واحدهای پردازشی حرکت کرده و در هر گام، با وزن اختصاصی خود ضرب و در انباشت‌گرها^۶ جمع می‌گردد. این عملیات دقیقاً معادل با عملگر ضرب نقطه‌ای^۷ در ریاضیات است که مشابه محاسبات نورو محسوب می‌شود. رابطه ریاضی این نگاشت سخت‌افزاری به مدل نورو عصبی، که در آن مقادیر ورودی در وزن‌ها ضرب شده و با بایاس جمع می‌شوند، در معادله

$$y_{\text{neuron}} = \left(\sum_{i=1}^N x_i \cdot w_i \right) + \text{bias}$$

نشان داده شده است.

زمانی که ابزار سنتز کدهای ورپلاگ را با پارامتر ($M = 1$) تفسیر می‌کند، تمامی گیت‌های منطقی، ضرب‌کننده‌های Wallace Tree و جمع‌کننده‌های CLA که مربوط به سطرهای دوم تا چهارم بودند را به عنوان منطق غیرقابل دسترس^۸ یا منابع اضافی حذف می‌کند. بنابراین، بدون نیاز به بازنویسی حتی یک خط از کدهای توصیف سخت‌افزار، به یک IP Core بهینه‌سازی شده برای دستگاه‌های اینترنت اشیا و Edge AI دست می‌یابیم که به منابع پردازشی محدودی نیاز دارند.

علاوه بر این، همان‌طور که در بخش‌های پیشین تحلیل شد، ماشین حالت سیستم نیز زمان‌بندی خود را با این حالت جدید تطبیق می‌دهد. با کاهش تعداد سطرها به یک، تاخیر انتشار^۹ داده‌ها در مسیر عمودی حذف شده و زمان تاخیر اولیه سیستم به طرز چشمگیری کاهش می‌یابد.

¹ Data Flow

² Vector

³ Features

⁴ Activations

⁵ Synaptic Weights

⁶ Accumulators

⁷ Dot Product

⁸ Unreachable Logic

⁹ Propagation Delay

فصل هشتم: تحلیل شبیه‌سازی موجی و Testbench

۸-۱ معرفی فایل Testbench و بردارهای تست اعمالی

برای اثبات صحت عملکرد شتاب‌دهنده سخت‌افزاری طراحی شده و اطمینان از صحت محاسبات مربوط به واحد آرایه سیستمولیک، یک محیط شبیه‌سازی^۱ با نام `tb_systolic_top` توسعه داده شده است. در این محیط، ماژول سطح بالای سیستم یعنی `systolic_array_top` نمونه‌سازی شده و سیگنال‌های تحریک لازم به آن اعمال می‌گردند. ساختار کلی این محیط تست بر پایه ایجاد یک جریان کنترلی خودکار استوار است که با تولید سیگنال کلاک، بازنشانی سیستم و سپس ارسال پالس شروع به کار (`start`)، عملیات سخت‌افزار را آغاز می‌کند. سیگنال کلاک تولید شده در این شبیه‌سازی دارای دوره تناوب ۱۰ نانوثانیه است که نمایانگر فرکانس کاری ۱۰۰ مگاهرتز است، می‌باشد.

روند اعتبارسنجی^۲ در این سیستم نیازمند مقایسه نتایج خروجی با مقادیر مورد انتظار است. بر اساس مستندات طراحی و شرایط تعیین شده، معماری باید بتواند عملگر ریاضی جمع و ضرب ماتریسی به همراه بایاس را مطابق رابطه $C = (A \times B) + Bias$ پیاده‌سازی کند. داده‌های خام مربوط به ورودی‌ها به صورت اعداد ۸ بیتی علامت‌دار و با فرمت هگزادسیمال در قالب دو فایل خارجی با نام‌های `IN.txt` و `WB.txt` ذخیره شده‌اند. این داده‌ها توسط بلوک لود کننده ورودی به داخل حافظه‌های سیستم در هنگام شروع به کار سیستم هدایت می‌شوند. ماتریس‌ها بر اساس رویکرد ترتیب سطری نگاشت شده و در ساختار پارامتری سیستم با ابعاد ۴ در ۴ مقداردهی می‌گردند.

```
module tb_systolic_top;

    reg clk;
    reg rst_n;
    reg start;

    wire busy;
    wire done;
    wire out_valid;
    wire [3:0] out_addr;
    wire signed [17:0] out_data;

    reg signed [17:0] expected_matrix [0:15];
    integer correct_tests;

    systolic_array_top u_sys_top (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),

        .busy(busy),
        .done(done),
        .out_valid(out_valid),
        .out_addr(out_addr),
        .out_data(out_data)
    );
```

شکل ۸-۱ تعریف متغیرها و مقادیر صحیح اعتبارسنجی در فایل تست‌بنج

^۱ Testbench

^۲ Validation

به منظور سنجش دقیق مقادیر پردازش شده توسط آرایه سیستمولیک، یک آرایه یک‌بعدی با نام expected_matrix به طول ۱۶ در نظر گرفته شده است. درون این آرایه، نتایج از پیش محاسبه شده‌ی ضرب ماتریس‌ها که مطابق سناریوی تست است، به صورت مقادیر ۱۸ بیتی علامت‌دار و در قالب ده‌دهی^۱ تعریف شده‌اند تا امکان بررسی ساده‌تر دقت سیستم و مقایسه با محدوده‌های محاسباتی خواسته شده فراهم شود.

سناریوی اعمال بردارهای تست به این صورت است که ابتدا پس از طی شدن ۲۵ واحد زمانی، مدار از حالت ریست خارج شده و با گذشت ۱۵ واحد زمانی دیگر، پالس start برای تحریک کنترل‌کننده مرکزی صادر می‌شود. محیط تست پس از شروع، وارد یک حالت توقف می‌شود و منتظر می‌ماند کنترل‌گر، پردازش خود را روی بردارهای تست تمام کرده و سیگنال اتمام عملیات (done) را صادر کند.

پس از پایان پردازش، مدار تست‌بنچ از طریق بررسی متوالی وضعیت سیگنال معتبر بودن خروجی (out_valid)، اقدام به خواندن نتایج از پورت‌های خروجی می‌کند. در هر کلاک که دیتای خروجی معتبر ارزیابی شود، مقدار قرار گرفته روی گذرگاه خروجی دیتا (out_data) توسط اندیس دریافت شده از گذرگاه آدرس (out_addr)، با داده‌های از پیش محاسبه شده در expected_matrix مورد ارزیابی و تطبیق قرار می‌گیرد. چنانچه داده محاسبه شده توسط واحد سخت‌افزاری کاملاً برابر با تئوری باشد، پیغام موفقیت برای آن آدرس به خصوص چاپ می‌شود. در انتهای تست اگر هر ۱۶ دیتای محاسبه شده با مقادیر مورد انتظار تطابق کامل داشته باشند، صحت عملکرد شتاب‌دهنده مورد تأیید نهایی قرار می‌گیرد.

```
VSIM 10> run -all
#-----
#   Systolic Mesh HW Initialization! Firing System FSM
#-----
# Clock Output Check Index Address[ 0]:
# - Computed HW Systolic Output Matrix Value :    1265
# - Exact Mathematical Document Value       :    1265
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 1]:
# - Computed HW Systolic Output Matrix Value :   -1279
# - Exact Mathematical Document Value       :   -1279
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 2]:
# - Computed HW Systolic Output Matrix Value :     927
# - Exact Mathematical Document Value       :     927
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 3]:
# - Computed HW Systolic Output Matrix Value :   -1912
# - Exact Mathematical Document Value       :   -1912
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 4]:
# - Computed HW Systolic Output Matrix Value :    -854
# - Exact Mathematical Document Value       :    -854
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 5]:
# - Computed HW Systolic Output Matrix Value :     721
# - Exact Mathematical Document Value       :     721
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 6]:
# - Computed HW Systolic Output Matrix Value :   -1114
# - Exact Mathematical Document Value       :   -1114
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 7]:
# - Computed HW Systolic Output Matrix Value :    1659
# - Exact Mathematical Document Value       :    1659
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 8]:
# - Computed HW Systolic Output Matrix Value :    1630
# - Exact Mathematical Document Value       :    1630
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 9]:
# - Computed HW Systolic Output Matrix Value :   -1193
# - Exact Mathematical Document Value       :   -1193
# -->> Verdict : OK
#
# Clock Output Check Index Address[10]:
# - Computed HW Systolic Output Matrix Value :    2698
# - Exact Mathematical Document Value       :    2698
# -->> Verdict : OK
#
# Clock Output Check Index Address[11]:
# - Computed HW Systolic Output Matrix Value :   -3303
# - Exact Mathematical Document Value       :   -3303
# -->> Verdict : OK
#
# Clock Output Check Index Address[12]:
# - Computed HW Systolic Output Matrix Value :   -1369
# - Exact Mathematical Document Value       :   -1369
# -->> Verdict : OK
#
# Clock Output Check Index Address[13]:
# - Computed HW Systolic Output Matrix Value :    1455
# - Exact Mathematical Document Value       :    1455
# -->> Verdict : OK
#
# Clock Output Check Index Address[14]:
# - Computed HW Systolic Output Matrix Value :   -1202
# - Exact Mathematical Document Value       :   -1202
# -->> Verdict : OK
#
# Clock Output Check Index Address[15]:
# - Computed HW Systolic Output Matrix Value :    2002
# - Exact Mathematical Document Value       :    2002
# -->> Verdict : OK
#
# All 16 results completed properly inside System Core Output Reg Logic bounds Limits.
# Matched Tests Rate Result Finalized Evaluation : [      16 / 16 PASSED].
# -> FANTASTIC ! MAC PE Arrays & Bias loading passed flawlessly accurately bounds.
# ** Note: $finish : C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v(80)
# Time: 365 ns Iteration: 0 Instance: /tb_systolic_top
# 1
# Break in Module tb_systolic_top at C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v line 80
```

Test 1

¹ Decimal

```

VSIM 36> run -all
#-----
#   Systolic Mesh HW Initialization! Firing System FSM
#-----
#
# Clock Output Check Index Address[ 0]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 1]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 2]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 3]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 4]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 5]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 6]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 7]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 8]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 9]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[10]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[11]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[12]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# Clock Output Check Index Address[13]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 122144
# -->> Verdict : ** FAILED HW VALUE CHECK ! **
#
# Clock Output Check Index Address[15]:
# - Computed HW Systolic Output Matrix Value : 64643
# - Exact Mathematical Document Value : 64643
# -->> Verdict : OK
#
# All 16 results completed properly inside System Core Output Reg Logic bounds Limits.
# Matched Tests Rate Result Finalized Evaluation : [ 15 / 16 PASSED].
# -> Errors mapped dynamically outputting. Requires re-sync mapping parameters validation logic check !
# ** Note: $finish : C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v(80)
# Time: 365 ns Iteration: 0 Instance: /tb_systolic_top
# 1
# Break in Module tb_systolic_top at C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v line 80

```

Test 2

```

VSIM 47> run -all
#-----
#   Systolic Mesh HW Initialization! Firing System FSM
#-----
#
# Clock Output Check Index Address[ 0]:
# - Computed HW Systolic Output Matrix Value : 0
# - Exact Mathematical Document Value : 0
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 1]:
# - Computed HW Systolic Output Matrix Value : 0
# - Exact Mathematical Document Value : 0
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 2]:
# - Computed HW Systolic Output Matrix Value : 0
# - Exact Mathematical Document Value : 0
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 3]:
# - Computed HW Systolic Output Matrix Value : 0
# - Exact Mathematical Document Value : 0
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 4]:
# - Computed HW Systolic Output Matrix Value : 176
# - Exact Mathematical Document Value : 176
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 5]:
# - Computed HW Systolic Output Matrix Value : 1330
# - Exact Mathematical Document Value : 1330
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 6]:
# - Computed HW Systolic Output Matrix Value : 800
# - Exact Mathematical Document Value : 800
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 7]:
# - Computed HW Systolic Output Matrix Value : 641
# - Exact Mathematical Document Value : 641
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 8]:
# - Computed HW Systolic Output Matrix Value : 248
# - Exact Mathematical Document Value : 248
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 9]:
# - Computed HW Systolic Output Matrix Value : 1804
# - Exact Mathematical Document Value : 1804
# -->> Verdict : OK
#
# Clock Output Check Index Address[10]:
# - Computed HW Systolic Output Matrix Value : 880
# - Exact Mathematical Document Value : 880
# -->> Verdict : OK
#
# Clock Output Check Index Address[11]:
# - Computed HW Systolic Output Matrix Value : 1129
# - Exact Mathematical Document Value : 1129
# -->> Verdict : OK
#
# Clock Output Check Index Address[12]:
# - Computed HW Systolic Output Matrix Value : 241
# - Exact Mathematical Document Value : 241
# -->> Verdict : OK
#
# Clock Output Check Index Address[13]:
# - Computed HW Systolic Output Matrix Value : 1267
# - Exact Mathematical Document Value : 1267
# -->> Verdict : OK
#
# Clock Output Check Index Address[14]:
# - Computed HW Systolic Output Matrix Value : 546
# - Exact Mathematical Document Value : 546
# -->> Verdict : OK
#
# Clock Output Check Index Address[15]:
# - Computed HW Systolic Output Matrix Value : 851
# - Exact Mathematical Document Value : 851
# -->> Verdict : OK
#
# All 16 results completed properly inside System Core Output Reg Logic bounds Limits.
# Matched Tests Rate Result Finalized Evaluation : [ 16 / 16 PASSED].
# -> FANTASTIC ! MAC PE Arrays & Bias loading passed flawlessly accurately bounds.
# ** Note: $finish : C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v(80)
# Time: 365 ns Iteration: 0 Instance: /tb_systolic_top
# 1
# Break in Module tb_systolic_top at C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v line 80
VSIM 48>

```

Test 3

```

/SIM 72> run -all
-----
Systolic Mesh HW Initialization! Firing System FSM
-----
# Clock Output Check Index Address[ 0]:
# - Computed HW Systolic Output Matrix Value : -2020
# - Exact Mathematical Document Value : -2020
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 1]:
# - Computed HW Systolic Output Matrix Value : 2152
# - Exact Mathematical Document Value : 2152
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 2]:
# - Computed HW Systolic Output Matrix Value : -2284
# - Exact Mathematical Document Value : -2284
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 3]:
# - Computed HW Systolic Output Matrix Value : 2416
# - Exact Mathematical Document Value : 2416
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 4]:
# - Computed HW Systolic Output Matrix Value : 4227
# - Exact Mathematical Document Value : 4227
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 5]:
# - Computed HW Systolic Output Matrix Value : -4428
# - Exact Mathematical Document Value : -4428
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 6]:
# - Computed HW Systolic Output Matrix Value : 4629
# - Exact Mathematical Document Value : 4629
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 7]:
# - Computed HW Systolic Output Matrix Value : -4830
# - Exact Mathematical Document Value : -4830
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 8]:
# - Computed HW Systolic Output Matrix Value : -5779
# - Exact Mathematical Document Value : -5779
# -->> Verdict : OK
#
# Clock Output Check Index Address[ 9]:
# - Computed HW Systolic Output Matrix Value : 6016
# - Exact Mathematical Document Value : 6016
# -->> Verdict : OK
#
# Clock Output Check Index Address[10]:
# - Computed HW Systolic Output Matrix Value : -6253
# - Exact Mathematical Document Value : -6253
# -->> Verdict : OK
#
# Clock Output Check Index Address[11]:
# - Computed HW Systolic Output Matrix Value : 6490
# - Exact Mathematical Document Value : 6490
# -->> Verdict : OK
#
# Clock Output Check Index Address[12]:
# - Computed HW Systolic Output Matrix Value : 2434
# - Exact Mathematical Document Value : 2434
# -->> Verdict : OK
#
# Clock Output Check Index Address[13]:
# - Computed HW Systolic Output Matrix Value : -2536
# - Exact Mathematical Document Value : -2536
# -->> Verdict : OK
#
# Clock Output Check Index Address[14]:
# - Computed HW Systolic Output Matrix Value : 2638
# - Exact Mathematical Document Value : 2638
# -->> Verdict : OK
#
# Clock Output Check Index Address[15]:
# - Computed HW Systolic Output Matrix Value : -2740
# - Exact Mathematical Document Value : -2740
# -->> Verdict : OK
#
# All 16 results completed properly inside System Core Output Reg Logic bounds Limits.
# Matched Tests Rate Result Finalized Evaluation : [ 16 / 16 PASSED].
# -> FANTASTIC ! MAC PE Arrays & Bias loading passed flawlessly accurately bounds.
# ** Note: $finish : C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v(80)
# Time: 365 ns Iteration: 0 Instance: /tb_systolic_top
# 1
# Break in Module tb_systolic_top at C:/Users/arshi/OneDrive/Desktop/VLSI_Questa/tb_systolic_top.v line 80
/SIM 73>

```

Test 5

شکل ۲-۸) گزارش لاگ نرم افزار از اجرای موفق محیط تست و چاپ خروجی نهایی پنج مورد اعتبارسنجی

C:\Users\arshi\OneDrive\Desktop\WB.txt		C:\Users\arshi\OneDrive\Desktop\IN.txt	
1	F1	1	7F
2	10	2	FF
3	EF	3	02
4	12	4	FD
5	13	5	FC
6	EC	6	5F
7	15	7	A0
8	EA	8	07
9	E9	9	08
10	18	10	F7
11	E7	11	6E
12	1A	12	91
13	1B	13	F4
14	E4	14	0D
15	1D	15	F2
16	E2	16	40
17	1F		
18	E0		
19	21		
20	DE		
21	DD		
22	24		
23	DB		
24	26		
25	27		
26	D8		
27	29		
28	D6		
29	D5		
30	2C		
31	D3		
32	2E		

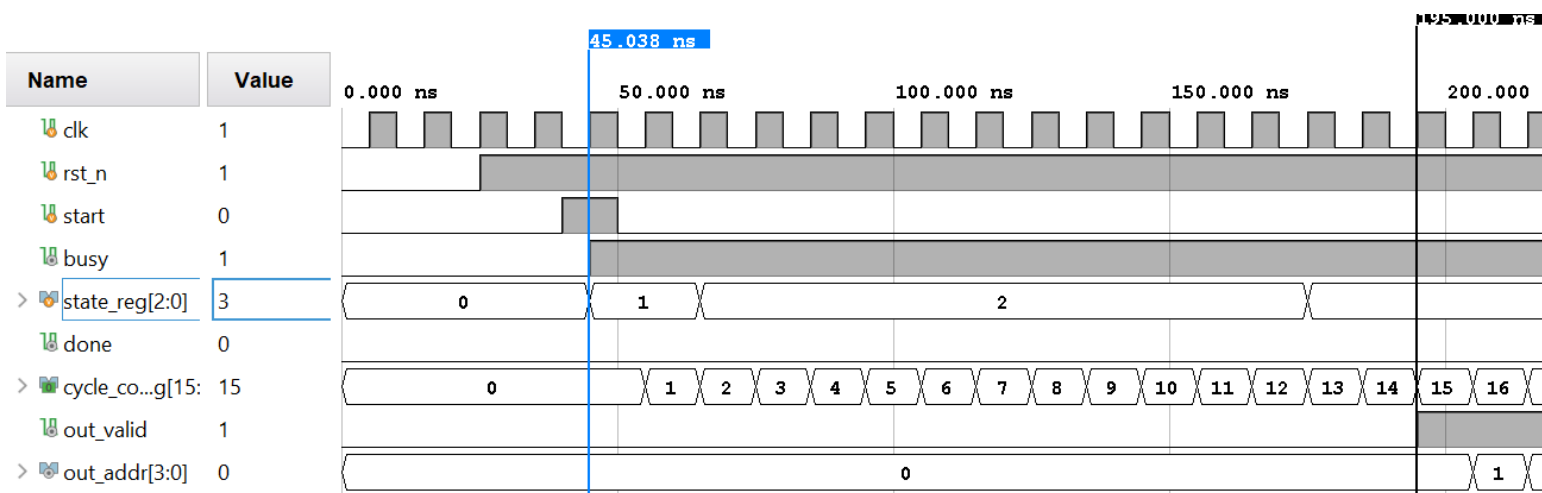
شکل ۳-۸) نمونه داده های ذخیره شده به فرمت هگزادسیمال برای مقداردهی حافظه های سیستم

۸-۲) گزارش تأخیر سیستم (Latency) و توان عملیاتی (Throughput)

گزارش کردن دقیق مقادیر تأخیر و توان عملیاتی مطابق تعریف پروژه، میزان کارایی پیاده‌سازی و توانایی خطوط انتقال (پایپ‌لاین) سیستم طراحی شده را مشخص می‌کند.

همان‌گونه که در مباحث نظری مربوط به شبکه‌های سیستمولیک تعریف گردیده است، تأخیر پردازشی برابر است با تعداد سیکل‌های پالس کلاکی که از آغاز دستور و لحظه‌ی ارسال پالس start، تا دریافت اولین عنصر خروجی معتبر زمان می‌برد. مطابق تئوری بررسی شده در فصل ششم برای سنجش تأخیر داریم:

$$\text{Latency} \approx \underbrace{T_{LOAD}}_{2 \text{ cyc}} + \underbrace{((M + N - 1) + T_{drain})}_{\approx 11 \text{ cyc}} + \underbrace{T_{Latch}}_{2 \text{ cyc}} = 15 \text{ clock cycles}$$



شکل ۸-۴) اندازه‌گیری فیزیکی فاصله زمانی تأخیر سیستم با استفاده از پایش گر (Cursor) شبیه‌ساز پس از صدور دستور start

مورد بعدی که باید بدان پردازیم توان عملیاتی در بخش پایپ‌لاین است. این شاخص معادل میزان اطلاعات مفید یا درایه‌های معتبر ثبت‌شده طی چرخه‌ی نرمال کاری پس از پرشدن و اشباع فضای محاسبه‌گرها معرفی می‌شود. با توجه به بررسی‌های انجام شده و شکل موج‌های اجرای پروژه در بخش‌های ۴-۶ و ۵-۶، برای توان عملیاتی داریم:

$$\text{Throughput(Readout)} = \frac{1 \text{ valid data (18-bit)}}{1 \text{ clock cycle}} \text{ (پس از پایپ‌لاین)}$$

- [1] H. T. Kung, "Why systolic architectures?," *Computer*, vol. 15, no. 1, pp. 37-46, Jan. 1982.
- [2] P. Jaswal, L. H. Krishna, and B. Srinivasu, "Energy Efficient Exact and Approximate Systolic Array Architecture for Matrix Multiplication," *arXiv preprint arXiv:2509.00778*, 2026.